

Implementation of RISC-V P-extension ALU

Haris Reyaz Sultanpuri

School of Electrical Engineering

Thesis submitted for examination for the degree of Master of
Science in Technology.

Espoo 07.07.2025

Supervisor

Prof. Marko Kosunen

Advisor

M.Sc Aleksi Korsman

Copyright © 2025 Haris Reyaz Sultanpuri



Author Haris Reyaz Sultanpuri

Title Implementation of RISC-V P-extension ALU

Degree programme Electronics and Nanotechnology

Major Microelectronic Circuit Design

Code of major ELEC3036

Supervisor Prof. Marko Kosunen

Advisor M.Sc Aleksi Korsman

Date 07.07.2025

Number of pages 73

Language English

Abstract

Instruction Set Architectures (ISAs) are foundational to modern computing systems, with dominant players like Intel's x86 and ARM which power most of the commercial devices available in the market. However, their proprietary nature imposes significant barriers, including high licensing costs and limited accessibility for small-scale innovators and academic institutions. Addressing these limitations, RISC-V, an open-source and royalty-free ISA, has emerged as a flexible and modular alternative. Among its many extensions, the P extension is designed to cater to Digital Signal Processing(DSP) applications which enables efficient data-parallel computations such as audio, image, and voice processing while maintaining area efficiency.

This thesis presents the implementation and integration of the RISC-V P-extension into Aalto University's in-house RISC-V processor, A-Core. The P-Extension ALU was developed in Chisel and employed a micro-architectural design that reused a single 4×9 -bit adder unit across multiple 8-bit and 16-bit arithmetic and logical operations, thereby minimising area overhead. Initial verification was conducted using ChiselTest. The new ALU and its control paths were then integrated into A-Core's existing five-stage pipeline. On the software side, the binutils component of the RISC-V toolchain was patched to correctly assemble and disassemble P-extension mnemonics, enabling the use of P-extension instructions within C programs via inline assembly.

Performance was evaluated through simulations using four kernels representative of both synthetic workloads and practical DSP applications: dot product, matrix multiplication, a 32-tap fixed-point FIR filter, and the Sobel edge detection algorithm. Compared to their scalar counterparts, the Single Instruction Multiple Data (SIMD) kernels leveraging the P-extension consistently exhibited reduced CPU cycle counts and fewer retired instructions. However, for smaller workloads, the performance gains were constrained by the overhead associated with packing data into SIMD registers and by the inherent limits of parallelism imposed by the fixed SIMD register width and supported element sizes.

This work not only demonstrated the potential of the P-extension to effectively address DSP challenges, but also contributed to the open-source hardware ecosystem by advancing accessible computing solutions and enabling future enhancements of the A-Core processor. Furthermore, it served as a practical validation of the draft P-extension, supporting its progression toward ratification as an official part of the RISC-V standard.

Keywords Instruction Set Architecture, RISC-V, P Extension, Digital Signal Processing, packed SIMD, CHISEL

Preface

In the name of Allah, the Most Gracious, the Most Merciful.

All praise is for Allah, who allows me to experience the wonderful opportunities and countless blessings that I benefit from and grants me the strength and courage to overcome the challenges that I face in life.

I would like to use this space to express my sheer and sincere gratitude to those who have been a part of my master's thesis journey. First of all, I am deeply thankful to my supervisor Marko Kosunen. He extended his support above and beyond the role of an academic supervisor. Thank you Marko for your patience, trust, encouragement and being accommodating at times when I needed it the most.

Secondly, to my incredibly talented advisor Aleksi Korsman. He has been like a wizard in this journey, with insightful answers and a source of precise guidance whenever I encountered any challenges. Thank you Aleksi for all the detailed responses on Zulip, discussions during our meetings, feedback that you provided and for being a thoughtful and supportive advisor.

Thirdly, to my loving and supportive parents. All the gratitude and some more that I could ever express can never fully suffice for the unconditional love and support that you have given me. Thank you Abuji and Amijan. To my loving and incredible brother, who relentlessly supported my master's studies in Finland. To my loving and funny sister, with whom I shared pearls of laughter and lightened some heavy days. To my loving and composed wife, whose emotional support helped me stay firm and motivated during this phase of my life.

Lastly, my sincere gratitude goes to the amazing Aalto University. The support from the staff, the incredible study spaces, and the abundance of academic and technical resources provided throughout this journey have been truly unmatched. I am deeply grateful to have had the opportunity of being a part of Aalto University.

Otakaari 1, 07.07.2025

Haris Reyaz Sultanpuri

Contents

Abstract	iii
Preface	iv
Contents	v
Symbols and abbreviations	vii
1 Introduction	1
1.1 Scope and structure	2
2 Background	4
2.1 Instruction Set Architecture (ISA)	4
2.2 RISC, CISC and the origin of RISC-V	5
2.3 RISC-V ISA	6
2.4 Instruction Encoding in RISC-V	8
2.5 SIMD and RISC-V ISA	9
2.5.1 P-Extension and V-Extension	11
2.6 RISC-V P-Extension	12
2.7 P-Extension Specification	13
2.8 Overview of A-Core	19
2.9 Agile hardware methodologies and origin of CHISEL	22
2.9.1 Advantages over traditional HDLs	23
2.9.2 Chisel Compilation Pipeline	24
2.10 Related Works	25
3 Implementation	27
3.1 Design considerations for P-Extension ALU	27
3.1.1 Single 4x9-bit adder unit	28
3.1.2 Two's Complement generation	31
3.1.3 Compare and Minimum/Maximum	31
3.1.4 Absolute	32
3.1.5 Clip	32
3.1.6 Multiply-Accumulate	33
3.1.7 Control and Status Register	34
3.2 Initial testing using Chiseltest	34
3.3 Integration with A-Core	36
3.4 Compiler support for P-Extension instructions	44
3.5 Validation testing and assembly code generation	48
4 Evaluation and Results	51
4.1 Dot Product	51
4.2 Matrix Multiplication	53
4.3 FIR filter	57

4.4 Sobel Edge Detection algorithm	61
5 Conclusion	65
References	67
Appendix A	73

Symbols and abbreviations

Abbreviations

AI	Artificial Intelligence
ALU	Arithmetic and Logic Unit
ASIC	Application-Specific Integrated Circuit
AVX	Advanced Vector Extensions
CHISEL	Constructing Hardware in a Scala Embedded Language
CISC	Complex Instruction Set Computer
CMO	Cache Management Operations
CMOS	Complementary Metal-Oxide-Semiconductor
CPU	Central Processing Unit
CSR	Control and Status Register
DLP	Data-Level Parallelism
DSP	Digital Signal Processing
EX	Execute
FIR	Finite Impulse Response
FIRRTL	Flexible Intermediate Representation for RTL
FPGA	Field-Programmable Gate Array
FPU	Floating Point Unit
GPIO	General Purpose Input/Output
GPR	General Purpose Register
GPUs	Graphics Processing Units
HDL	Hardware Description Language
ID	Instruction Decode
IF	Instruction Fetch
IoT	Internet of Things

IP	Intellectual Property
IR	Intermediate Representation
ISA	Instruction Set Architecture
LLVM	Low Level Virtual Machine
LSU	Load-Store Unit
LUT	Lookup Table
MAC	Multiply-Accumulate
MEM	Memory Access
NRE	Non-Recurring Engineering
NT	Near-Threshold
OV	Overflow
PU	Processing Unit
PULP	Parallel Ultra-Low Power
RISC	Reduced Instruction Set Computer
SIMD	Single Instruction Multiple Data
SIMT	Single Instruction, Multiple Threads
SISD	Single Instruction Single Data
SoC	System-on-Chip
SSE	Streaming SIMD Extensions
TDR	Test Data Register
ULP	Ultralow-Power
VLSI	Very-Large-Scale Integration
WB	Write Back

Symbols

$\vec{a}$	n-element vector a
$\vec{b}$	n-element vector b
$h[k]$	FIR filter coefficient element

$x[n]$	FIR input sequence element
$y[n]$	FIR output sequence element
G_x	Horizontal Sobel gradient
G_y	Vertical Sobel gradient

1 Introduction

Instruction set architecture (ISA) represents a fundamental interface through which software interacts with the underlying hardware, forming the basis for operation in nearly all digital systems [1]. While proprietary ISAs like Intel’s x86 and ARM dominate the market, their closed licensing models can significantly inflate the development expenses by tens of millions of dollars and lead to substantial project delays. Although open standards like TCP/IP and Linux have repeatedly shown that transparency accelerates innovation, the ISA layer has largely remained closed [2]. Such proprietary status of mainstream ISAs is increasingly at odds with industry practice. Consequently, this imbalance spurred the development of RISC-V, an open-source instruction set architecture initiated at the University of California, Berkeley in 2010, and now overseen by a dedicated non-profit organization, RISC-V International [3].

The flexible and modular architecture of RISC-V, reinforced by its royalty-free licensing model, has driven widespread industry adoption, with its total economic output projected to exceed 11.5 billion USD by 2025. Leading technology companies including Google, NVIDIA, Qualcomm, and Intel have openly committed to supporting the RISC-V architecture [4]. Although RISC-V is reaching maturity in the field of embedded processors, it is still in its infancy in other areas such as cloud computing, high-performance computing [1]. With Artificial Intelligence (AI) becoming an indisputable reality, the embedded-systems landscape is shifting toward edge computing and edge AI more than ever, where billions of devices process data locally rather than in distant, centralised data-centres [5].

Performing computations locally significantly reduces latency and minimises the energy consumption associated with transmitting data over the Internet [6]. This efficiency becomes increasingly vital as the global Internet of Things (IoT) device base is projected to nearly double, reaching approximately 30 billion units by 2030 [7]. A recent study [6] underscores that the full potential of edge computing and edge AI is contingent upon continued progress in semiconductor innovation and processor performance enhancements. Traditionally, Single Instruction Multiple Data (SIMD) architectures have been employed to accelerate specific workloads in desktop systems and data centers. However, recently this performance-enhancing SIMD technology has also found its way into the microprocessors designed for embedded devices, for example, ARM’s DSP extension NEON and Helium extensions.

With the emergence of RISC-V, efforts to combine the open-source nature of ISAs with the energy-efficient advantages of SIMD have led to a few RISC-V extension proposals enabling SIMD execution. Among these, the P-extension [8] introduces packed-SIMD instructions, similar to those found in other instruction set architectures. At the time of writing this thesis, the P-Extension remains unratified and only a few designs have implemented it using existing RISC-V processors [9][10][11]. An in-house 32-bit RISC-V microprocessor called A-Core has been developed at Aalto University [12]. Under active development, A-Core currently supports any combination of the M, F, and C extensions.

The aim of this thesis is to implement and integrate the P-extension into the A-Core processor, and assess the performance improvements it offers in comparison to conventional sequential execution. The thesis uses Chisel 3.5.1 as the Hardware Description Language (HDL) for development. The P-Extension specification will be studied and a subset of P-Extension instructions will be identified for implementation. The P-Extension allows creating area-efficient DSP-oriented processors. In order to achieve this, the Arithmetic and Logic Unit (ALU) will be designed with a focus on hardware-reuse minimising the number of distinct modules required to support multiple instructions. Concurrently, the verification process will follow agile development practices by using Chiseltest, Chisel’s integrated testing framework, ensuring thorough and iterative validation of the design during the development. Integrating the P-Extension ALU with A-Core processor will be accomplished by modifying the relevant A-Core blocks and adding the P-Extension ALU such that processor can correctly recognize, decode, and execute P-extension instructions. Currently, there is no compiler support for P-Extension. The assembler and disassembler components will be modified such that the riscv-gnu-toolchain can reliably assemble and disassemble packed-SIMD instructions.

In order to benchmark the performance improvement of P-Extension, suitable DSP applications that can benefit from SIMD execution will be identified and studied. For each selected application, both baseline sequential C kernels and SIMD-optimised variants will be implemented. A detailed performance comparison will be carried out based on clock cycle and retired instruction metrics.

1.1 Scope and structure

The scope of this thesis will be limited to the documenting performance improvements obtained through simulations only. The synthesis results showcasing the additional hardware area usage and consequently the effects on critical path and clock frequency of the processor have been excluded from the scope of this thesis.

The thesis is organised as follows:

Chapter 2 provides the necessary background. It begins with a concise overview of instruction set architectures (Section 2.1) and the historical context of RISC and CISC machines leading to RISC-V (Section 2.2). Sections 2.3–2.6 introduce the RISC-V ISA, its encoding rules, and the role of SIMD within that ISA, culminating in a survey of the draft Packed-SIMD (P) extension instructions implemented in this thesis (Sections 2.7). The next section presents the baseline A-Core micro-architecture (Section 2.8) and details the role of agile hardware design methodologies leading to the origin of Chisel. Finally, Section 2.10 places this work in context by comparing related P-extension implementations in commercial (Andes D-series) and open-source (PULP, CVA6, Ibex) cores.

Chapter 3 describes the implementation of the P-extension on A-Core. Section 3.1 details design choices made for the ALU micro-architecture, including hardware reuse, a single 4×9 -bit adder, and datapaths for various processing units within

the ALU. Subsequent sections outline verification with ChiselTest (Section 3.2), the integration steps across A-Core’s pipeline stages (Sections 3.3), and the GNU tool-chain modifications required for software support (Section 3.4). Section 3.5 reports functional validation through self-checking assembly tests.

Chapter 4 presents quantitative results. Four representative DSP kernels: dot product (Section 4.1), blocked matrix multiplication (Section 4.2), fixed-point Finite Impulse Response (FIR) filtering (Section 4.3), and Sobel edge detection (Section 4.4), are implemented in both scalar and using P-extension instructions. Metrics based on cycle counts and retired-instruction counts are used to characterise the performance advantage of P-Extension over sequential execution.

Chapter 5 concludes the thesis, summarising the contributions and outlining directions for future work.

2 Background

This chapter presents the necessary background for the thesis. It begins with a brief overview of instruction-set architectures and the evolution from Complex Instruction Set Computer (CISC) to RISC-V. It then introduces the RISC-V ISA, with a focus on SIMD support and the draft Packed-SIMD (P) extension used in this work. The chapter also describes the A-Core baseline microarchitecture, the role of agile hardware methodologies, and the Chisel HDL. Finally, it surveys related P-extension implementations in both commercial and open-source RISC-V cores.

2.1 Instruction Set Architecture (ISA)

A fundamental aspect of computer architecture is the design and specification of the commands that a computer can interpret and execute [13]. In technical literature, this collection of commands is called Instruction Set. The conceptual design and the operational structure of an instruction set that define how these instructions function is referred to as the Instruction Set Architecture (ISA). The ISA encompasses various aspects, including the semantics of instructions, the types of operands used, supported data types & formats, hardware-level memory management, and the mechanisms through which the processor interfaces with peripheral devices [14, 15].

Essentially, the ISA provides a blueprint for how software can control hardware to perform computations. It has been defined as the language of computer and an interface between software and hardware [14, 16]. However, software compatibility with the underlying hardware has to be ensured. For example, Intel and AMD CPUs use the x86 ISA while as the ARM processors use the ARM ISA. As such, a binary compiled for Intel systems will not run on ARM-based systems because their instruction sets are not compatible.

With the technological advancements, ISAs can also evolve, for example by adding new instructions or by expanding support for larger addresses and data values [15, 17]. Moreover, as per the needs and requirements, ISA can be tailored to have only specific instructions [18, 14]. For example, a processor for string processing does not require the instructions that support operations for floating-point numbers. Such flexibility in customising the ISA is a significant aspect of RISC-V processors as discussed in Section 2.3.

In order to design an efficient ISA, it needs to have some necessary and desirable properties. The only necessary property is that it must be *complete* [14]. This implies that an ISA should be able to generate machine code for any program written for that specific processor. For example, if the processor is designed to handle basic math operations, the ISA should include instructions for addition, subtraction, multiplication, and division. However, it does not need to cover every single type of operation, like complex loop structures, to be considered as complete. The base RISC-V ISA, for instance, comprises only 40 instructions [19], yet it is as a complete ISA.

The remaining properties fall under *desirable* traits, including generality, con-

ciseness, and simplicity [14]. Simple instructions allow processors to execute tasks swiftly and efficiently, even if this requires breaking down complex operations into a sequence of smaller steps. These streamlined instructions are often executed more quickly than complex, multi-functional ones. Although the difference in complexity is evident, comparing the overall performance of different different architectures remains challenging. However, research shows that ARM-based Central Processing Units (CPUs), which use a greater number of simpler instructions, are generally more power-efficient than x86 CPUs, which use fewer but more complex instructions [20].

Such a distinction in complexity has, in fact, led to the classification of ISAs into two main types: Reduced Instruction Set Computer (RISC) and Complex Instruction Set Computer (CISC). RISC-V’s design principles are rooted in the RISC philosophy, emphasising simplicity, regularity, and efficiency [19].

2.2 RISC, CISC and the origin of RISC-V

RISC and CISC are the two primary types of processor architectures that have dominated the field. The key difference between these two lies in their approach to instruction execution. RISC’s simplicity allows for faster instruction execution and efficient pipelining, making it suitable for applications where performance and energy efficiency are critical, such as in embedded systems and mobile devices[21, 22]. In contrast, CISC’s complexity can reduce the number of instructions per program but often results in slower execution per instruction due to the complexity of the decoding and execution stages [13, 23, 24].

The historical development of both these principles reveals a cyclic pattern in CPU design philosophies. Fig. 1 illustrates Apple’s transitions across major CPU architectures, highlighting the cyclic evolution between CISC and RISC philosophies. For instance, Apple transitioned from the CISC-based Motorola 68000 to the RISC-based PowerPC in 1994, later moving back to CISC with Intel x86 in 2005, and most recently adopting the RISC-based ARM architecture for its Apple Silicon in 2020 [25]. This shift underscores the continuing relevance of both architectures in the face of evolving technological demands and capabilities.

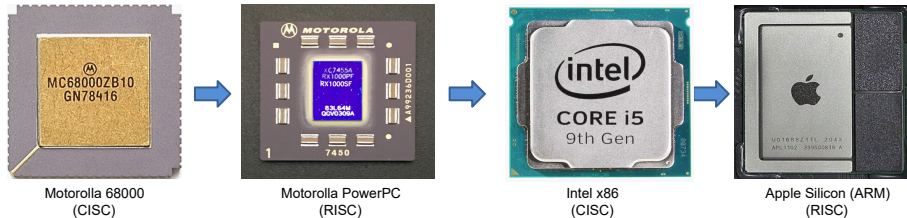


Figure 1: Apple chips embracing the two architectures over the generations.

Technological advancements have, in fact, led to a convergence of RISC and CISC features, blurring the distinctions between the two[23]. For example, research suggests that at performance levels in the range of ARM’s Cortex-A8 processor and higher, RISC vs CISC debate is irrelevant for performance, power, and energy. For

modern processors it is the micro-architecture and design methodology that really matters for performance differences [26].

Despite the popularity of CISC-based x86 ISA and RISC-based ARM processors, the development of RISC-V emerged from a perceived need for an alternative ISA. It is noteworthy that the creation of this new ISA was not driven by any significant technological gap in existing legacy ISAs. Instead the development of RISC-V was primarily motivated by two key factors. Firstly, the widely-used commercial ISAs are proprietary in nature [14]. Designing new processors even from scratch using these legacy proprietary ISAs came with numerous legal restrictions and licensing requirements. Moreover, it also created a barrier for the commercialisation of successful research ideas. Secondly, the popular commercial instruction sets are highly complex. This complexity made it challenging to fully implement them in hardware, even though research had shown that much simpler instruction sets can result in systems with comparable performance [18].

RISC-V started in 2010 at the University of California, Berkeley, when a group of researchers set out to design an instruction set architecture (ISA) that was simple, efficient, and flexible, mainly for educational and research purposes [18]. What began as a small project quickly grew and transformed into a widely recognised open standard around the world. At the time of writing this thesis, RISC-V boasts over 4,200 members from 70 countries and has been integrated into more than 13 billion cores in the market [27]. The name RISC-V was selected to signify the fifth major RISC ISA design developed at UC Berkeley, following the earlier designs of RISC-I, RISC-II, SOAR, and SPUR [28].

2.3 RISC-V ISA

Fig. 2 shows the classification of RISC-V ISA, split into privileged and non-privileged architecture. RISC-V provides privileged architecture for running operating systems and supporting hardware virtualisation. At any time RISC-V ISA must work in a defined privileged mode. These modes which basically represent different levels of hardware support needed to run an operating system are included in the privileged ISA. The different modes under privileged ISA are: Machine mode (M), Supervisor mode (S), User mode (U) [29, 30], as shown in Fig. 2. Systems support different privilege modes based on their software's complexity and security needs. Simple embedded devices running a trusted application may use only Machine (M) mode, while systems requiring isolation between applications and hardware often include both Machine (M) and User (U) modes.

The non-privileged ISA governs the base instruction set. As shown in Fig. 2, this is not just one instruction set, but rather a collection of various instruction sets, for example RV32, RV64 etc. These instruction sets are called ISA Base Versions or Cores. The developers extend the functionality of RISC-V ISA by choosing a base version first and then build on top of that core by adding extensions [14]. These extensions are categorised under the non-privileged ISA. Fig. 2 lists some ratified extensions as well as shows the placement of P-Extension in RISC-V ISA classification.

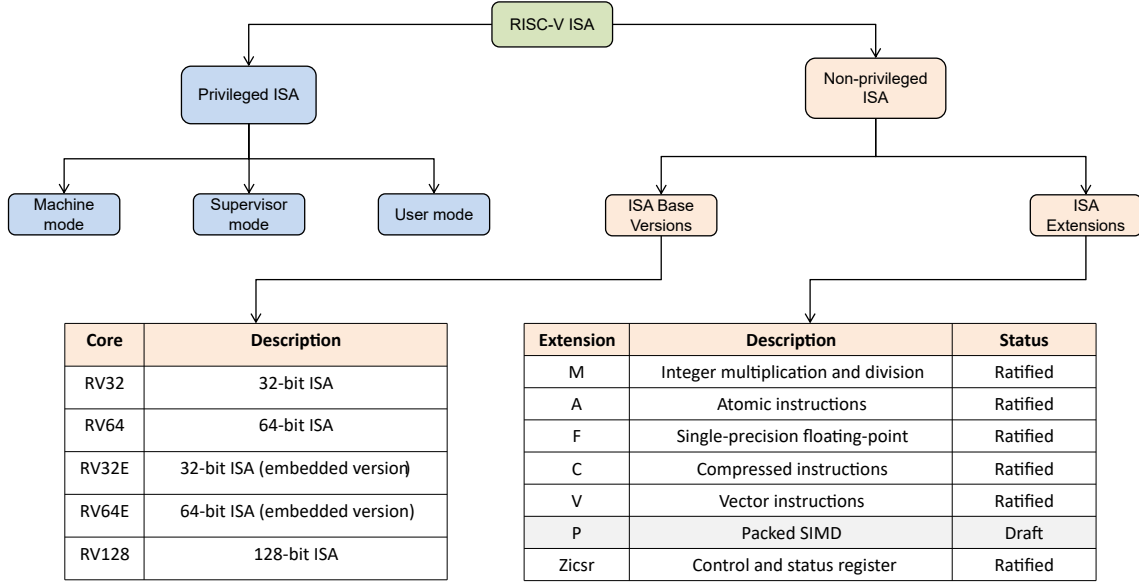


Figure 2: RISC-V ISA classification.

Such flexibility in offering multiple ISA base versions rather than a single instruction set also aligns with RISC-V's core philosophy of adaptability, ensuring it can suit a wide range of processors, making RISC-V versatile for various applications.

RISC-V is an example of general-purpose register architecture. Such an architecture allows all the registers to be used for any purpose. RISC-V consists of three base instruction set versions: RV32, RV64, and RV128. "RV" stands for RISC-V, and the numbers 32, 64, and 128 represent the corresponding bit widths of the registers. The standard RISC-V ISA defines 32 general-purpose registers, each 32 bits wide [14]. This thesis also adopts this standard processor configuration for implementing the P-Extension. This version has an advantage of offering a reduced cost which is better suited for low-cost processors [19]. Although, embedded "E" versions consist of a reduced number of registers [14], saving register-file area and lowering power for cost-sensitive micro-controllers.

RISC-V not only stands out for being open-source but its true strength lies in its modular design unlike most of the earlier ISAs. This means that at its core the base ISA remains fixed, offering a fixed target for developers while the optional standard extensions can be included based on the needs of the application. Such a modular nature of RISC-V is captured in its naming convention as well. For example, RV32IMAF represents the inclusion of the multiply extension (M), atomic extension (A), and single-precision floating point extension (F), on top of the core instruction set, RV32I. This ability to customise the ISA based on the requirement de-clutters the instruction set base and by informing RISC-V compiler about the included extensions, an optimised code can be generated for the hardware [31]. In this thesis, the RV32IMP configuration is employed, where 'P' specifies the non-privileged P-Extension of the RISC-V ISA that adds the SIMD functionality to the core.

Another important characteristic of RISC-V, contrary to other ISAs, is that

regardless of width of data path (i.e., bits of data the processor can handle at once), the size of each instruction in the instruction set across all versions always remains the same, i.e., 32 bits. This simplifies the process of compiling and decoding instructions [14, 19]. The main differences between various versions of base RISC-V ISAs, however, are the set of supported instructions, the register width and the size of memory address [14]. But, there are exceptions to this generalisation found in the compressed 'E' versions of RISC-V ISA [28, 14].

2.4 Instruction Encoding in RISC-V

RISC-V ISA follows the principle of simplicity which encompasses the fact that all its instructions are designed to be 32-bit long. The 32-bit instruction is divided into smaller groups of binary values called fields. The process of arranging these fields according to some predefined rules laid out by RISC-V ISA is referred to as Instruction Encoding and the resultant arrangement of fields is known as an Instruction Format. Due to different nature of operations performed by a RISC-V processor, for example arithmetic operation involving registers as operands, store operation which saves data from registers to memory etc, a single instruction format for all the operations cannot be maintained. As a result, RISC-V ISA defines a set of instruction format shown in Fig. 3.

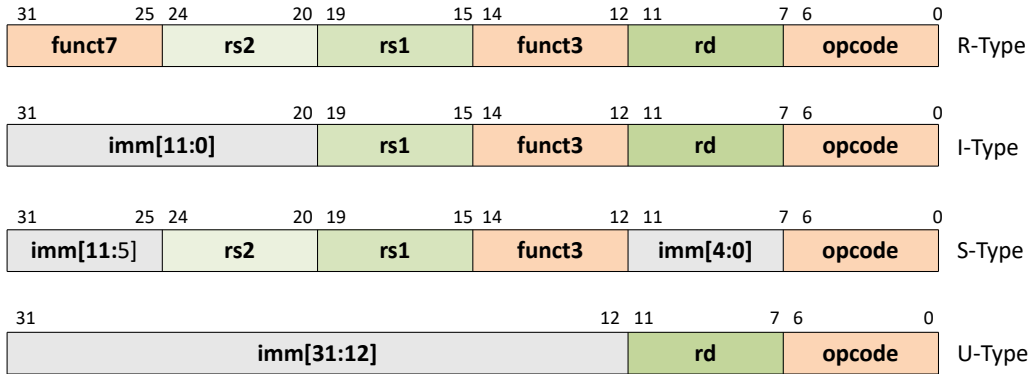


Figure 3: RISC-V base instruction formats.

The formats are distinguished by the values in the opcode field. Each format is assigned a distinct set of opcode value in the first field such that the hardware knows how to treat the rest of the instruction. In the official draft version of the P-Extension specification, all instructions utilise the opcode space of `0b1110111`, with the majority following the R-type instruction format. The opcode field together with funct7, funct3 are used by the decoder to distinguish between various types of instruction present in the specification as discussed later under Decoder and Control block modifications.

RISC-V is under constant ongoing development with an ultimate aim of becoming a universal instruction set architecture and make it as popular for hardware as Linux is for operating systems, supporting all kinds of processors ranging from small embedded controllers to the fastest high-performance computers [31]. One of the

modules that is under development is the P-Extension which adds the packed SIMD functionality to the RISC-V architecture.

2.5 SIMD and RISC-V ISA

Multi-level parallelism has become a central principle in modern computer design, with energy efficiency and cost emerging as the dominant limiting factors. One prominent form of parallelism is Data-Level Parallelism (DLP) [32], which arises in applications where numerous data elements can be processed simultaneously, for example DSP applications. In hardware, DLP can be exploited using SIMD technology wherein a common set of instructions is executed repeatedly on different data items, without any dependencies between individual operations. As a result, performance can be enhanced by such increased parallelism, allowing more work to be completed using fewer instructions.

Single Instruction, Multiple Data (SIMD) is one of the four architectural classifications defined in Flynn's Taxonomy, a foundational model for parallel computing introduced in the 1960s [33]. This category describes systems in which a single instruction stream operates concurrently on multiple data streams, thereby exploiting data-level parallelism. In modern computing, SIMD is commonly implemented through three major paradigms: packed SIMD (e.g., multimedia instruction sets like Streaming SIMD Extensions (SSE), Advanced Vector Extensions (AVX), or RISC-V P extension), vector SIMD (e.g., Cray-style or RISC-V Vector extension), and Graphics Processing Units (GPUs). GPUs execute many threads (sequence of instructions with their own data) using Single Instruction, Multiple Threads (SIMT) which is a form of wide SIMD execution [32].

Packed-SIMD extensions emerged from the observation that many multimedia applications operate on data types narrower than the standard 32-bit width for which most processors are traditionally optimised [32]. By partitioning functional units, such as adders, a processor can perform simultaneous operations on short vectors. An illustration in Fig. 4 shows various scenarios of packing multiple data values in a single word. Within a 32-bit adder, operations can be executed on four

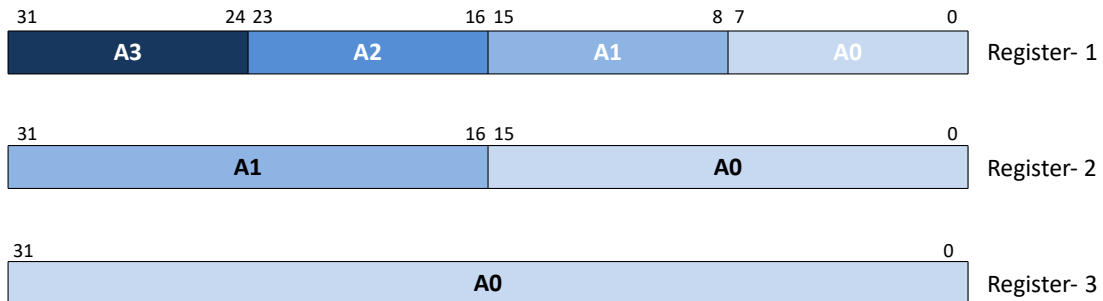


Figure 4: Various scenarios of packing a register.

8-bit operands, two 16-bit operands, or one 32-bit operand. This allows for "packed" SIMD operations, where multiple smaller data elements of same size and type are packed into larger registers and processed in parallel. Moreover in such a scenario,

single instruction fetch and decode logic can be reused for multiple operands, thereby amortising the cost of fetching and decoding the instruction over multiple operations.

In addition to delivering the expected speed-up and reduction in instruction count, SIMD also facilitates efficient and optimised data movement [34]. Memory accesses are typically slow and energy-intensive. However, SIMD offers a more energy-efficient alternative than Single Instruction Single Data (SISD) by minimizing the frequency and volume of such memory access operations. An illustration of SISD and SIMD memory access patterns, adapted from [35], is shown in Fig. 5. When

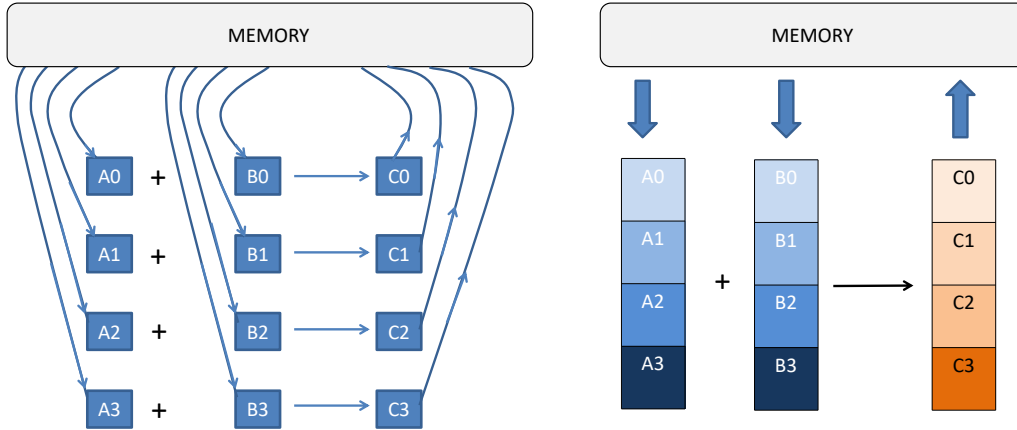


Figure 5: Arithmetic and memory access in SISD (left) and SIMD (right).

using SISD to operate on two vectors containing four elements each, the computation requires 12 memory accesses and 4 separate arithmetic operations. In contrast, packed SIMD allows all four operands to be loaded into a single register, enabling a single parallel operation across all elements. As a result, only three memory accesses and one arithmetic operation are needed. Hence significantly improving the efficiency compared to SISD execution.

The advantage of Packed-SIMD architecture over other SIMD alternatives is that it allows the reuse of the same register file and functional units for both SISD and SIMD operations. This shared-resource design results in lower area usage and reduced power consumption compared to vector-SIMD and GPU architectures which have their own separate register file and functional units. Although packed-SIMD remains a popular choice for small embedded devices due to low initial cost, ease of implementation, and the minimal extra processor state required, it has some notable limitations as well [32]. These are briefly described in the subsequent paragraphs.

Firstly, this architecture fixes the number of data operands within an opcode leading to proliferation of instructions every time the width doubles, further discussed in the next section. For example, since the introduction of packed-SIMD in x86, the ISA has significantly expanded from having about 166 instructions in 1994 to about 3600 instructions in 2015 [11]. This is also evident in RISC-V P-Extension specification which has over 300 proposed instructions making it one of the largest of the RISC-V ISA Extensions.

Secondly, generating efficient code for applications using packed-SIMD is more

challenging, as it historically lacks advanced addressing modes such as strided or gather/scatter data transfers. Packed-SIMD implementation requires fetching and storing the data in one perfectly aligned, consecutive block. In contrast, real-world data are seldom laid out so neatly. Older vector processors and modern GPUs, on the other hand, address these data layout challenges more effectively through built-in support for stride-based and gather/scatter memory access patterns.

Thirdly, the extent of parallelism achievable with the P-Extension is limited by the number of data elements that can be packed into a general-purpose register. For instance, on a 32-bit RISC-V processor, operating on 8-bit data elements yields a theoretical speed-up of $4\times$, whereas for 16-bit elements, the parallelism is reduced to a two-fold improvement. Lastly, packed-SIMD traditionally lacks mask registers to support conditional execution of elements, a feature common in vector architectures. This means that packed-SIMD implementation operates on several data lanes in one go but it lacks built-in support for turning some lanes off at runtime.

2.5.1 P-Extension and V-Extension

RISC-V ISA proposes two extensions that introduce support for SIMD operations. These are the P-Extension and the V-Extension[1] with the former developed earlier than the latter. Upon comparing the specifications of the two extensions, there is a substantial functional overlap between their instruction sets. However, the difference lies in the implementation of the instructions and moreover, overtime the P-extension has been developed focusing more on fixed-point operations while as V-extension has been standardised for large floating-point SIMD operations [36]. This dedicated focus arises from the fact that there are more variations of the fixed-point operations in the P-extension to facilitate the different needs in different DSP/Audio/Speech processing applications [37].

The SIMD floating-point instructions have been dropped from P-Extension, but both the P- and V- extensions support SIMD fixed-point operations. While fixed-point arithmetic offers an alternative for handling fractional values, it cannot fully replace floating-point arithmetic in all scenarios. As a result, the additional Floating Point Unit (FPU) in the V-Extension increases its area and complexity compared to P-Extension but the former is more widely applicable compared to the latter.

The P-Extension introduces packed-SIMD instructions that operate on fixed-length registers, whereas the V-Extension supports vector-SIMD instructions that work with variable-length registers configured using some specialised instructions. V-Extension adds 32 dedicated vector registers to the base RISC-V. Their vector settings are configured via seven additional Control and Status Registers (CSRs) and the *Configuration-Setting* instructions change the values of these CSRs. The P-Extension, on the other hand, does not introduce a new register file but reuses the General Purpose Registers (GPRs) of RISC-V ISA. The size of these GPRs depends on base architecture variant. For example, RV32I has thirty-two 32-bit general purpose registers.

The approach of handling the data elements of varying sizes is also an aspect of

distinction between the two extensions in question. In the P-Extension, each element size (e.g., 8-bit, 16-bit, 32-bit) has a distinct instruction defined for it. For example, this results in separate addition instructions for 8-, 16-, and 32-bit data elements. On the contrary, V-Extension instructions are independent of the size of data element. The element width, to be processed by the register, is explicitly defined by the extension, allowing selection from 8 to 64 bits, with reserved encodings extending up to 1024 bits. This flexibility enables V-Extension code to run across processors with varying vector register widths. Additionally, the V-Extension supports register grouping, allowing multiple vector registers to be combined for operations on larger datasets using a single instruction. The P-Extension, however, lacks such grouping capabilities and is constrained by the fixed 32- or 64-bit register widths of RV32I and RV64I, respectively, thereby limiting both the number of elements processed and the achievable performance, as discussed in the previous section.

In conclusion, the P-extension is meant for smaller and low-power applications while as V-extension caters to high power-and-performance SIMD designs. P-extension uses the general purpose registers and the scalar datapath of the base RISC-V architecture for executing the SIMD operations. As such, these operations are performed within the confines of standard 32-bit or 64-bit GPRs, depending on the processor used (RV32I or RV64I respectively). This enables parallel processing of relatively smaller data elements and hence offering limited parallelism. On the contrary, V-extension with its own set of vector registers and additional datapaths of processing elements like ALU, multiplier and FPU, allows it to handle larger datasets and perform more complex parallel computations. For example, the variable length vector registers allow a greater number of elements to be processed in parallel, with minimum being 128-bits and configurable up to 1024-bits.

2.6 RISC-V P-Extension

P-Extension aims to enhance the processing capabilities of Digital Signal Processing (DSP) algorithms executed on RISC-V processors [8]. The extension achieves this by introducing SIMD instructions for integer and fixed-point data types of 8-bit, 16-bit and 32-bit sizes. In addition, it includes several non-SIMD instructions, primarily targeting fixed-point arithmetic.

The official specification for the proposed P-extension remains in its draft stage and is yet to be ratified. At the time of writing this thesis, the specification stands at version 0.9.11 and is available in the github repository [8]. The original specification includes over 300 instructions. However, this thesis references a revised version, v0.01.01-draft-20230915, in which the instructions that are still under discussion are greyed out, while those under review or pending naming have been moved into a separate appendix. The revised specification features over 150 greyed and non-greyed instructions. This thesis focuses on implementing a subset of these instructions and integrating them into the A-core processor.

2.7 P-Extension Specification

Table I outlines all the categories of instructions defined in the P-Extension specification, along with the types of operations included within each category. The instructions are grouped into five distinct categories based on their operational focus, data-handling capabilities, and architectural applicability. These are SIMD Data Processing Instructions, Partial-SIMD Data Processing Instructions, 64-bit Data Computation Instructions, Non-SIMD Instructions and RV64 Only Instructions.

Table I: Complete list of P-Extension instructions, adapted from [10].

1. SIMD Data Processing Instructions	1.1. 16-bit Addition & Subtraction Instructions 1.2. 8-bit Addition & Subtraction Instructions 1.3. 16-bit Shift Instructions 1.4. 8-bit Shift Instructions 1.5. 16-bit Compare Instructions 1.6. 8-bit Compare Instructions 1.7. 16-bit Multiply Instructions 1.8. 8-bit Multiply Instructions 1.9. 16-bit Misc Instructions 1.10. 8-bit Misc Instructions 1.11. 8-bit Packing Instructions
2. Partial-SIMD Data Processing Instructions	2.1. 8-bit Packing Instructions 2.2. Most Significant Word “32×32” Multiply & Add Instructions 2.3. Most Significant Word “32×16” Multiply & Add Instructions 2.4. Signed 16-bit Multiply with 32-bit Add/Subtract Instructions 2.5. Signed 16-bit Multiply with 64-bit Add/Subtract Instructions 2.6.a. Miscellaneous Instructions (32-bit) 2.6.b. Miscellaneous Instructions (8-bit) 2.7. 8-bit Multiply with 32-bit Add Instructions
3. 64-bit Profile Instructions	3.1. 64-bit Addition & Subtraction Instructions 3.2. 32-bit Multiply with 64-bit Add/Subtract Instructions 3.3. Signed 16-bit Multiply with 64-bit Add/Subtract Instructions
4. Non-SIMD Instructions	4.1. Q15 Saturation Instructions 4.2. Q31 Saturation Instructions 4.3. 32-bit Computation Instructions 4.4. Overflow/Saturation Status Manipulation Instructions 4.5. Miscellaneous Instructions
5. RV64-Only Instructions	5.1. 32-bit Addition & Subtraction Instructions 5.2. 32-bit Shift Instructions 5.3. 32-bit Miscellaneous Instructions 5.4. Q15 Saturating Multiply Instructions 5.5. 32-bit Multiply Instructions 5.6. 32-bit Multiply & Add Instructions 5.7. 8-bit Parallel Multiply & Add Instructions 5.8. Non-SIMD 32-bit Shift Instructions 5.9. 32-bit Packing Instructions

The instructions selected for implementation in this thesis represent an intersection of three key criteria. First, priority was given to the non-greyed instructions in the specification, indicating those that are considered stable. Second, emphasis was placed on instructions fundamental to basic DSP operations such as addition, subtraction, comparison etc. Finally, instruction selection was guided by the specific requirements of the DSP applications chosen for benchmarking the performance of the P-extension, which are discussed in Chapter 4 of this thesis. A description of these instructions are discussed in the next paragraphs.

Addition and Subtraction

Based on how overflow conditions are handled during addition and subtraction operations, three distinct types of operations emerge: wrap-around, averaging, and saturation. Each of these types further includes variants depending on whether the operands are signed or unsigned. These three types of operations are described in the next few paragraphs using an example of 8-bit addition. For all variants, an extra 9th bit in red is internally used for overflow and sign preservation. The role of this overflow bit and how it influences the final result varies depending on the type of operation.

Fig. 6 illustrates the wrap-around operation using 8-bit operands. In this

$$\begin{array}{r}
 +124_{10} \text{ } 00111100_2 \\
 + \quad +10_{10} \text{ } 00001010_2 \\
 \hline
 +134_{10} \text{ } 010000110_2 \\
 \downarrow \\
 \text{Result} = -122_{10} \rightarrow 10000110_2 \\
 \text{In 2's complement}
 \end{array}$$

Figure 6: Wrap-around operation.

operation, the results exceeding the maximum representable value of a register wraps-around to the start of the range. The overflow bit from the operation is simply discarded and the consequences on the result are ignored. This applies to both signed and unsigned cases of wrap-around operation.

Fig. 7 illustrates the averaging operation using 8-bit operands. . This operation

$$\begin{array}{r}
 +124_{10} \text{ } 00111100_2 \\
 + \quad +10_{10} \text{ } 00001010_2 \\
 \hline
 +134_{10} \text{ } 010000110_2 \\
 \text{implicit sign in result} \\
 \text{ } 010000111_2 \\
 \text{output} \gg 1 \\
 \boxed{01000011}_2 \\
 \downarrow \\
 \text{Result} = +67_{10} \rightarrow 01000011_2
 \end{array}$$

Figure 7: Averaging operation.

handles the overflow event in a more careful approach. The intermediate result is right-shifted by one effectively halving the result and preventing the occurrence of an overflow. In case of signed averaging addition, an arithmetic right-shift is performed implying that the sign bits are preserved during the shifting operation. Since the

adder here uses 9-bit operation for 8-bit operands, the sign is preserved in the 9th bit and is used for averaging the output (right-shift by 1 bit) to get the result.

Another approach to preventing the wrap-around in case of an overflow is to saturate the result to the largest or smallest value of the register. Fig. 8 illustrates the saturating operation using 8-bit operands. The saturation values depend on the data-size used by the instruction and also on whether the instruction is signed or unsigned. For example in the example shown in Fig. 8, the saturating addition exceeds the maximum representable value of 8-bit signed register. As such the result gets saturated to +127.

$$\begin{array}{r}
 +124_{10} \quad 00111100_2 \\
 + \\
 +10_{10} \quad 00001010_2 \\
 \hline
 +134_{10} \quad 010000110_2 \\
 \text{positive saturation +127} \\
 00111111_2 \\
 00111111_2 \\
 \downarrow \\
 \text{Result} = +127_{10} \rightarrow 01111111_2
 \end{array}$$

Figure 8: Saturating operation.

Since subtraction is implemented as two's complement addition, it also inherits the same three types, namely wrap-around subtraction, saturating subtraction, and averaging subtraction. In addition to the signed and unsigned types for these addition and subtraction operations, there are 8-bit and 16-bit variants for these instructions as well as listed out in Table I.

Cross operations

A major downside of the SIMD operations is the overhead involved in packing SIMD registers with input elements in the correct order. The P-Extension specification attempts to address this drawback by providing cross variants of certain instructions. Fig. 9 and Fig. 10 illustrate two such operations, 16-bit Signed Averaging Cross Add-Sub and 16-bit Signed Saturating Cross Sub-Add operation respectively. These cross variants are defined only for the 16-bit instruction formats. Each of these crossed variants further supports the three overflow handling types of wrap-around, averaging, and saturation.

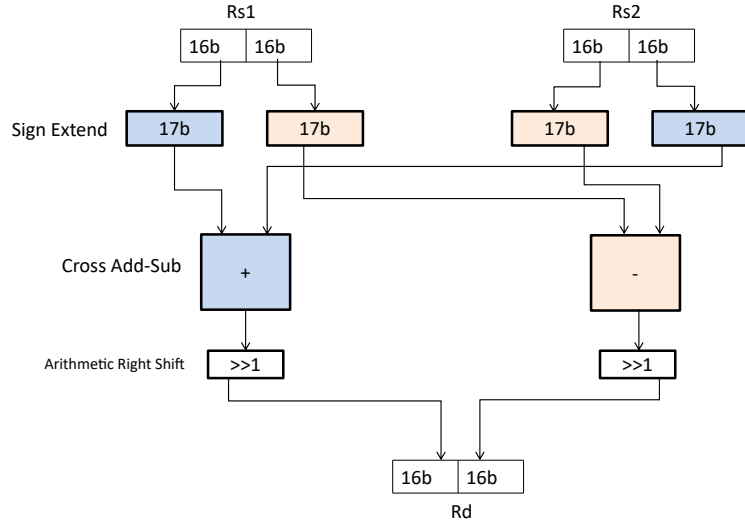


Figure 9: P-Extension Cross Add Sub Averaging operation.

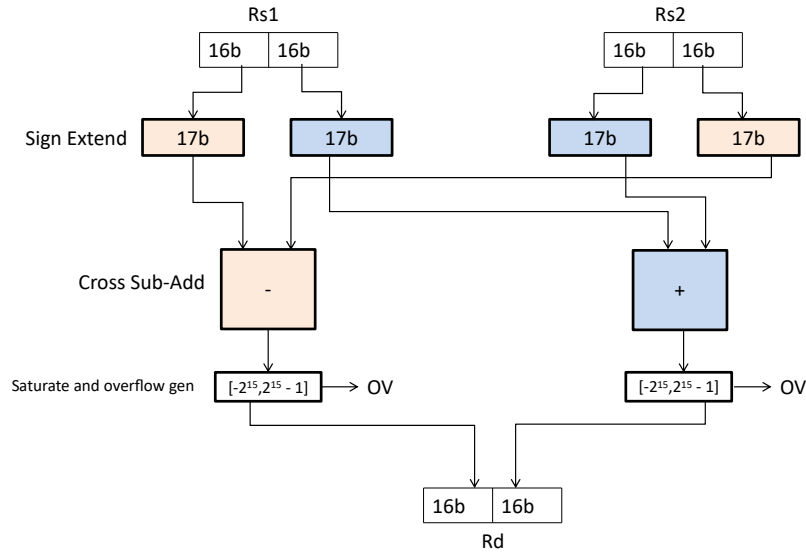


Figure 10: P-Extension Cross Sub Add Saturating operation.

Compare

An illustration of 16-bit equality compare operation is shown in Fig. 11 (a). These instructions perform 8-bit or 16-bit integer comparisons in parallel. The extension specifically introduces comparison instructions for equality, less-than, and less-than-or-equal conditions, each provided in both signed and unsigned variants for 8- and 16-bit data elements. The specification includes such minimal comparison logic since the rest of the cases can be generated easily generated using a logical OR and NOT depending on the requirement. Based on the instruction, the data elements in one register are compared with the data elements in another register for comparison operations. If the condition evaluates to true for an element, its corresponding destination register value is filled with ones otherwise it is set to all zeros.

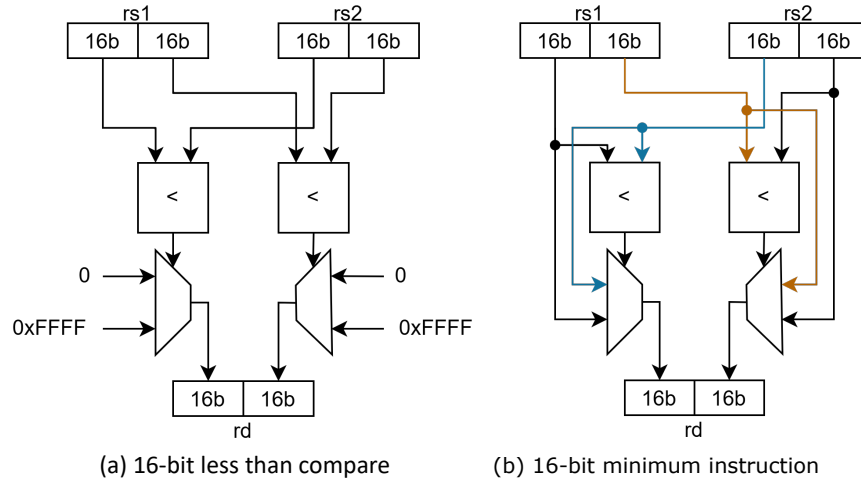


Figure 11: Less than and minimum operations.

Minimum and maximum

An illustration of the 16-bit Signed Minimum instruction is shown in Fig. 11(b). These instructions also perform element-wise comparison between two registers in the same manner as the compare instructions. However, instead of setting the destination register to all ones or zeros based on the condition, the minimum and maximum instructions return the smaller or larger of the two values, respectively.

Multiply and Accumulate

A Multiply-Accumulate (MAC) operation combines multiplication and addition into a single operation, making it essential for repetitive computations like matrix multiplications, where a sum of products is calculated. MAC operation in P-Extension specification has various forms depending on the data element sizes in the source register (e.g. words, half-words, or bytes).

This thesis implements three variants of 8-bit MAC operation. The operation of the 8-bit MAC operation is illustrated in Fig. 12. This instruction performs a dot-product operation by multiplying the four 8-bit elements within each 32-bit word of one source register (Rs1) with the corresponding signed 8-bit elements in another source register (Rs2). The four intermediate products are then summed and added to the content of the corresponding 32-bit word in the destination register (Rd). The final result is written back to the respective 32-bit segment in Rd. The other two variants differ based on whether the operands used in the MAC operation are signed, unsigned, or a combination of both.

Clip

An illustration of the 16-bit signed clip instruction is shown in Fig. 13. The clip instructions limit the value of an element to a signed integer range defined by an unsigned immediate value. The signed `clip` instruction limits each element to the

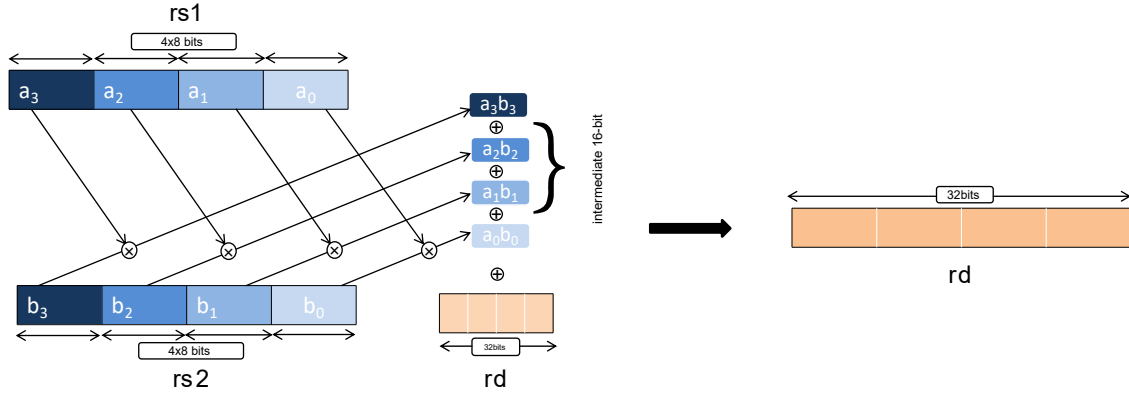


Figure 12: 8-bit MAC operation.

range from -2^{imm} to $2^{\text{imm}} - 1$, where imm is the immediate value. In contrast, the unsigned clip instruction clamps all negative values to zero, resulting in an output range from 0 to $2^{\text{imm}} - 1$.

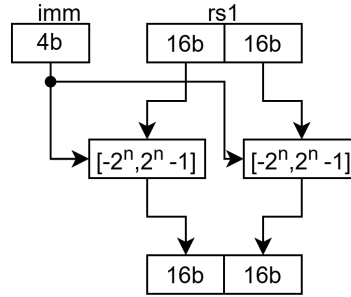


Figure 13: 16-bit Signed Clip operation.

Absolute

Fig.14 illustrates the example of lane-wise 16-bit absolute operation. These instructions compute the absolute value of signed integer elements packed within a 32-bit word, operating on each SIMD lane independently and in parallel. For each element in the source register Rs1, its absolute value is calculated and stored in the corresponding lane of the destination register Rd. A special case arises when the lane contains the most negative value. For example the most negative 16-bit value is 0x8000 (which represents -32768 in two's complement). Since the positive counterpart ($+32768$) cannot be represented in a 16-bit signed format (whose maximum value is $+32767$), the result saturates to 0x7FFF.

Fig. 15 shows the instruction encoding for variants of absolute instructions. Unlike most other P-extension instructions, these operate on a single source operand and differ in their encoding format. In addition to the standard funct3 and funct7 fields used for decoding, the rs2 field [24:20] is also utilized to distinguish between

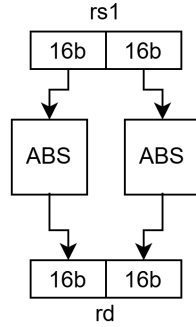


Figure 14: 16-bit absolute instruction.

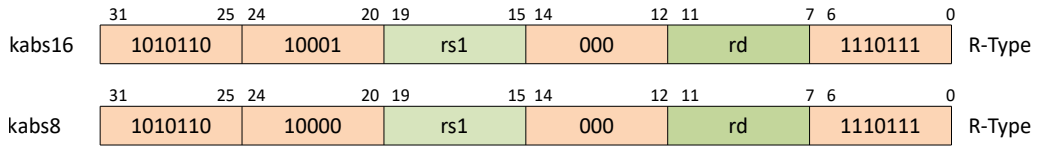


Figure 15: Instruction encoding in P-Extension absolute instructions.

data-size variants. For example, the rs2 field is set to 10000 for **kabs8** and 10001 for **kabs16**.

2.8 Overview of A-Core

A-Core is a 32-bit RISC-V processor developed at Aalto University and implemented using the Chisel hardware construction language. Fig. 16 shows the Chisel design of A-Core which is divided into three levels: **ACoreBase** which is the core level, **ACoreChip** which is the chip level and the **ACoreChip_wio** which is the System-on-Chip (SoC) level [38].

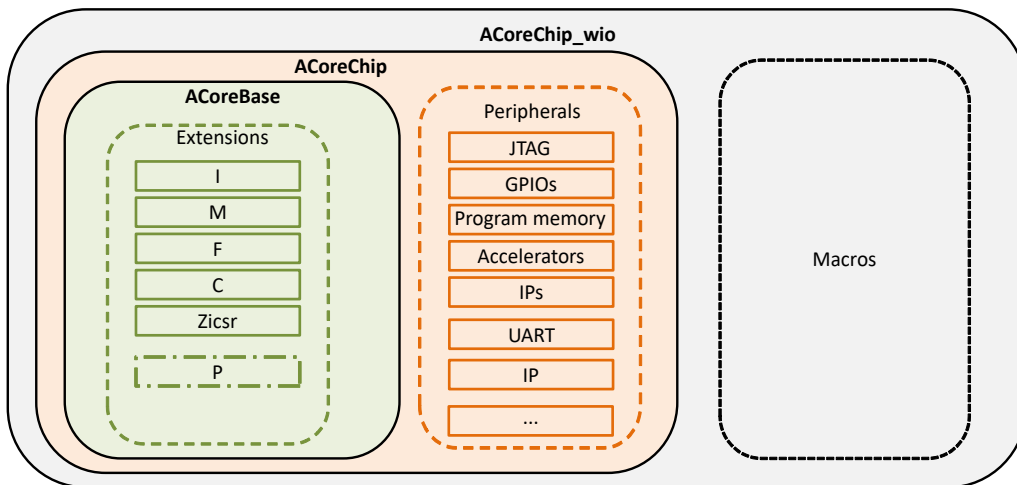


Figure 16: Chisel design of A-Core, adapted from [38].

The high-level architecture of ACoreChip is shown in Fig. 17. ACoreBase

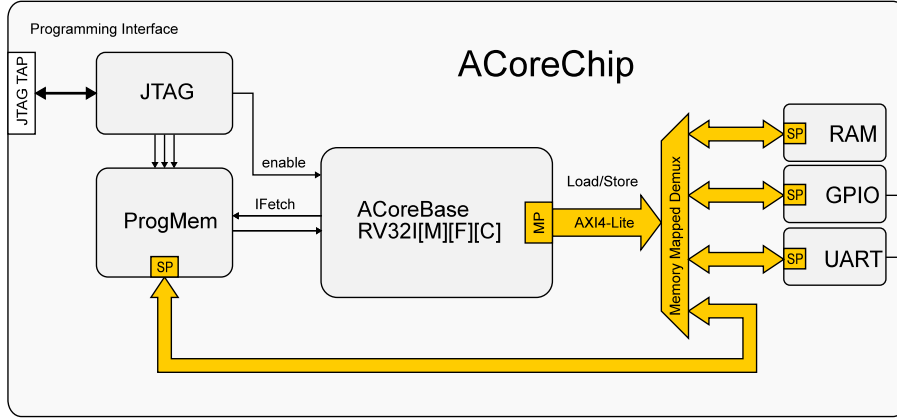


Figure 17: High-level architecture of ACoreChip, replicated from [12].

represents the 32-bit single processor core which is configurable for any combination of M, F, and C extensions. The core fetches its instructions from an internal program memory (ProgMem) that is loaded via a JTAG interface, having three Test Data Register (TDR) attached to the program memory. For data memory and I/O communication, the ACoreChip utilizes an AXI4-Lite interconnect, which implements a memory-mapped bus protocol for connecting peripheral. This way the peripherals can be controlled by writing to a memory address, allowing a wide range of possible peripherals, such as memory blocks, General Purpose Input/Output (GPIOs), Intellectual Property (IP) or accelerators, to be added in standard manner [38, 39].

A high-level structure of ACoreBase, as shown in Fig. 18, holds all the main processor structure, such as instruction decoding, integer and float files, control and status registers, ALU and the control block of the processor.

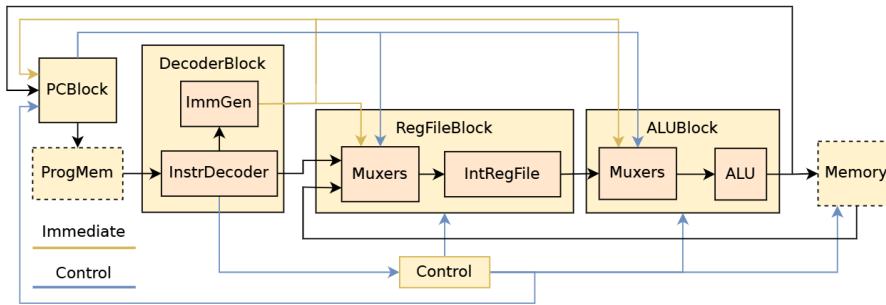


Figure 18: High-level structure of ACoreBase, taken from [38].

With just a single command, Verilog can be generated from the Chisel source code for any combination of the available extensions. When building the core, the desired extensions can be generated using the *isastring* configure option. For example, specifying *rv32imp* generated the verilog for both the M-extension (integer multiplication and division) and the custom P-extension for packed-SIMD operations.

During the integration phase of the P-Extension with A-Core, as discussed in Section 3.3, these blocks within the ACoreBase are modified.

The pipeline structure of ACoreBase is shown in Fig. 19. ACoreBase implements a classic five-stage RISC-V pipeline comprising Instruction Fetch (IF) Instruction Decode (ID) Execute (EX) Memory Access (MEM) and Write Back (WB) stages [39].

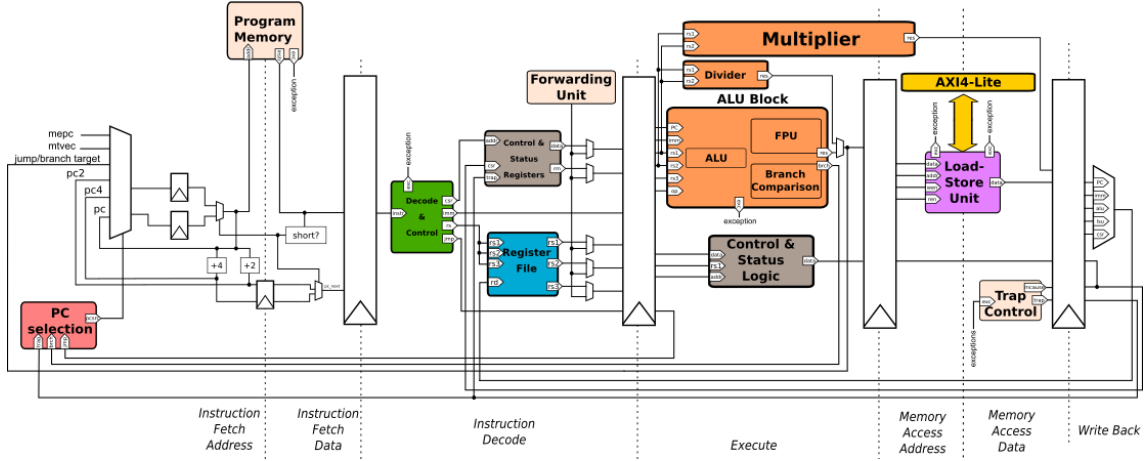


Figure 19: Pipelining in A-Core, taken from [12].

However, it extends this baseline architecture with additional features such as hazard management and multi-extension support. The single-cycle operation of ACoreBase is described in the following paragraphs:

During the IF stage an instruction is read from the on-chip program memory according to the program counter. Output from Program Memory is moved to pipeline register.

In the ID stage, the fetched instruction is passed to the instruction decoder, which interprets the instruction and forwards the relevant data. At this stage, the instruction type is also identified. The decoded information is sent to the control block, which determines the appropriate data paths for the register file and the ALU based on the instruction type. Simultaneously, the register file and CSR (Control and Status Register) block are read. If required, register and CSR forwarding is applied to resolve data hazards. Within the control block, the instruction fields are used to generate the inputs for the ALU, and the ALU targets are identified. Once the data path is configured, the register file provides the operand data to the ALU using the specified register addresses.

In the EX stage, the processor performs arithmetic and logic operations via the ALU, executes CSR data manipulation, and computes target addresses for branch and jump instructions. The ALU output is sent to the appropriate destination based on the instruction. When extensions are enabled, the execution process remains largely unchanged except the architecture accommodates two separate register files, Float file and Integer file along with dedicated arithmetic blocks for each. Additionally,

the control and status registers can be accessed for both read and write operations during this stage.

In the MEM stage, the calculated read or write address is forwarded to the Load-Store Unit (LSU). At this point, exception checks are performed, and any detected exception halts LSU write operations. The data retrieved from or intended for memory is then passed to the pipeline register, and its validity is verified before proceeding to the next stage.

Finally, in the WB stage, Data is written to register file and the CSR block. Each stage described above communicates through ready/valid pipeline registers, giving the controller cycle-accurate visibility to insert stalls or flushes without violating timing.

2.9 Agile hardware methodologies and origin of CHISEL

In the post-Moore’s Law era, the advances in transistor scaling has seen an end to rapid performance improvements in terms of energy-efficiency. Moreover, advancement in Very-Large-Scale Integration (VLSI) technology has increased the complexity of processor design which, inherently, has also become a cause for slowing down the design productivity of processors [40]. For example, modern SoCs contain extremely specialised hardware units which are often organised into multiple clock frequency domains to further save energy under varying loads. Such design complexities have caused chip production to exceed both budget constraints and time schedules, ultimately slowing down hardware innovation. This design productivity crisis stimulated the development of new tools and methodologies.[40]

Inspired from the software world, hardware designers and architects borrowed the principles of software agile development model [41] and applied them to hardware development. Out of this new hardware design thinking, an agile hardware manifesto was proposed in [40]. The authors of this manifesto had employed this methodology for 11 RISC-V microprocessor tape-outs on modern 28-nm and 45-nm Complementary Metal-Oxide-Semiconductor (CMOS) processes over a span of five years. The manifesto, at its core, encapsulates the research group’s collective experience gained over this period. This agile hardware approach allows to gradually improve the prototypes which are not fully complete but functional enough to test and build upon, enabling continuous improvement during the development process.

A case study is presented in [40] highlighting the potential of this agile hardware approach in post-Moore’s era. The authors used the agile principles while designing the Raven-3 microprocessor, featuring complex designs tasks like custom voltage conversion and adaptive clocking. By applying these principles, the team quickly identified and resolved a critical voltage regulation bug using a simple state machine in the early stages of the development. On the contrary, the traditional waterfall methods might have missed or caught it too late. The authors thus concluded that iterative and collaborative processes can accelerate hardware innovation in the post-Moore’s Law era.

In order to expedite agile hardware development, a new RTL design tool called

Chisel was developed at the University of Berkeley [42]. While traditional Hardware Description Languages (HDLs) like VHDL and Verilog are widely used in the industry, they fall short in terms of productivity and flexibility compared to modern programming languages [43]. Chisel was created to overcome these limitations and bring a more efficient approach to hardware design [44, 40].

CHISEL (Constructing Hardware in a Scala Embedded Language) as the name suggests is not a stand-alone hardware description language like VHDL or Verilog, but it is a set of open-source Scala libraries built on top of Scala programming language [44]. These libraries define new hardware data-types, classes, and functions that represent primitives to convert a hardware data structure into either a fast C++ simulator or low-level Verilog for emulation or synthesis [45, 44]. Traditional HDLs, despite minor updates in the early 2000s, looked outdated and lacked productivity, leading to verbose code that increased design time and complexity, especially in domains like Digital Signal Processing [46].

2.9.1 Advantages over traditional HDLs

Chisel improves the productivity through reuse and thereby reduces the Non-Recurring Engineering (NRE) cost. With Chisel, a designer does not have to write the hardware description every time but instead writes parametrised hardware generators which generate the hardware [45]. This improves code reuse within a given design and across generations of design iterations. Moreover, Chisel introduces modern programming paradigms such as object-oriented and functional programming into hardware design [47]. This enables developers to define hardware at a higher level of abstraction, offering greater flexibility in the design process. As such the designers are able to build more in less time, while amortising the cost of verification through reuse.

By leveraging Scala’s object oriented constructs, Chisel considerably reduces the code density of a hardware design compared to other HDLs. For example, as shown in Fig. 20, Chisel offers a high code density compared to other HDLs when declaring an 8-bit register with a synchronous reset. Chisel combines declaration, bit-width, and synchronous reset all in one line.

<pre> 1/--CHISEL--// 2// 8-bit register reset to 0 3val reg = RegInit(0.U(8.W)) </pre> (a)	<pre> 1/--Verilog--// 2// 8-bit register reset to 0 3 4reg [7:0] reg; 5 6always @(posedge clk or negedge rst_n) begin 7 if (!rst_n) 8 reg <= 8'd0; 9 else 10 reg <= reg_next; // user logic 11end </pre> (b)	<pre> 1--VHDL-- 2signal reg : std_logic_vector(7 downto 0); 3 4process (clk) begin 5 if rising_edge(clk) then 6 if rst = '1' then 7 reg <= (others => '0'); -- reset to 0 8 else 9 reg <= reg; -- hold current value 10 end if; 11 end if; 12end process; </pre> (c)
---	--	--

Figure 20: Declaring an 8-bit register with synchronous reset in (a) Chisel (b) Verilog (c) VHDL.

A study presented in [48] conducts a comparative analysis of two equivalent 64-bit RISC-V processor cores, one implemented in Verilog and the other in Chisel. The

code density, determined by the total number of source-code lines, is examined in each case. The Chisel implementation required only 586 lines of code to realise the basic five-stage pipeline structure of the RISC-V core, whereas Verilog implementation needed 1,256 lines. In another study to measure designer productivity offered by Chisel, a 3-stage 32-bit RISC processor, which was originally written in Verilog, was converted to equivalent Chisel code [44]. A significant 3x reduction in code density was recorded, with the original implementation consisting of 3020 lines of code, reduced to only 1046 lines in the latter implementation.

Being built on Scala, Chisel also provides type checking advantage of the Scala compiler, ensuring the design errors and type mismatches are caught at compile-time [43]. As such, the hardware generated is more reliable and predictable. On the contrary, the traditional HDLs with a less stricter approach to type checking often lead to run-time errors which are harder to detect and debug. Moreover, the hardware described in Chisel always produces synthesizable Verilog representation eliminating the need to manually ensure synthesizability [43].

Chisel also has its own built-in testing framework known as ChiselTest [49], facilitating quick verification of modules and designs before integration. Like Chisel, ChiselTest is also a Scala library which provides primitives to write testbenches. It provides an interface into several simulators through *peek* (read value from circuit), *poke* (write value to circuit), and *step* (advance time) operations [50].

The central component of any compiler is the Intermediate Representation (IR), which serves as the foundation for all subsequent compiler transformations. Similarly, in a hardware compiler framework, a hardware-specific IR is a crucial element. Flexible Intermediate Representation for RTL (FIRRTL) is an open-source hardware IR that acts as a bridge between Chisel’s high-level hardware descriptions and the low-level representations used for hardware synthesis and implementation [51].

2.9.2 Chisel Compilation Pipeline

The journey from Scala to silicon begins with the hardware design being written in Chisel. The Chisel compilation pipeline, as illustrated in Fig. 21, outlines this transformation flow.

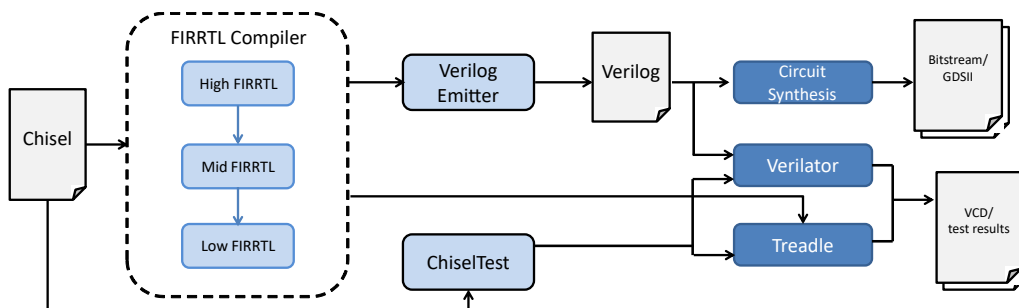


Figure 21: Chisel compilation pipeline, adapted from [50].

The Chisel compiler’s frontends parse the hardware designs into FIRRTL IR,

similar to how languages like C are lowered into Low Level Virtual Machine (LLVM) IR before becoming machine code. FIRRTL itself exists in three increasingly smaller, stricter and simpler forms of FIRRTL features: High form, middle form, low form. During the lowering phase, a series of compiler transformations are applied that are targetted at performing optimisations and simplifications of the design. Once the design reaches Low FIRRTL, it is passed to the Verilog emitter, which generates a conservative subset of Verilog code. From here, using the back-end tools the design can be (i) synthesised into an FPGA bitstream or ASIC GDSII, or (ii) simulated with standard Verilog simulators such as Verilator.

Chisel also supports early functional validation without leaving the Scala environment. The Treadle execution engine interprets FIRRTL directly, enabling rapid prototyping and testing. Developers can write Scala-based testbenches using the ChiselTest library to validate functionality long before synthesis or Verilog generation.

However, being relatively new, Chisel’s adoption in the industry is not as widespread. Moreover, it has a smaller knowledge base, fewer resources and limited community support compared to VHDL and Verilog. Despite these challenges, Chisel was chosen for the development of the RISC-V A-Core processor at Aalto University [12], including the design of P-Extension ALU which is the focus of this thesis.

2.10 Related Works

This section discusses the works related to this thesis.

AndesCore D-series

The AndesCore D23 is a 32-bit entry-level RISC-V CPU core featuring a 3-stage pipeline, designed for low power and high efficiency embedded processing and IoT applications in a compact footprint. In addition to supporting the RISC-V RV32GC base with single- and double-precision floating-point units (FPUs), it also includes several recently ratified extensions such as B, K, Cache Management Operations (CMO) extension, the Code Size Reduction extension, and the draft version of the P-Extension [9, 52].

Another core in the series, the D23-SE, builds upon the same base ISA as the D23 but incorporates additional safety features making it suitable for safety-critical applications [53]. For higher-performance use cases, such as real-time compute-intensive edge computing tasks, the AndesCore D45 serves as a DSP/SIMD-focused application processor with an 8-stage pipeline architecture [54, 55]. Although these cores support the P-extension, its design is proprietary and not openly accessible. As such not much information is available about the design.

PULP Core with SIMD and DSP-oriented Extensions

The Parallel Ultra-Low Power (PULP) platform, presented in [56], is a cluster architecture featuring RISC-V cores specifically designed for Ultralow-Power (ULP) operation in Internet-of-Things (IoT) endpoint devices. The platform achieves high

energy efficiency through Near-Threshold (NT) operation and gains performance scalability through parallelism. The core in this work is based on an open-source RISC-V RV32IM architecture, which has been optimised with various instruction extensions and micro-architectural enhancements. These optimisations are designed to enhance computational density and reduce pressure on the shared memory hierarchy (though SIMD extensions). However, this work predates the official proposal of the P-extension and therefore utilizes a custom instruction set which, while conceptually similar, deviates from the current P-extension specification.

CVA6 P-extension

The CVA6 is an open-source, 6-stage pipelined, application-class RISC-V CPU core originally developed by ETH Zürich and the University of Bologna, now maintained by the OpenHW Group. Written in SystemVerilog, it supports both 32-bit (RV32) and 64-bit (RV64) instruction sets and is suitable for production-quality Application-Specific Integrated Circuit (ASIC) and Field-Programmable Gate Array (FPGA) implementations [57, 58, 59].

The P-extension was implemented on a 64-bit CVA6 RISC-V core as part of a master’s thesis at Delft University of Technology [10]. The design was evaluated on an FPGA, where the integration of the P-extension resulted in an approximate 14.5% increase in Lookup Table (LUT) usage, while delivering an average performance improvement of $3.3\times$ on real-world machine learning benchmarks.

Ibex P-Extension

Another implementation of the draft P-extension specification was carried out using the embedded-class Ibex CPU, as part of a master’s thesis at the Norwegian University of Science and Technology. Ibex is a two-stage pipelined RISC-V CPU core that can support the RV32IMCB instruction set.

The implementation was synthesized for ASIC, aiming to evaluate the cost and performance impact of integrating packed SIMD ISA extensions. With the P-extension enabled, the design achieved a $3.1\times$ improvement in real-world machine learning benchmarks. Although a 35% increase in area and a 24% reduction in maximum operating frequency were observed, the energy required to complete a task was reduced by an average factor of 3.45.

This thesis implements the draft RISC-V P-Extension on Aalto University’s A-Core pipeline, marking the first chisel implementation to natively support sub-word SIMD operations on this five-stage processor core. This work contributes both to the development of the A-Core architecture and to the broader RISC-V ecosystem’s efforts toward a ratified, resource-efficient SIMD standard.

3 Implementation

This chapter presents the implementation of the P-extension in the A-Core processor. It begins with the design decisions for the ALU architecture, followed by verification using ChiselTest. The chapter then details how the extension was integrated across different pipeline stages and concludes with toolchain updates and validation through assembly-level testing.

3.1 Design considerations for P-Extension ALU

The design of P-Extension ALU in this thesis is heavily inspired by the concept of hardware reuse. It is a well-known circuit optimisation approach, which is traditionally employed for saving system and circuit area [60]. In modern embedded systems domains, circuit area and power consumptions are important aspects [61]. While the designers often let the synthesis tools to be in-charge of applying design optimisations in order to optimise these parameters but the results are not always satisfactory [62]. In fact, hardware reuse offers a more direct means to demonstrate a big influence on all three physical parameters viz circuit area, energy consumption and computation speed [62].

The basis of successful hardware reuse is the mutually exclusive nature of similar RTL-level operations. It works by sharing one or more functional units (FUs), like an adder, to implement several operations described in HDL, thereby producing less hardware components in the final netlist representation. However, it can also be observed that hardware reuse of complex units like adders leads to inclusion of additional simpler components like multiplexers which introduces a drawback of possible additional latency.

Fig. 22 shows a high-level diagram of P-Extension ALU.

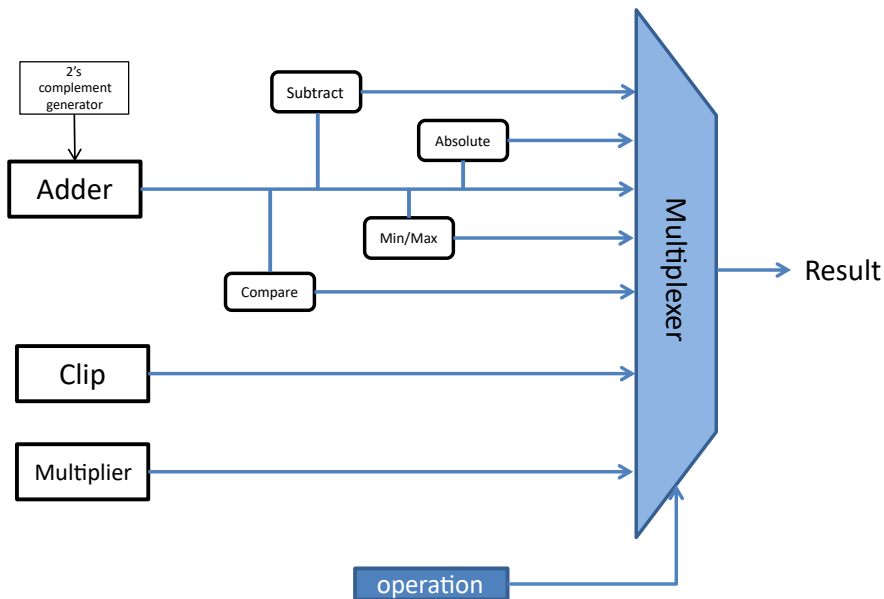


Figure 22: High-level block structure of P-Extension ALU.

The ALU is implemented as a single-cycle, fully combinational datapath composed of a set of parallel Processing Units (PUs) and a large Mux. Each PU is responsible for executing a specific packed-SIMD primitive. The large multiplexer selects the output from the appropriate PU based on the *operation* input, which is represented as an *enumeration* type. The design is developed in Chisel using a hierarchical, type-safe module structure, in which each PU is implemented as a *Module* class.

3.1.1 Single 4x9-bit adder unit

As discussed in Section 2.7, the P-extension specification contains multiple variants of the same instruction based on the size of the data elements. Moreover, for an instruction defined for one data element size (e.g., 8-bit), there are also multiple variants listed for that instruction. The instruction variants performing the addition operation in the P-specification are shown in Table II.

Table II: Addition instruction variants.

Element size	Addition Instruction
8-bit instructions	Wrap around Addition
	Signed Averaging Addition
	Unsigned Averaging Addition
	Signed Saturating Addition
	Unsigned Saturating Addition
16-bit instructions	Wrap around Addition
	Signed Averaging Addition
	Unsigned Averaging Addition
	Signed Saturating Addition
	Unsigned Saturating Addition
	Cross Add Sub
	Cross Averaging Add Sub
	Cross Saturating Add Sub
	Cross Sub Add
	Cross Averaging Sub Add
	Cross Saturating Sub Add

Additionally, many instructions across different Processing Units (PUs), such as comparison and absolute operations, share the same fundamental principle as addition. After analysing the nature of such operations throughout the specification, it was realised that the concept of hardware reuse could be employed by having a single functional adder unit which would implement all such arithmetic operations. This approach also met the objective of the P-Extension in offering area efficient DSP solutions by minimising the number of distinct functional blocks.

At the heart of the ALU design is the 4x9-bit adder functional unit, exemplifying the principle of hardware reuse. Instead of implementing separate hardware for each type of operation across various data widths, the design leverages four 9-bit adders

for both byte-level and half-word level operations. A single 4x9-bit adder module tends to be a sweet spot for achieving area efficiency and subword parallelism. For a 32-bit word, this approach allows straightforward four independent 8-bit parallel operations in case of 8-bit instructions while as, in case of 16-,32-bit operations, carry propagations can be easily routed at the each 8-bit boundary without introducing additional logic as would be required in case of a single 32-bit adder module or 2x16-bit adder module. This reuse minimizes the area footprint, reduces power consumption, and simplifies the design, all of which are vital considerations for embedded and DSP systems.

Although the elemental data-size is 8-bits, the adder unit operates on 9-bit operands. This is done so as to store and propagate the carry-out bit from the 8-bit arithmetic operations. Moreover, generating a 9-bit result from the adder unit is also useful in performing the arithmetic right-shift operation essential in signed-averaging addition and subtraction operations as illustrated in the example in Fig. 7.

The design of the 4x9-bit adder unit is shown in Fig. 23. The 8-bit data elements

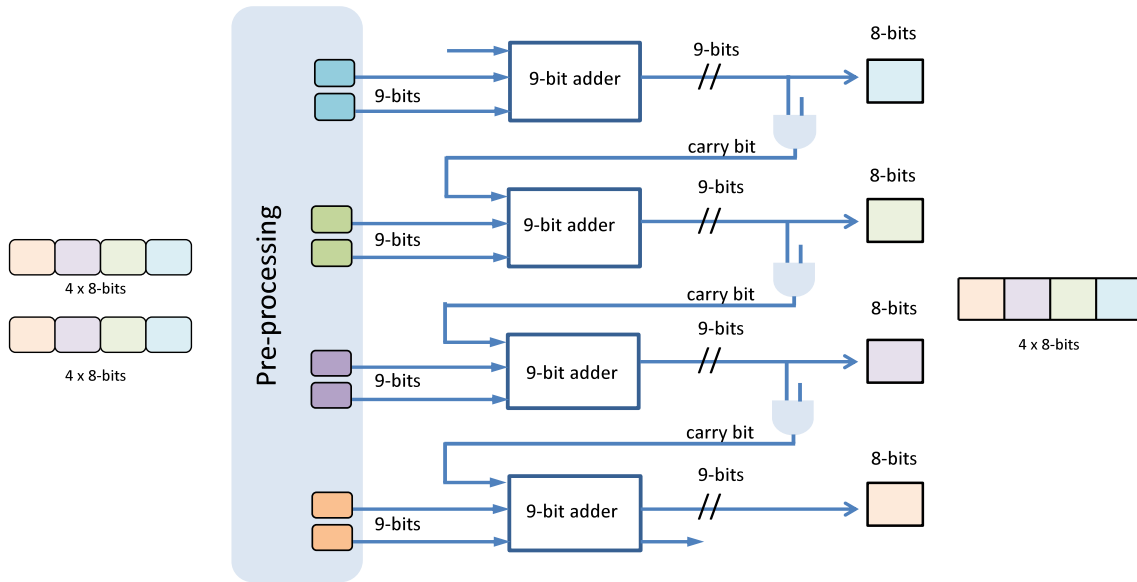


Figure 23: 4x9bit adder.

in the 32-bit operands are sign-extended to 9-bits before forwarding to the adder units. This is represented by the pre-processing block. The *Cat* construct of chisel is used to sign-extend the data points. Each adder performs the addition operation in parallel. The lower 8-bits from the result are then sent to the corresponding bit position in the 32-bit output. The 9th bit of the result is conditionally sent to the next adder unit based on whether the instruction is for 8-bit or 16-bit data sizes. In case of instructions with 8-bit data size, the four lanes operate completely independently and the carry-out of zero is sent to the next adder and while as in case of 16-bit data size, the adder pair belonging to each 16-bit half-word is chained. The carry-out of lane 0 becomes the carry-in of lane 1 so the two-byte groups form true 16-bit additions while still keeping the upper and lower half-words independent.

of each other.

The wrap-around addition behaviour, across the data-sizes, is implemented by concatenating only the least-significant eight bits of every lane to build the 32-bit result, naturally discarding any ninth-bit overflow. In case of averaging addition, the behaviour for 8-bit data lanes is implemented in hardware by simply discarding the least-significant bit of each 9-bit sum from the adder units. This essentially performs an arithmetic right-shift by one. While in case of 16-bit data lanes, the averaged result is constructed by concatenating the upper nine bits [8:0] of the high-byte sum with bits [7:1] of the low-byte sum, essentially implementing the divide-by-two (average) through simple wiring rather than logic. This is achieved using the *Cat* construct in chisel. The saturation behaviour is detected using the MSBs of the result as detailed in Table III using a generic 9-bit result from an 8-bit operation. This applies for both addition and subtraction operations. In case of unsigned numbers, the saturation is simply detected by observing the MSB of the result. However detecting saturation in case of signed numbers is done by taking a XOR between the two MSBs of the result. The final output is generated using a MUX which will choose

Table III: Signed and unsigned saturation detection logic.

	9-bit binary	Description	Saturation
Signed Saturation	b1_1XXX_XXXX	Must be a negative number larger or equal to -128	No
	b0_0XXX_XXXX	Must be a positive number smaller or equal to 127	No
	b0_1XXX_XXXX	Must be a positive number larger than 127	Yes
	b1_0XXX_XXXX	Must be a negative number smaller than -128	Yes
Unsigned Saturation	b1_0XXX_XXXX	Must be larger than 255	Yes
	b0_0XXX_XXXX	Must be between 0 and 255	No

between the appropriate saturating output (maximum 8-bit/16-bit values (0x7F / 0x7FFF), minimum 8-bit/16-bit values (x80 / 0x8000)) or the actual result. The final result is constructed using the *Cat* construct which concatenates the least-significant eight bits of every lane to build the 32-bit result.

The subtraction instructions have variants analogous to those of the addition instructions discussed earlier. Subtraction is implemented using two's complement addition, where the negated form of the second operand is first generated. Once these two's complement values are correctly routed to the second operand input, the subtraction instructions follow the same methodology outlined for addition operations above.

3.1.2 Two's Complement generation

A two's complement generator supports operations on both signed and unsigned numbers by generating on-the-fly two's complement representation for subtraction operations. This module, designed for enhanced flexibility and modularity in the design, seamlessly serves both 4×8 -bit and 2×16 -bit packed data paths. A single boolean flag selects between the data size granularity. When false, each of the four 9-bit lanes independently performs the two's complement operation, producing sign-extended 9-bit results for byte-level operations. When true, the logic concatenates adjacent byte pairs into two sign-extended 17-bit half-words, performs two 2's-complement (upper and lower half-word) at that wider width of 17-bits, and then re-splits the result so that the upper bits retain the correct sign.

The addition and subtraction operations within the cross operations are implemented in a similar fashion as the addition and subtraction variants discussed above. The *Cat* operator is used strategically to select and arrange the appropriate output chunks to form the final results in the destination register. These selections are made in accordance with the instruction descriptions provided in the P-extension specification, as was also discussed in Section 2.7.

3.1.3 Compare and Minimum/Maximum

As far as the comparison operations are concerned, the P-Extension specification contains instructions for equality and less than conditions. These operations reuse the same adder to maintain hardware efficiency. The equality condition is checked by performing a SIMD subtraction on the two inputs and checking if the result is zero. The less than condition is evaluated by observing the sign of the inputs and the result. The relationships between the MSBs of the inputs and the subtraction result, used to determine the less-than condition, are summarised in Table IV. For example, when

Table IV: Truth Table for obtaining less than comparison based on MSBs, [10].

MSB (a)	MSB (b)	MSB (result)	signed (a < b)	unsigned (a < b)
0	0	0	0	0
0	0	1	1	1
0	1	0	0	1
0	1	1	0	1
1	0	0	1	0
1	0	1	1	0
1	1	0	0	0
1	1	1	1	1

both inputs are positive (MSBs = 0) and the result of the subtraction is negative (MSB = 1), it indicates that the subtrahend (b) is larger than the minuend (a). The minimum and maximum operations follow the same implementation strategy as the comparison operations, with the only difference lying in the value sent to the outputs under various conditions, as already discussed in Section 2.7.

3.1.4 Absolute

Absolute value operations are also implemented using the same adder unit by generating a negated version of the operand. This is achieved by performing a subtraction-from-zero operation for each SIMD lane in parallel. However, this requires the single operand of the absolute instruction to be placed in the second input of the adder whilst the first input being zero. Output selection for positive, negative, and most negative values is handled using a multiplexer. The sign of the original operand is evaluated, and based on this, the multiplexer conditionally selects the output: the original value for non-negative inputs, the negated value for negative inputs, or the maximum representable value of the data-size element.

3.1.5 Clip

The clipping operations in the P-Extension ALU are handled by a dedicated processing unit (PU), as illustrated earlier in Fig. 27. The module implements both signed and unsigned clipping variants across 8-bit and 16-bit SIMD lanes. Instead of using the costly magnitude comparators to directly compare input values against upper and lower bounds, the module implements a compact, mask-based overflow detection scheme. A single control bit selects whether these bytes are interpreted individually (for 8-bit operations) or grouped into two half-words (for 16-bit operations). Fig. 24 and Fig. 25 show the internal logic used in the clipping unit using 8-bit signed and unsigned examples to illustrate how the masking mechanism detects overflows.

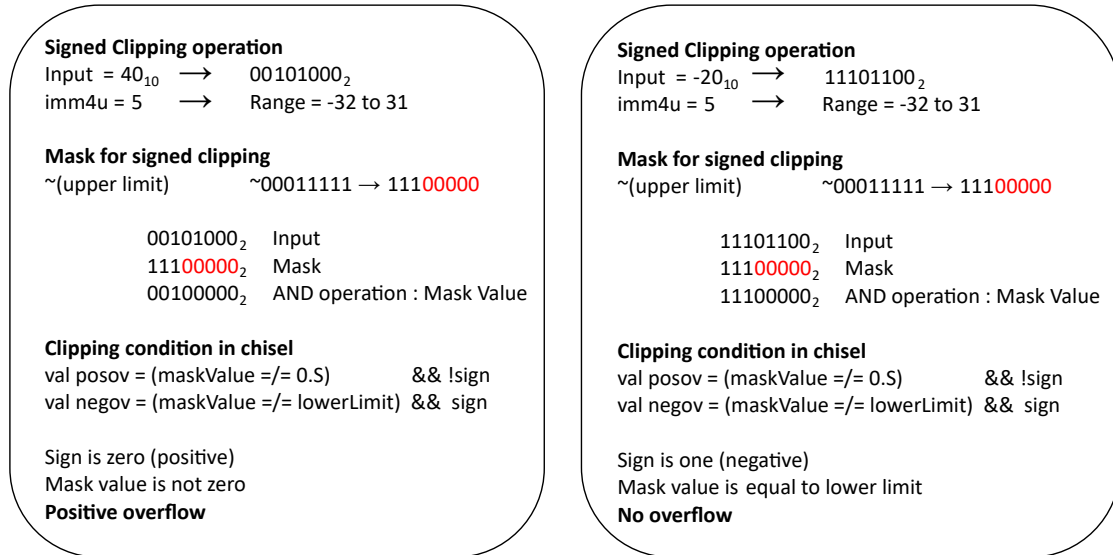


Figure 24: Signed Clipping operation.

To detect whether an input exceeds the specified clipping bounds, the inverse of the upper limit is selected as a mask. This mask is bitwise ANDed with each SIMD lane element, and a masked value is computed for each lane within the 32-bit word. The result from this AND operation is finally used in a boolean expression to check whether the lane value exceeds the clipping bounds. According to the expression, if

the result is non-zero while the sign bit is zero, a positive overflow occurs, and if it differs from the lower limit while the sign bit is one a negative overflow is detected. The two Boolean outcomes then feed a single 3-input multiplexer. On a positive overflow it forwards the upper-limit, on a negative overflow lower-limit, otherwise it passes the unmodified operand. It is important to note that in case of unsigned clipping operations, the input lane elements may still be signed integers however, the clipping limits are interpreted as unsigned thresholds during the comparison and saturation logic, Fig. 25.

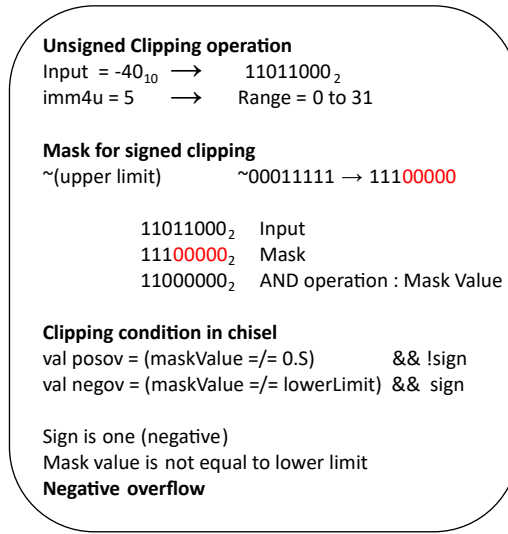


Figure 25: Unsigned Clipping operation.

3.1.6 Multiply-Accumulate

MAC operations are implemented using a separate PU. The top-level structure is shown in Fig. 26. It houses four 9x9 kernels which execute four MAC operations

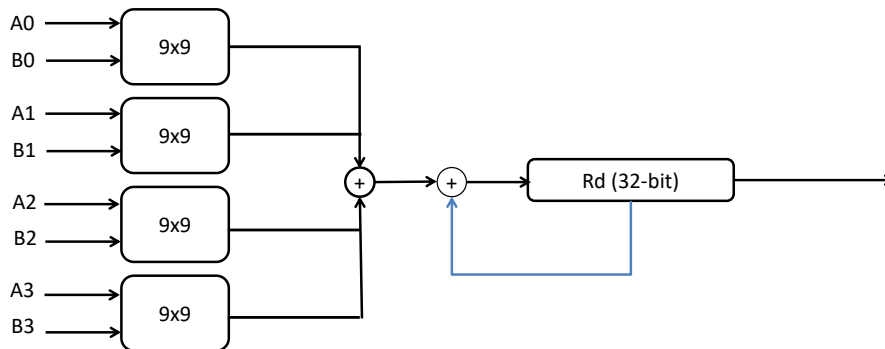


Figure 26: Top-level module for MAC module.

in parallel. Each operand byte from the 32-bit inputs is sign-extended to 9 bits by concatenating its most significant bit (Cat(MSB, byte)), resulting in four 9-bit values. Such sign extension ensures proper 8-bit signed multiplication wherein 8-bits are used

for operands and 9th bit is used for proper signed multiplication during the operation. These 9×9 operands are then fed into four parallel multipliers instantiated using a Chisel for loop, where each kernel uses the `*` operator to produce an 18-bit product. The four products are accumulated with full carry propagation using `reduce(_+&_)` construct. This sum is then added to the current 32-bit value in the destination (accumulator) register. The final result is truncated to 32 bits (with wrap-around and no saturation) and written back to the destination register.

3.1.7 Control and Status Register

The P-extension ALU is designed with two dedicated ports for interacting with the `vxsat` CSR: `vxsat_in` and `vxsat_out`. The `vxsat_in` port reads the current value of the `vxsat` register and forwards it unchanged to `vxsat_out` when the executed instruction does not specify an update to the overflow bit. In other words, the overflow (OV) bit in the `vxsat` register is implemented as sticky. This design choice ensures reliable detection of saturation events across blocks of DSP operations. Without this approach, repeated toggling of the overflow bit during a sequence of SIMD operations could result in ambiguity, making it difficult to determine whether or not overflow occurred within the particular data block.

The final I/O ports of the P-extension ALU are shown in Fig. 27. The inputs `Rs1` and `Rs2` represent the two source operands. The ports `vxsat_in` and `vxsat_out` handle the read and write operations for the `vxsat` CSR. The operation input port receives the instruction opcode, determining the function to be executed. An additional input `acc` which is required for single-cycle multiply-accumulate (MAC) instructions, provides the current value of the destination register. Finally, the ALU output is sent out through result port.

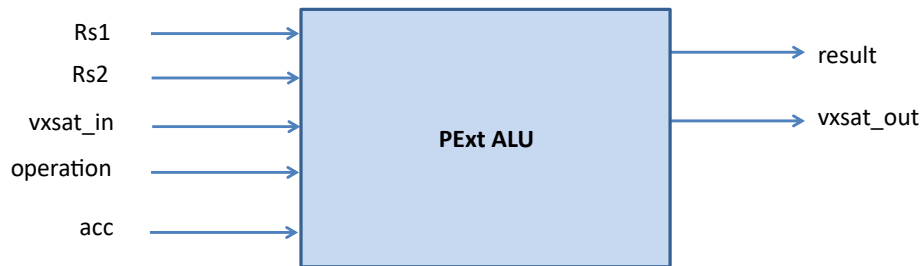


Figure 27: IO ports of P-Extension ALU.

3.2 Initial testing using Chiseltest

The built-in Chiseltest framework facilitated the development of lightweight and efficient tests. An agile approach was adopted during the development, enabling testing to be conducted iteratively as new instructions were developed, rather than deferring validation until the end of the process. An example snippet of the test for 16-bit Signed Saturating Addition test case sequence is shown in Fig. 28. To

verify the functional correctness of each implemented instruction, a set of corner cases and randomly selected examples were manually computed in both hexadecimal and decimal formats. These test cases were carefully chosen to ensure coverage of:

- Normal operation with no overflow
- Positive overflow, negative overflow and wrap around (e.g., $0x7F \rightarrow 0x80$, $0xFF \rightarrow 0x00$)
- Signed vs. unsigned behaviour in both 8-bit and 16-bit lanes
- The saturation flag behaviour of instructions that can raise the OV flag in the `vxsat` CSR

The hand-computed cases were organised as a table of test-vectors using `Seq` within the `chiseltest`. Each test-vector was declared as a tuple containing the following fields: (opcode, Rs1, Rs2, expected Rd, expected `vxsat`).

```
//=====
//PSSUB.H -- SIMD 16-bit Signed Saturating Subtraction
//=====
// Rs1 = 0x3F2E_5D4C -> [0x3F2E, 0x5D4C] (Decimal: [16174, 23884])
// Rs2 = 0x1A1B_2C3D -> [0x1A1B, 0x2C3D] (Decimal: [6683, 11325])
// Expected Rd = 0x2513_310F -> [0x2513, 0x310F] (Decimal: [9491, 12559])
// vxsat = 0x00000000 (No saturation occurred)
(ALUOps.PSSUBH, "h3F2E_5D4C".U, "h1A1B_2C3D".U, "h2513_310F".U, "h00000000".U), // Status register updates to 0
// Rs1 = 0x7FFF_8000 -> [0x7FFF, 0x8000] (Decimal: [32767, -32768])
// Rs2 = 0x8000_7FFF -> [0x8000, 0x7FFF] (Decimal: [-32768, 32767])
// Expected Rd = 0x7FFF_8000 -> [0x7FFF, 0x8000] (Decimal: [32767, -32768])
// vxsat = 0x00000001 (Overflow occurred)
(ALUOps.PSSUBH, "h7FFF_8000".U, "h8000_7FFF".U, "h7FFF_8000".U, "h00000001".U), // Status register updates to 1
// Rs1 = 0x4000_8001 -> [0x4000, 0x8001] (Decimal: [16384, -32767])
// Rs2 = 0x2000_0001 -> [0x2000, 0x0001] (Decimal: [8192, 1])
// Expected Rd = 0x2000_8000 -> [0x2000, 0x8000] (Decimal: [8192, -32768])
// vxsat = 0x00000000 (No overflow occurred)
(ALUOps.PSSUBH, "h4000_8001".U, "h2000_0001".U, "h2000_8000".U, "h00000000".U), // Status register updates to 0
```

Figure 28: 16-bit Saturating Addition test cases in `chiseltest`.

Fig. 29 shows the `peek poke` construct used in `chiseltest`. A `for` loop was used to iterate over each entry, where the inputs were applied using `poke`, the clock was advanced by one cycle, and `expect` assertions were used to compare the ALU's `Rd` and `vxsat` outputs against the expected values. Any mismatch aborted the test and reported the failing opcode and operands. For improved and interactive debugging, additional `println` statements were included.

As the P-Extension ALU included one CSR for capturing overflow information and as well as the accumulator register, a wrapper module surrounded the DUT. This wrapper instantiated the actual P-Extension ALU and provided two registers emulating the architectural accumulator and the single-bit `vxsat` CSR, and feeds the prior-cycle values back into the ALU. This mirrored the real pipeline feedback without involving the full core and let the testbench observe both the result (`Rd`) and the updated overflow flag in the same cycle.

```

// Test each case
for ((operation, rs1, rs2, expectedRd, expectedVxsat) <- testCases) {
  // Apply inputs
  dut.io.Rs1.poke(rs1)
  dut.io.Rs2.poke(rs2)
  dut.io.operation.poke(operation)

  // Step the clock and check the result
  dut.clock.step(1)

  // Expect values
  dut.io.Rd.expect(expectedRd, s"Failed on operation $operation with Rs1=$rs1, Rs2=$rs2")
  dut.io.vxsat.expect(expectedVxsat, s"Overflow flag failed on operation $operation with Rs1=$rs1, Rs2=$rs2")

  // Log information for clarity
  println(s"Tested $operation with Rs1 = ${rs1.litValue}, Rs2 = ${rs2.litValue}")
  println(s"Expected Rd = ${expectedRd.litValue}, Got Rd = ${dut.io.Rd.peek().litValue}")
  println(s"Expected vxsat = ${expectedVxsat.litValue}, Got vxsat = ${dut.io.vxsat.peek().litValue}")

  // Extra step for waveform clarity
  dut.clock.step(1)
}

```

Figure 29: Peek poke chiseltest used for testing the P-Extension instructions.

3.3 Integration with A-Core

After the ALU was designed and tested in Chisel environment, the internals of the A-Core processor were studied such that the P-Extension can be integrated with A-Core. An A-Core Chisel guide available here [63] explains the flow of development within the A-Core project. The process of integration involved modifying library files and various functional units of the processor such as decoder, control, register file, ALU. The changes have been pushed under the branch `dev/pextalu` in the A-Core project [64]. For ease of understanding, the modification performed have been categorised under different heading as follows:

Modifying `a_core_common` Scala library

`a_core_common.scala` and `a_core_config.scala` were modified in this Scala library. The former is a foundational source file for the ACore RISC-V processor, providing shared type definitions, enums, instruction formats, and configuration utilities essential for core design and simulation. Specifically four key modifications were made for P-Extension integration

Fig. 30 shows the listings with the four primary changes that were done in the `a_core_common.scala` file. Firstly, a new `PExtOp` Chisel enumeration was introduced to represent all the implemented P-Extension instructions. These custom instructions are mapped into the 0xb1110111 opcode space as specified in the P-Extension specification. This opcode space has been named as `reserved_3` in the `a_core_common.scala` file. Secondly, the ALU result selection logic was extended by introducing a new `PEXT_OUT` option to the `AluBlockResultSrc` enumeration. This addition allows the processor pipeline to route packed SIMD results from the P-extension unit alongside existing outputs from the integer ALU, multiply/divide, and floating-point units. Thirdly, a new case 'p' for P-Extension was added to the `ISAExtension trait` and associated ISA string parsing logic so as to recognise and process the 'P' extension. Lastly, the construction of the `misa` register value was updated to set the corresponding bit (bit 15) when the P-extension is present.

```

424 /* Enumeration of P-Extension instructions */
425 object PExtOp extends ChiselEnum {
426   val PADDB,
427     PAADDDB,
428     PAADDUB,
429     PSADDUB,
430     PSUBB,

```

(a)

```

577 object ALUBlockResultSrc extends ChiselEnum {
578   val ALU_OUT = Value(0.U)
579   val MULT_OUT = Value(1.U)
580   val SPFP_OUT = Value(2.U)
581   val PEXT_OUT = Value(3.U)
582 }

```

(b)

```

697 sealed trait ISAExtension
698 /** Enumeration of RISC-V ISA extensions */
699 object ISAExtension {
700   case object M extends ISAExtension
701   case object A extends ISAExtension
702   case object F extends ISAExtension
703   case object D extends ISAExtension
704   case object G extends ISAExtension
705   case object C extends ISAExtension
706   case object P extends ISAExtension

```

(c)

```

794 for (e <- exts) {
795   e match {
796     case ISAExtension.A => misa |= (1 << 0)
797     case ISAExtension.C => misa |= (1 << 2)
798     case ISAExtension.D => misa |= (1 << 3)
799     case ISAExtension.F => misa |= (1 << 5)
800     case ISAExtension.M => misa |= (1 << 12)
801     case ISAExtension.P => misa |= (1 << 15)

```

(d)

Figure 30: Listings displaying `a_core_common` modifications.

As for `a_core_config.scala`, this file defines the configuration system for the A-Core processor, enabling the specification of key architectural parameters such as data and address widths, memory sizes, cache behaviour, supported ISA extensions, and DMA capabilities. Three key modifications were made: P-extension was added to the list of recognized ISA extensions, its presence was validated during ISA string parsing, and the logic constructing the `misa` register was updated to correctly set the corresponding bit for the P-extension. These changes enable the processor to be configured with P-Extension support using a simple ISA string such as `rv32imp`, ensuring that the feature is enabled and integrated into the system during elaboration or synthesis.

Modifying ALU block

The P-Extension was introduced by adding a new Chisel source file (`PExtALU.scala`) in the `alublock` repository located inside the `ACoreBase` project of A-Core. The `alublock.scala` is the top-level ALU block having `ALUBlock` as the main class. A new P-extension I/O bundle (`ALUBlockPIO`), patterned after the existing `ALUBlockFIO` bundle for RV32F, was added. Table V lists the new signals that were added in the new bundle. The bundle cleanly packages datapath and control signal that are unique to the P-Extension. These are also added as new ports to the top-level ALU block.

Table V: New signal bundle for P-Extension ALU.

Category	Signals	Purpose
Datapath in	<code>vxsat_in</code> (1 bit)	Previous cycle's overflow flag, forwarded from the CSR so the ALU can propagate or clear it.
	<code>acc</code> (32 bit)	Accumulator operand required by MAC instructions such as <code>SMAQA</code> .
Control in	<code>pext_op</code> : <code>PExtOp()</code>	One-hot decode of the P-operation (e.g., <code>PADDB</code> , <code>PAADDDB</code> , <code>PABSH</code>).
	<code>pext_en</code> : <code>Bool</code>	Global enable that gates the P slice when the extension is disabled at run-time.
Datapath out	<code>vxsat_out</code> (1 bit)	Updated overflow flag, routed back to the CSR block.

P-Extension ALU sits as a child module inside the main ALUBlock such that it is only elaborated when the core's configuration string includes the 'p' extension. The *pext_en* enable signal is added which gates the operation of the P-extension ALU at run-time, controlling whether it is actively driven during a given cycle. This signal is sourced from the bit 15 of the *misa* register. A new *PEXT_OUT* case was also added to the existing *block_result_src* multiplexer so that for a P-Extension instruction, the output of P-Extension ALU is steered to the shared *result* bus of the core. Fig. 31 shows the top-level wiring of P-Extension ALU within the A-Core's ALU. The existing and the newly defined P-bundle signals were connected directly to the *PExtALU* ports while the 32-bit output (Rd) was routed to the shared *result* bus via the existing multiplexer when *PEXT_OUT* is selected.

```

308 if (config.extension_set.contains(ISAExtension.P)) {
309   when (io.in.control.p_ext.pext_en) {
310     // PExtALU connections
311     pext_alu.get.io.operation      := io.in.control.p_ext.pext_op
312     pext_alu.get.io.Rs1           := operand_a
313     pext_alu.get.io.Rs2           := operand_b
314     pext_alu.get.io.acc           := io.in.data.p_ext.acc
315     pext_alu.get.io.vxsat_in      := io.in.data.p_ext.vxsat_in
316     io.out.data.p_ext.vxsat_out   := pext_alu.get.io.vxsat_out
317
318     // select final result from ALU or PExtALU
319     when (io.in.control.block_result_src === AluBlockResultSrc.PEXT_OUT) {
320       io.out.data.result := pext_alu.get.io.Rd
321     }

```

Figure 31: ALU block connections with P-Extension ALU submodule.

Modifying Register file block

The MAC instructions accumulate results into the destination register, which implies that the current value stored in the destination register must be accessible to the ALU prior to execution. Consequently, three registers viz rs1, rs2, and rd need to be read for P-Extension MAC operations. While the A-Core implementation supports reading three registers, it originally assumed that the third register would always be sourced from the floating-point register file. To support the P-extension, this behaviour had to be modified: when the P-extension is enabled, rs3 should read rd and all three registers required for MAC instructions must be read from the integer register file.

Fig. 32 shows the addition of a third combinational read port to the *IntRegFile*. Consequently, the associated I/O bundle was extended to include *read_rd_addr* and *read_rd_data* signals. The internal integer register file array *x* is accessed using this *read_rd_addr*, enabling the ALU to retrieve the prior value of the destination register needed for accumulation.

Fig. 33 shows the changes that were made in the register file block associated with forwarding of *rs3* register in case of P-Extension. A-Core includes a forwarding unit to resolve data hazards by allowing data destined for the register file to be forwarded to instructions currently in the ID stage. However, the existing forwarding


```

19 // Read destination register
20 val read_rd_addr = Input(UInt(5.W))
21 val read_rd_data = Output(UInt(dataWidth.W))

33 // connect read ports to registers
34 io.read_a_data := x(io.read_a_addr)
35 io.read_b_data := x(io.read_b_addr)
36 io.read_rd_data := x(io.read_rd_addr)
37
38 when (io.write_en) {
39   when (io.write_addr > 0.U) {
40     x(io.write_addr) := io.write_data
41   }
42 }

```

Figure 32: Adding Rs3 and reading the destination register.

logic for rs3 was hardcoded to assume that the source and destination was always the floating-point register file. To support the P-extension MAC instructions, which use the integer register file for rs3, this logic was modified to accommodate selectable register sources. A new control signal, *rs3_out_src*, was introduced which was generated and propagated by the Control unit alongside the existing *rs1_out_src*, *rs2_out_src* indicators. This new signal now indicated the origin of the rs3 data, based on which a flag (*same_regfile_rs3*) was generated which ensured forwarding to only occur when the source and destination register files for rs3 matched.

```

51 // Control signals from Control
52 val control = new Bundle {
53   // Select register file to read from
54   val rs1_out_src = RegFileSelect()
55   val rs2_out_src = RegFileSelect()
56   val rs3_out_src = RegFileSelect()

```

(a) New port for source of rs3

```

170 if (config.extension_set.contains(ISAExtension.P)){
171   io.readOut.data.rs3_rdata := regfile.io.read_rd_data
172 }

```

(b) Enabling rs3 read from rd in case of P-Extension

```

175 // Forwarding data if addresses are the same
176 val same_regfile_rs1 = io.readIn.control.rs1_out_src === io.writeIn.control.wr_reg_select
177 val same_regfile_rs2 = io.readIn.control.rs2_out_src === io.writeIn.control.wr_reg_select
178 val same_regfile_rs3 = io.readIn.control.rs3_out_src === io.writeIn.control.wr_reg_select
179 when ((io.readIn.data.rs1_addr === io.writeIn.data.rd_waddr) && same_regfile_rs1 && io.writeIn.control.wr_en) {
180   io.readOut.data.rs1_rdata := rd_wdata
181 }
182
183 when ((io.readIn.data.rs2_addr === io.writeIn.data.rd_waddr) && same_regfile_rs2 && io.writeIn.control.wr_en) {
184   io.readOut.data.rs2_rdata := rd_wdata
185 }
186
187 when ((io.readIn.data.rs3_addr === io.writeIn.data.rd_waddr) && same_regfile_rs3 && io.writeIn.control.wr_en) {
188   io.readOut.data.rs3_rdata := rd_wdata
189 }
---
```

(c) Flag generation and forwarding based on the source and destination register files

Figure 33: Modifications in register file block.

Modifying Control and Status Register block

The overflow tracking logic for the new CSR register, *vxsat*, follows a similar implementation pattern as the CSR logic used for the F-extension. Three files were modified in total: *controlstatusregs*, *csrex*, *controlstatusblock*.

Fig. 34(c) shows the new 32-bit *vxsat* register that was added in the *controlstatusregs* file. Fig. 34(a),(b) show two input ports, *ov_en* (an overflow enable flag) and *vxsat_out*, and one output port, *p_en*, that were introduced in the I/O definitions of the same file. Fig. 34(f) shows the branch where the *ov_en* flag is used

```

(a) 33    // Pext status port
    34    val ov_en      = Input(Bool())
    35    val vxsat_out  = Input(UInt(1.W))

(b) 52    val f_en      = Output(Bool())
    53    val p_en      = Output(Bool())

(c) 63    val fcsr      = RegInit(0.U(32.W))
    64    val vxsat      = RegInit(0.U(32.W))

(d) 205    io.f_en := misaReg(5)
    206    io.p_en := misaReg(15)

(e) 220    } .elsewhen (io.csr_waddr === "h009".U) {
    221        vxsat := Cat(0.U(31.W), io.write_data(0))

(f) 265    } .elsewhen(io.ov_en) {
    266        vxsat := (vxsat(0) | io.vxsat_out).pad(32)

(g) 276    } .elsewhen (io.csr_raddr === "h009".U) {
    277        io.read_data := Cat(0.U(31.W), vxsat(0))

```

Figure 34: Modified listings in *controlstatusregs* file.

for enabling the overflow tracking for saturation-producing P-extension instructions. This flag is originally generated by the control unit during the decoding of P-extension instructions and further routed to this block. When the flag is asserted (i.e., for saturation-producing instructions), it enables the update of the *vxsat* register using the *vxsat_out* signal which carries the overflow information from the P-Extension ALU. The overflow flag is made sticky using a logical OR operation between its existing value and the incoming *vxsat_out* signal from the P-extension ALU.

Reading and writing to the *vxsat* CSR is implemented by mapping CSR address 0x009 to the *vxsat* register, Fig. 34(e), (g). Additionally, bit 15 of the *misa* register is assigned to the *p_en* output port, Fig. 34(d). This bit serves as the same gate-enable signal used in the top-level ALU to activate the P-extension ALU. Forwarding logic for this bit is also required and is implemented in the top-level ACOREBase, discussed later.

As for the *csrex* modifications, since the OV flag in the *vxsat* register is implemented as sticky, a CSR write operation would be required to clear the flag and begin from a clean state. In the *csrex* file, writing to the *vxsat* CSR was enabled by mapping the CSR address 0x009 to the *vxsat* register. The existing logic for writing to various

other CSRs was simply extended to vxsat CSR. Fig. 35 shows handling *misa* register bit for P-Extension inside the *csrex* file. The existing *parseMisa* function, which already handled bit 5 for the F-extension, was extended to allow software to toggle bit 15 of the *misa* register, provided the core is built with P-extension support.

```

125 def parseMisa(misa: UInt) : UInt = {
126   val default_misa = config.default_misa.asUInt
127
128   //F extension
129   val bit5 = if (config.extension_set.contains(ISAExtension.F)) prepared_data(5) else default_misa(5)
130   //P extension
131   val bit15 = if (config.extension_set.contains(ISAExtension.P)) prepared_data(15) else default_misa(15)
132   Cat( default_misa(31, 16), bit15, default_misa(14, 6), bit5, default_misa(4, 0) )
133 }
```

Figure 35: Modified listing in *csrex* file.

The top-level wiring and instantiation of the child modules are handled within the *controlstatusblock*. The ports related to overflow enable (ov_en), overflow data from the P-Extension ALU (vxsat_out), and the P-extension enable signal (p_en) were declared as part of separate I/O bundles.

Modifying Pipeline Controller block

The *pipelinecontroller* unit houses the forwarding support in case of hazards. Fig. 36 shows the modified listings in the *pipelinecontroller* file. The forwarding logic for P-

```

72 // P-extension overflow flag
(a) 73 val ovflag = new PipelineBundle(UInt(1.W))
74 // P-extension instruction generating overflow flag
75 val op_gens_ovflag = new PipelineBundle(Bool())

308 val ovflag_vec = VecInit(io.in.ovflag.ex_mema, io.in.ovflag.mema_memd, io.in.ovflag.memd_wb)
(b) 309 val op_gens_ovflag_vec = VecInit(io.in.op_gens_ovflag.ex_mema, io.in.op_gens_ovflag.mema_memd,
   io.in.op_gens_ovflag.memd_wb)

328 i = 0
329 val op_modifies_ovflag = VecInit.fill(ovflag_vec.length) {
(c) 330 val res = op_gens_ovflag_vec(i) || ( csr_wr_en_vec(i) && (csr_dest_addr_vec(i) === "h009".U) )
331   i += 1
332   res
333 }

380 // PEXT addition
381 // Reading from vxsat AND overflow flags are modified(=either by P-ext instruction or CSR write instruction)
382 } .elsewhen ( ( csr_src_addr === "h009".U ) && op_modifies_ovflag_exists ) {
383   // Overflow flag forwarding - Saturating P-extension instruction in the later pipeline stage generating new flag info
(d) 384   when ( op_gens_ovflag_vec(op_modifies_ovflag_idx) ) {
385     // Forward data comes from later pipeline stage
386     io.out.csr_forward_data := Cat( csr_src_data(31,1), ovflag_vec(op_modifies_ovflag_idx) )
387   } .otherwise {
388     // Forward data comes from CSR rdata
389     io.out.csr_forward_data := Cat( csr_src_data(31,1), csr_wdata_vec(op_modifies_ovflag_idx)(0) )
390   }
391   io.out.forward_csr := true.B
---
```

Figure 36: Modified listings for *pipelinecontroller* block.

extension's saturation flag (vxsat CSR) was added here. The logic adopted the same approach used for the F-extension's *fflags*. New CSR data hazard signal were added across the pipeline stages, Fig. 36 (a). Vectors are used to track both the overflow flag values (ovflag_vec) and whether each instruction in the EX/MEM, MEM/MEMD,

and MEMD/WB pipeline stages modifies the overflow flag (`op_gens_ovflag_vec`), Fig. 36 (b). The logic handles cases when overflow flag gets modified either as a result of a saturating P-extension operation or a CSR write to the *vxsat* register, Fig. 36 (c).

Fig. 36 (d) shows the listing for overflow flag forwarding and CSR write handling for *vxsat*. For each of the three pipeline stages, the vector `op_modifies_ovflag` is constructed to indicate whether the overflow flag will be updated. This is done using the logical OR of `op_gens_ovflag_vec(i)` and a CSR write enable targeting the *vxsat* CSR. When a CSR read operation addresses the *vxsat* register (`csr_src_addr === "h009".U`), the logic checks if any instruction in the pipeline is scheduled to modify the overflow flag using the `op_modifies_ovflag_exists` and `op_modifies_ovflag_idx` signals. If such an instruction is present, the most recent overflow flag value is forwarded from the relevant pipeline stage using `ovflag_vec(op_modifies_ovflag_idx)`, ensuring that `io.out.csr_forward_data` reflects the latest flag state. If the modifying instruction is instead a CSR write to *vxsat*, the forwarded data is taken from the CSR write data vector at the same pipeline stage, with the upper 31-bits left unchanged.

Modifying Decoder and Control block

Although this block is one unit of the processor as shown in Fig. 18, it has been split into two separate entities in the A-Core project repository: *decoderblock* and *control.scala*.

Within the *decoderblock*, *instrdecoder.scala* file assigns the instruction-type to the opcode space of the A-Core processor. Although most of the P-extension operations are decoded as R-type instructions, clipping instructions are classified as I-type due to their use of the *rs2* field as an immediate operand. As such, a new decode branch was introduced to handle this opcode space, with instruction-type differentiation based on the *funct7* and *funct3* fields: clipping instructions are classified as I-type, while all other P-extension instructions default to the R-type format.

Fig. 37 shows the modified control logic for P-extension instruction parsing and *rs3* operand sourcing. The parsing of P-Extension instructions was introduced in

```

644 // P-Extension
645 }.elsewhen (io.opcode === Opcode.reserved_3) {
646   if (config.extension_set.contains(ISAExtension.P)) {
(a) 647     io.regfile_block_write.wr_en      := true.B
648     io.rd_src                         := RegFileSrc.ALU_RESULT
649     io.regfile_block_write.wr_reg_select := RegFileSelect.INT
650     io.regfile_block_read.rs1_out_src  := RegFileSelect.INT
651     io.regfile_block_read.rs2_out_src  := RegFileSelect.INT
652     io.regfile_block_read.rs3_out_src  := RegFileSelect.INT

766   }.elsewhen ( io.funct7 === "b1000010".U && io.funct3 === "b000".U && io.rs2(4) === "b0".U ) { //SCLIP16
(b) 767     io.alu_block.operand_a_src      := AluOperandSrc.RS1
768     io.alu_block.operand_b_src      := AluOperandSrc.IMM
769     io.alu_block.p_ext.pext_op      := PExtOp.PCLIPH
770     io.csr_block_write.ov_en        := true.B

```

Figure 37: Modifications in *control* file.

the control file. A new conditional branch was added which used *funct7*, *funct3* and *rs2* instruction fields and generated the necessary control signals that are used by

various other units within the processor. The source file for rs3 operand was also generated here which was mentioned earlier while discussing the modifications in Register File block.

While performing modifications in the control unit, it was observed that the existing implementation did not forward the rs2 field from the decoder block. However, since the P-extension relies on the rs2 field for decoding certain instructions, for example absolute value instructions, a new input *val rs2* was declared in the I/O definitions to make this field available for instruction parsing.

Modifying the top-level ACoreBase

This module serves as the top-level entry point for the A-Core implementation in Chisel and encapsulates the pipeline stages of the core. Modifications were made to the ID, EX, and WB stages to enable support for the RISC-V P-extension.

Fig. 38 shows the ID stage modifications inside the ACoreBse file. The rs2 field was routed from the decoder to the control block to enable proper instruction parsing, as discussed in previous section, Fig. 38 (a). Forwarding support was also added for the current status of the P-extension enable bit (*p_en*) across the pipeline. If the pipeline controller (*ppl_ctrl*) indicates that the *misa* register is being forwarded, it takes the enable bit from the forwarded *misa* data; otherwise, it uses the value from the CSR register block, Fig. 38 (b). The related mechanisms were discussed in CSR and Pipeline controller blocks. Additionally, the rs3 register address was also modified to support selection between integer and floating-point register files, depending on the instruction-type, Fig. 38 (c)(d).

```

294 control.io.rd      := decoder_block.io.rd
295 control.io.rs1     := decoder_block.io.rs1
(a) 296 control.io.rs2     := decoder_block.io.rs2
297 control.io.opcode  := decoder_block.io.opcode

314 // Forwarding P-extension enable bit
315 val current_p_en = Mux(
316     ppl_ctrl.io.out.forward_misa,
(b) 317     ppl_ctrl.io.out.misa_forward_data(15),
318     csreg_block.io.dataOut.p_en
319 )

329 // Extension-specific
330 regfile_block.io.readIn.data.rs3_addr := Mux(
331     control.io.regfile_block_read.rs3_out_src == RegFileSelect.FLOAT,
(c) 332     decoder_block.io.f_ext.rs3,
333     decoder_block.io.rd
334 )

340 id_ex_next.rs1     := decoder_block.io.rs1
341 id_ex_next.rs2     := decoder_block.io.rs2
342 id_ex_next.rs3     := Mux(
343     control.io.regfile_block_read.rs3_out_src == RegFileSelect.FLOAT,
(d) 344     decoder_block.io.f_ext.rs3,
345     decoder_block.io.rd)
346 id_ex_next.rd      := decoder_block.io.rd

```

Figure 38: Modified listings in ID stage.

Fig. 39 shows the EX stage datapath modifications. The datapath was enhanced to connect the P-extension ALU inputs and control signals. The saturation flag input (vxsat_in) was routed directly from the CSR block to the ALU. The P-extension enable signal (pext_en) was passed from the ID stage to activate the P-extension ALU unit. The accumulator input (acc) was sourced from the rs3 register data.

```

456 alu_block.io.in.data.p_ext.vxsat_in    := csreg_block.io.data0Out.csr_rdata(0)
457 alu_block.io.in.control.p_ext.pext_en  := id_ex_reg.p_en
458 alu_block.io.in.data.p_ext.acc         := id_ex_reg.rs3_rdata

```

Figure 39: Modified listing in EX stage.

Fig. 40 shows the WB stage modifications, managing rs3 routing and saturation flag forwarding. In the WB stage, the saturation flag output (vxsat_out) from the P-extension ALU is written back to the CSR block Fig. 40 (a). The remaining data signals are implicitly routed using the existing pipeline register paths. Lastly, pipeline controller was also enhanced to manage the routing and selection of the rs3 source register across the pipeline, Fig. 40 (b), and to track and forward saturation flag of vxsat CSR using the newly added pipeline registers, Fig. 40 (c).

```

663 csreg_block.io.writeIn.data.csr_waddr := memd_wb_reg.csr_waddr
664 csreg_block.io.writeIn.data.csr_wdata := memd_wb_reg.csr_wdata
665 csreg_block.io.writeIn.data.fault_data := memd_wb_reg.alu_data.f_ext.fault_data
666 csreg_block.io.writeIn.data.vxsat_out  := memd_wb_reg.alu_data.p_ext.vxsat_out

```

(a)

```

701 // Pipeline Controller connections for register and CSR forwarding
702 ppl_ctrl.io.in.rs1_addr.id_ex := decoder_block.io.rs1
703 ppl_ctrl.io.in.rs2_addr.id_ex := decoder_block.io.rs2
704 ppl_ctrl.io.in.rs3_addr.id_ex := Mux(
705     control.io.regfile_block_read.rs3_out_src == RegFileSelect.FLOAT,
706     decoder_block.io.f_ext.rs3,
707     decoder_block.io.rd)
708 ppl_ctrl.io.in.rs1_select.id_ex := control.io.regfile_block_read.rs1_out_src
709 ppl_ctrl.io.in.rs2_select.id_ex := control.io.regfile_block_read.rs2_out_src
710 // rs3 can come only from float regfile. EDIT: based on the current instruction reg file src
711 ppl_ctrl.io.in.rs3_select.id_ex := control.io.regfile_block_read.rs3_out_src |

```

(b)

```

755 ppl_ctrl.io.in.ovflag.ex_mema := ex_mema_next.alu_data.p_ext.vxsat_out
756 ppl_ctrl.io.in.ovflag.mema_memd := mema_memd_next.alu_data.p_ext.vxsat_out
757 ppl_ctrl.io.in.ovflag.memd_wb := memd_wb_next.alu_data.p_ext.vxsat_out
758 ppl_ctrl.io.in.op_gens_ovflag.ex_mema := ex_mema_next.csrblock_write_control.ov_en
759 ppl_ctrl.io.in.op_gens_ovflag.mema_memd := mema_memd_next.csrblock_write_control.ov_en
760 ppl_ctrl.io.in.op_gens_ovflag.memd_wb := memd_wb_next.csrblock_write_control.ov_en

```

(c)

Figure 40: Modified listings in WB stage and pipeline controller.

3.4 Compiler support for P-Extension instructions

After integrating the design with A-Core, the next objective was to first develop simple assembly programs and subsequently write more complex programs utilising the P-extension instructions, with the ultimate goal of benchmarking their performance benefits compared to regular sequential execution. However, at present, there is no official compiler support for the P-extension. This limitation implies that programs written using P-extension instructions cannot be directly compiled into RISC-V machine code. There were two possible approaches to developing the assembly programs: (1) manually writing the instructions by directly specifying their

binary encodings, or (2) modifying the compiler to support the newly implemented instructions, allowing them to be written using standard assembly mnemonics.

The second approach was chosen for this thesis due to various advantages. Firstly, this approach leads to human-readable program which clearly communicates the intent, especially useful when writing programs with many P-Extension instructions. Secondly, it enables faster development and offers a seamless integration with C using in-line assembly code. Thirdly, this approach ensures the infrastructure is future-proof, allowing more complex programs to be developed using the implemented instructions without requiring any changes to be made to compilation toolchain.

To enable compiler support for the custom P-extension instructions, the RISC-V GNU toolchain was cloned with the target of adding support for custom P-extension instructions so they can be written in C code (using inline assembly) and correctly be assembled and disassembled by the toolchain. Specifically, the *binutils* component of the toolchain was targeted for modification, as it contains both the assembler (*gas*) and the disassembler (*objdump*). The assembler is responsible for reading *.S* assembly files and generating *.o* object binaries, while the disassembler performs the inverse operation, translating *.o* or executable binaries back into human-readable assembly instructions. The community-driven resources [65] [66] [67] provided valuable insight into the structure and integration strategy for custom instruction support within the RISC-V GNU toolchain. The *binutils* patches are pushed to this [68] branch for reference and the modifications that were performed are discussed under the following headings:

Defining MASK and MATCH values

Fig. 41(a) illustrates how the MASK value is used to identify the instruction by ignoring variable fields, using 8-bit addition instruction PADDB.

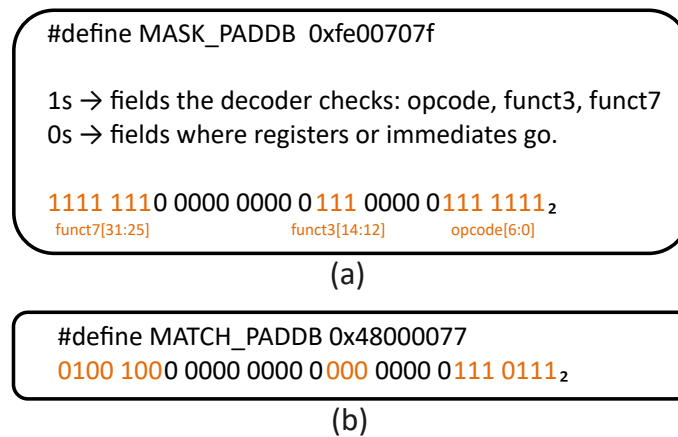


Figure 41: MASK (a) and MATCH (b) values for PADDB instruction.

The MASK value is a 32-bit bitmask used to determine which bits of an instruction are significant when identifying its type during assembly and disassembly. It enables the toolchain to ignore variable fields such as *rs1*, *rs2*, or *rd*, which vary between

different instruction. The MASK is instruction-type specific. For example, R-type instructions use a MASK value of 0xfe00707f, where the bits corresponding to fixed fields such as opcode, funct3, and funct7 are set to 1, and the bits corresponding to register fields (rd, rs1, rs2) are set to 0.

Fig. 41(b) shows the MATCH value used to uniquely identify an instruction based on its fixed fields. The MATCH value is the exact 32-bit binary encoding of an instruction's fixed fields, such as opcode, funct3, funct7, and any other constant bits that uniquely identify the instruction. These fixed bits are defined by the instruction specification (e.g., the RISC-V P-extension draft). For each instruction, the corresponding MASK and MATCH values, along with a DECLARE_INS macro, are defined in a header file. This macro specifies the instruction mnemonic and its encoding, registering it in both the assembler and disassembler tables of the toolchain. These declarations are made in the following header file:

```
riscv-gnu-toolchain/binutils/include/opcode/riscv-opc.h
```

These MASK and MATCH values are utilized by the *match_opcode* function, defined in the file discussed in the next section, to compute a unique identifier for each custom instruction.

Defining Instructions in the Global Opcode Table

Fig. 42 illustrates the global opcode table, which defines the instruction set using a structured set of parameters. Each instruction is added to this table using a specific set of parameters that describe its encoding and behaviour. This definition informs the assembler and disassembler how to associate a binary encoding with a named instruction mnemonic and vice versa. The global opcode table is located in this file:

```
riscv-gnu-toolchain/binutils/opcodes/riscv-opc.c
```

```
const struct riscv_opcode riscv_opcodes[] =
{
/* name, xlen, isa, operands, match, mask, match_func, pinfo. */
/* RVP instructions. */
{"paddb",      0, INSN_CLASS_P, "d,s,t", MATCH_PADDDB, MASK_PADDDB, match_opcode, 0},
{"paaddb",     0, INSN_CLASS_P, "d,s,t", MATCH_PAADDDB, MASK_PAADDDB, match_opcode, 0},
{"paaddub",    0, INSN_CLASS_P, "d,s,t", MATCH_PAADDUB, MASK_PAADDUB, match_opcode, 0},
{"psaddub",    0, INSN_CLASS_P, "d,s,t", MATCH_PSADDUB, MASK_PSADDUB, match_opcode, 0},

```

Figure 42: Global Opcode Table with P-Extension instructions.

Three fields needed careful consideration while adding the entries for P-Extension instructions: name, isa, operands. The *name* field is self-explanatory and specifies the mnemonic of the instruction. The *ISA* and *operands* fields influence conditional inclusion of the instruction and operand parsing as discussed in subsequent paragraphs.

ISA instruction class parameter for new instructions: A new instruction class, `INSN_CLASS_P`, was introduced to categorise all P-extension instructions. Like other extension specific instructions, this mechanism allows the toolchain to selectively recognise and process P-extension instructions based on target configuration. This new class was added in the file listed below:

```
/riscv-gnu-toolchain/binutils/include/opcode/riscv.h
```

Operands parameter for new instructions: This parameter specifies the operand formatting by defining which character codes represent the operands for a given instruction. The meaning and behaviour of these character codes are defined in a configuration file located at:

```
/riscv-gnu-toolchain/binutils/gas/config/tc-riscv.c
```

By examining the main switch statement in this file, it can be determined how each operand character is parsed and processed. Once the appropriate character codes for the P-extension instruction operands were identified, based on the P-Extension specification, they were plugged in the instruction definition. For example, "d,s,t" for destination and two sources representing regular R-Type instruction as shown in Fig. 42.

However some instructions, such as `pcliph`, used unique operand formats (e.g., destination, source, `immediate_4bits`). To ensure that the assembler (gas) recognises such P-Extension instructions, a new case 'p' was defined under the existing case 'X' in the main parsing switch statement. Thus the operand string "d,s,Xpu4@20" was specified in the opcode table which specified destination, source and 4-bit-unsigned-immediate value starting from bit 20 of the 32-bit instruction, as specified in the P-Extension specification.

Two case 'p' constructs were defined in separate operand-parsing stages: one handles the operand format string used to identify RVP-specific immediate operand patterns (e.g., signed or unsigned), while the other is responsible for decoding the actual immediate strings like "Xpu4@20". The logic was adapted from the existing similar V-Extension instructions.

Updating the disassembler to decode custom instructions: The disassembler uses the same global opcode table (`riscv_opcodes[ ]`) to decode and reconstruct the textual representation of P-extension instructions from their binary encoding. The operand template string (e.g., "d,s,t" or "d,s,Xpu4@20") guides how the decoded binary values are mapped back to human-readable register and immediate values. The disassembler source file was updated to include a new case for the 'p' extension, similar to how it was registered in the assembler configuration file. This ensures that when a binary containing P-extension instructions is processed (e.g., via `objdump`), the disassembler correctly interprets their fields, and displays the appropriate mnemonic and operands. The disassembler source file is located here:

```
/riscv-gnu-toolchain/binutils/opcodes/riscv-dis.c
```


Adding extension support and extension dependencies

The P-Extension was registered and its implicit dependencies were added in this file:

```
/riscv-gnu-toolchain/binutils/bfd/elfxx-riscv.c
```

The 'p' extension was configured to automatically include the standard base extensions ('i', 'm', and 'zicsr') required for correct execution. Furthermore, the instruction class `INSN_CLASS_P`, discussed earlier, was associated with the extension letter 'p' and the extension itself was added to the toolchain's internal ISA specification table.

CSR for P-Extension

The cloned RISC-V GNU toolchain already included support for the Vector (V) extension, which also uses the *vxrsat* CSR. Consequently, the *vxrsat* CSR value and its associated `DECLARE_CSR` macro was already defined in the same header file where the `MASK` and `MATCH` values were earlier specified. No additional modifications were done.

Finally, after all the required modifications were completed, the toolchain was compiled with support for the P-extension by including an additional ISA option 'p'. The compilation took almost two hours and this was done using the following configuration command:

```
./configure --prefix=/path/to/toolchain/folder
--with-multilib-generator="rv32imp_zicsr-ilp32--"
```

3.5 Validation testing and assembly code generation

The compiler was successfully built with the modifications to support P-extension instructions, and the A-Core was also compiled with its corresponding P-extension integration. With both components in place, the next step was to validate the integration and correctness of the final design. This was achieved by writing self-contained C test-suite to functionally validate every instruction. This was accomplished by targeting the same random and corner cases used during the initial Chisel-based testing, as discussed in Section 3.2.

Three C preprocessor macros were defined to generate the necessary inline assembly and runtime checks. The macros were categorised by operand types: three-register, two-register, and register-immediate instructions. Fig. 43(a) shows the three-register macro used for testing. Each macro uses inline assembly to execute the P-Extension instruction under test, captures the result in a C variable (`rd`), compares it against an expected value, and updates global counters (for counting successful tests) or prints diagnostic messages in case of a failed test. Fig. 43(b) shows the dedicated test function encapsulating all P-extension instruction tests that was separately written and invoked from the `main()` function.

```

1 // Test macro for 3-operand instructions: op rd, rs1, rs2
2 #define TEST_INSTRUCTION(name, op, rs1_val, rs2_val, expected_rd) \
3 do { \
4     uint32_t rd; \
5     asm volatile ( \
6         op " %0, %1, %2" \
7         : "=r" (rd) \
8         : "r" (rs1_val), "r" (rs2_val) \
9     ); \
10    if (rd == expected_rd) { \
11        tests_passed++; \
12    } else { \
13        tests_failed++; \
14        printf("[FAIL] %s: rs1=0x%08x rs2=0x%08x | Expected 0x%08x Got 0x%08x\n", \
15            name, rs1_val, rs2_val, (unsigned int)expected_rd, (unsigned int)rd); \
16    } \
17 } while (0)

```

(a)

```

1 void test_p_extension_instructions() {
2
3     // Test PADD.B (SIMD 8-bit Addition)
4     TEST_INSTRUCTION("PADD.B", "paddb",
5         0xFE10F0F0, // [0xFE, 0x10, 0xF0, 0xF0] (Decimal: [254, 16, 240, 240])
6         0x01FF1011, // [0x01, 0xFF, 0x10, 0x11] (Decimal: [1, 255, 16, 17])
7         0xFF0F0001); // Result: [0xFF, 0x0F, 0x00, 0x01] (Decimal: [255, 15, 0, 1])
8
9     // Test PADD.B (Signed Case)
10    TEST_INSTRUCTION("PADD.B", "paddb",
11        0x9B47E3B2, // [0x9B, 0x47, 0xE3, 0xB2] (Decimal: [-101, 71, -29, -78])
12        0x2D98C4FF, // [0x2D, 0x98, 0xC4, 0xFF] (Decimal: [45, -104, -60, -1])
13        0xC8DFA7B1); // Result: [0xC8, 0xDF, 0xA7, 0xB1] (Decimal: [-63, -33, -89, -79])
14
15    // Test PADD.B (Unsigned Case)
16    TEST_INSTRUCTION("PADD.B", "paddb",
17        0x9B47E3B2, // [0x9B, 0x47, 0xE3, 0xB2] (Decimal: [155, 71, 227, 178])
18        0x2D98C4FF, // [0x2D, 0x98, 0xC4, 0xFF] (Decimal: [45, 152, 196, 255])
19        0xC8DFA7B1); // Result: [0xC8, 0xDF, 0xA7, 0xB1] (Decimal: [193, 223, 167, 177])
20 }

```

(b)

Figure 43: C-test listings for validating P-Extension instructions.

Since the new instructions were not yet available via compiler intrinsics, inline assembly was used to invoke P-extension instructions within the C test program. The resulting binary was disassembled using `objdump`, and the output was saved to a file for inspection. Fig. 44 shows the terminal output (top), which reports zero failed tests, and the disassembly file (bottom), where the emitted P-Extension mnemonics are correctly recognised. This confirms that the assembler/disassembler round-trip successfully handled the new P-Extension instructions. Together, these outputs validate the *binutils* patches, the integration of the P-extension ALU into A-Core, and the functional correctness of the ALU implementation.

```

Welcome to A-Core!
=== RISC-V P-Extension Instruction Tests ===

Test Summary:
Passed: 76
Failed: 0

```

```

34 1060: 0f060613      addi    a2,a2,240 # fe10f0f0 <__stack_top+0xde0f0f0>
35 1064: 01168693      addi    a3,a3,17 # 1ff1011 <DATAMEM_LENGTH+0x1ef1011>
36 1068: 48d607f7      paddb   a5,a2,a3
37 106c: ff0f0737      lui     a4,0xff0f0
38 1070: 00170713      addi    a4,a4,1 # ff0f0001 <__stack_top+0xdef0001>
39 1074: 3ee790e3      bne     a5,a4,1c54 <test_p_extension_instructions+0xc0c>
40 1078: 8601a783      lw      a5,-1952(gp) # 20000060 <tests_passed>
41 107c: 9b47e637      lui     a2,0x9b47e
42 1080: 2d98c6b7      lui     a3,0x2d98c
43 1084: 00178793      addi    a5,a5,1
44 1088: 86f1a023      sw      a5,-1952(gp) # 20000060 <tests_passed>
45 108c: 3b260613      addi    a2,a2,946 # 9b47e3b2 <__stack_top+0x7b37e3b2>
46 1090: 4ff68693      addi    a3,a3,1279 # 2d98c4ff <__stack_top+0xd88c4ff>
47 1094: 48d607f7      paddb   a5,a2,a3
48 1098: c8dfa737      lui     a4,0xc8dfa
49 109c: 7b170713      addi    a4,a4,1969 # c8dfa7b1 <__stack_top+0xa8cfa7b1>
50 10a0: 3ee79ae3      bne     a5,a4,1c94 <test_p_extension_instructions+0xc4c>
51 10a4: 8601a783      lw      a5,-1952(gp) # 20000060 <tests_passed>
52 10a8: 9b47e637      lui     a2,0x9b47e
53 10ac: 2d98c6b7      lui     a3,0x2d98c
54 10b0: 00178793      addi    a5,a5,1
55 10b4: 86f1a023      sw      a5,-1952(gp) # 20000060 <tests_passed>
56 10b8: 3b260613      addi    a2,a2,946 # 9b47e3b2 <__stack_top+0x7b37e3b2>
57 10bc: 4ff68693      addi    a3,a3,1279 # 2d98c4ff <__stack_top+0xd88c4ff>
58 10c0: 48d607f7      paddb   a5,a2,a3
59 10c4: c8dfa737      lui     a4,0xc8dfa
60 10c8: 7b170713      addi    a4,a4,1969 # c8dfa7b1 <__stack_top+0xa8cfa7b1>

```

Figure 44: (top) Terminal output after running the C-test program, (bottom) Emitted assembly code showing the P-Extension mnemonics.

4 Evaluation and Results

The final phase of this thesis involved identifying representative DSP applications that utilise P-extension instructions and benchmarking their performance advantage over conventional sequential execution. This section presents a selection of such applications, including Dot Product, Matrix Multiplication, FIR Filtering, and the Sobel Edge Detection algorithm. Performance comparisons were carried out by measuring both the cycle counts and the number of retired instructions for two versions of each application: the baseline implementation using regular C kernels and an optimised SIMD variant that leverages P-extension instructions. The tests are located on the `pextalu` branch inside the `A_Core_Software` repository [69].

The cycle count measures the total number of processor clock cycles taken to execute a function. This is captured using the C standard library’s `clock()` function, which returns the processor time consumed by the program. The difference between the end and start times provides a direct measure of the elapsed processor time for the kernel.

The retired instruction count measures the total number of instructions executed (retired) by the processor during the function call. This is obtained using the `get_instret()` function, which reads the value of the `instret` hardware performance counter. The difference between the values before and after the function call yields the number of instructions retired.

Table VI presents the derived performance metrics used alongside the primary measures for a more comprehensive evaluation. As shown in the table, these secondary metrics are computed from the two primary metrics defined above.

Table VI: Derived metrics used for benchmarking performance.

Metric	Formula	Description
Cycle Reduction (%)	$\frac{\text{Cycles}_{\text{SISD}} - \text{Cycles}_{\text{SIMD}}}{\text{Cycles}_{\text{SISD}}} \times 100$	Percentage decrease in total cycles with SIMD
Retired Instr. Reduction (%)	$\frac{\text{Instr}_{\text{SISD}} - \text{Instr}_{\text{SIMD}}}{\text{Instr}_{\text{SISD}}} \times 100$	Percentage decrease in retired instructions with SIMD
Speedup	$\frac{\text{Cycles}_{\text{SISD}}}{\text{Cycles}_{\text{SIMD}}}$	Ratio of scalar cycles to SIMD cycles
IPC (Instructions per Cycle)	$\frac{\text{Retired Instructions}}{\text{Cycle Count}}$	Instructions retired per processor cycle

4.1 Dot Product

The dot product is selected as a benchmark application due to its simplicity and widespread use in scientific computing, computer graphics, and machine learning. Mathematically, the dot product of two n -element vectors $\vec{a}$ and $\vec{b}$ is given by

$$\vec{a} \cdot \vec{b} = \sum_{i=0}^{n-1} a_i \cdot b_i. \quad (1)$$

In case of the scalar loop, one pair of elements per loop iteration is processed where each multiplication and addition is performed as a separate instruction. In each iteration, the SISD kernel takes two input vectors of 8-bit integers, *a* and *b*, and iterates through each element pair using a simple for-loop.

The SIMD kernel uses custom *smaq* instruction, which performs a packed multiply-accumulate operation on four pairs of 8-bit integers in parallel. Within the kernel's main loop, two 32-bit SIMD registers are defined to hold packed blocks of four consecutive 8-bit elements from the input vectors. These packed values are passed to a helper function *mac4_smaq*, which invokes the *smaq* instruction using inline assembly.

Fig. 45 shows the performance comparison between SIMD and scalar dot product implementations in terms of (a) CPU cycles and (b) Retired instructions. By processing four elements per loop iteration, the SIMD implementation reduces the total number of loop iterations and the number of arithmetic instructions required to perform the dot product. This basically leads to the performance improvement compared to SISD implementation. The SIMD dot product kernel was evaluated against its scalar counterpart across a range of vector lengths (8 to 128 elements). In each bar chart, a green inverted triangle, annotates the percentage reduction achieved by the SIMD version compared to its sequential counterpart. These annotations highlight the efficiency gain provided by SIMD kernels. For example, at a vector length of 128 elements, the SIMD implementation achieves a 73% reduction in CPU cycles, and a 77% reduction in retired instructions.

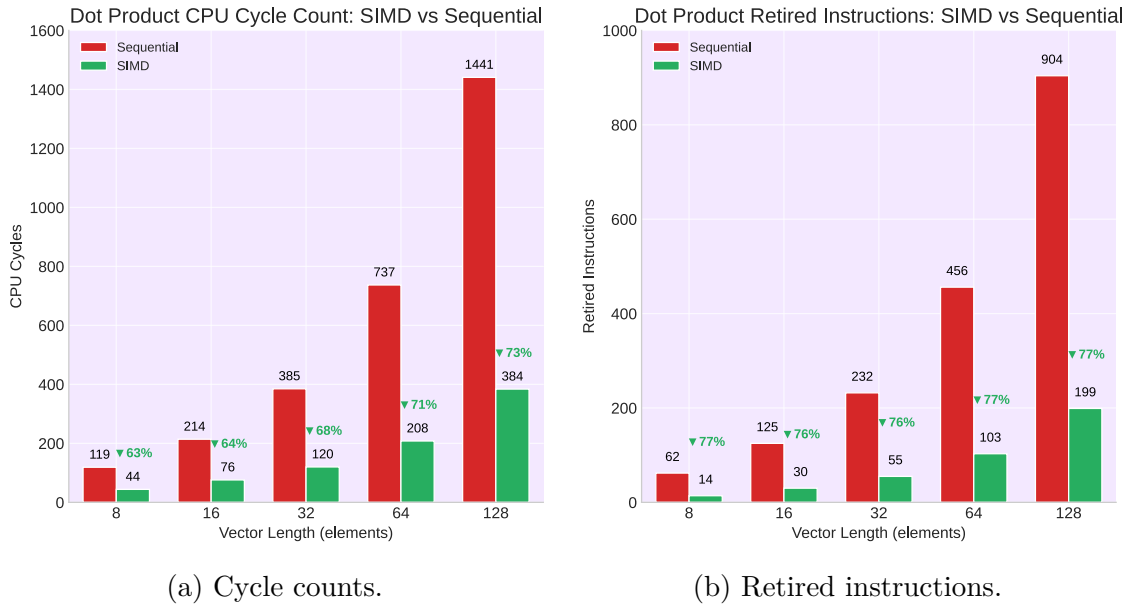


Figure 45: Performance comparison plots between SISD and SIMD dot product kernels.

Plot (a) in Fig. 46 shows that the IPC remains relatively stable across the vector sizes. In SIMD, lesser instructions are issued but more work is done per cycle. Plot

(b) in Fig. 46 shows the speed-up comparison for SISD and SIMD kernels. It can be seen as the vector lengths increase the packing, loop overheads get paid and speed-up is seen rising to theoretical four-fold.

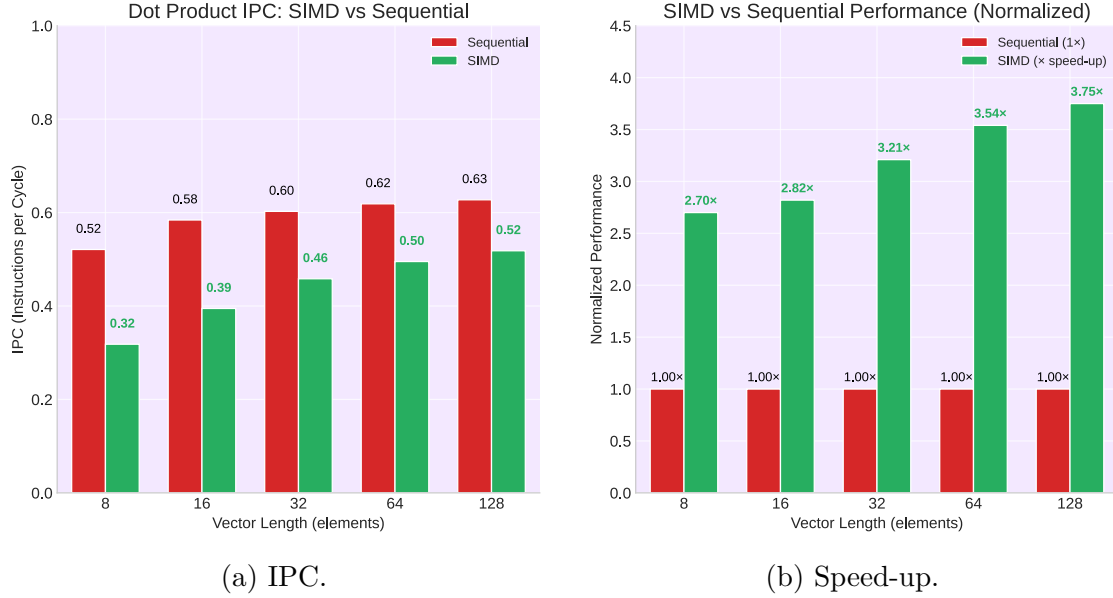


Figure 46: Derived performance metrics for SISD and SIMD dot product kernels.

4.2 Matrix Multiplication

Fig. 47 shows straightforward triple-nested loop implementation of matrix multiplication in C. For each output element C_{ij} , the outer loops fix row i of A and column j of B , while the inner loop computes the dot product by sequentially multiplying and summing $A_{ik} \cdot B_{kj}$ over entire width of A denoted by k . The kernel streams across an entire row of A and an entire column of B for calculating each output, using each element only once in an iteration. As a result, the kernel suffers from poor spatial locality, leading to a large volume of redundant memory accesses.

```

1 for (int i = 0; i < hA; ++i)
2     for (int j = 0; j < wB; ++j) {
3         int32_t s = 0;
4         for (int k = 0; k < wA; ++k)
5             s += (int32_t)A[i * wA + k] * (int32_t)B[k * wB + j];
6         C[i * wB + j] = s;
7     }

```

Figure 47: Triple loop in sequential matrix multiplication.

An optimised sequential matrix-multiplication kernel [70], employs a 4×4 *register-blocking* strategy, in which a dedicated set of registers is used to compute sub-blocks or tiles of the result matrix. Fig. 48(b), (c) show a step-by-step operation of this optimised matrix multiplication using an example of two 8×8 matrices. An 8×8 matrix has four such tiles. Rather than operating on the entire matrices at

once, the kernel processes one 4×4 tile of the output matrix C at a time, thereby markedly improving both temporal and spatial locality when compared with the naïve triple-nested loop.

The two outer loops now progress in increments of four rows of A and four columns of B , while the innermost loop sweeps across the shared dimension k (width of B). During each k -iteration the kernel loads four consecutive elements from the current row of A ($A_0 \dots A_3$) and four from the current column of B ($B_0 \dots B_3$). Consequently, every element of A and B participates in four multiply-accumulate operations before leaving the register file. Sixteen partial sums are held in dedicated registers ($C_{0,0} \dots C_{3,3}$), eliminating redundant memory traffic and improving the performance of matrix multiplication. However, this optimised version is suitable for matrices having shared dimension k as a multiple of 4.

In relation to this thesis, the objective was to optimise the performance of this sequential matrix multiplication by leveraging the use of SIMD execution. Fig. 49 shows how the SIMD kernel accelerates matrix multiplication using the *smaqa* instruction with packed 8-bit data. The SIMD kernel extends the 4×4 blocked scheme of the scalar baseline by exploiting the P-extension's packed-SIMD instruction *smaqa*. In each iteration of the innermost loop, four adjacent 8-bit elements of a row block of A and four 8-bit elements of the corresponding column block of B are packed into 32-bit registers. Because a single *smaqa* performs four independent 8-bit MACs, the loop index k now advances in steps of four, and sixteen accumulators ($C_{0,0} \dots C_{3,3}$) are updated via sixteen parallel four-lane MAC operations per iteration. This is where the SIMD functionality improves the performance of existing matrix multiplication logic.

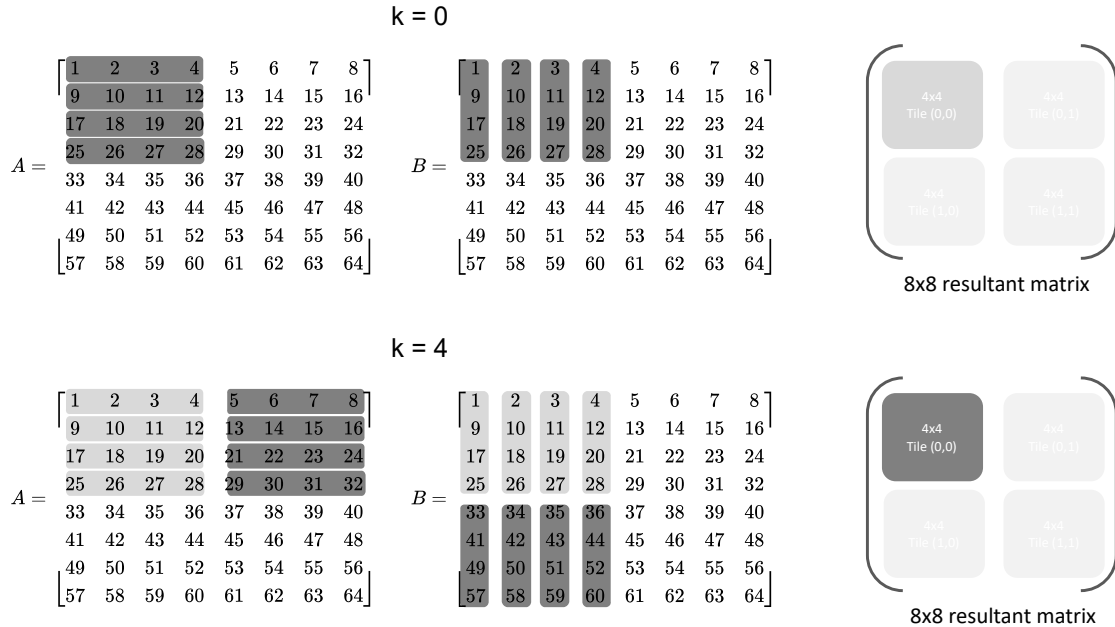
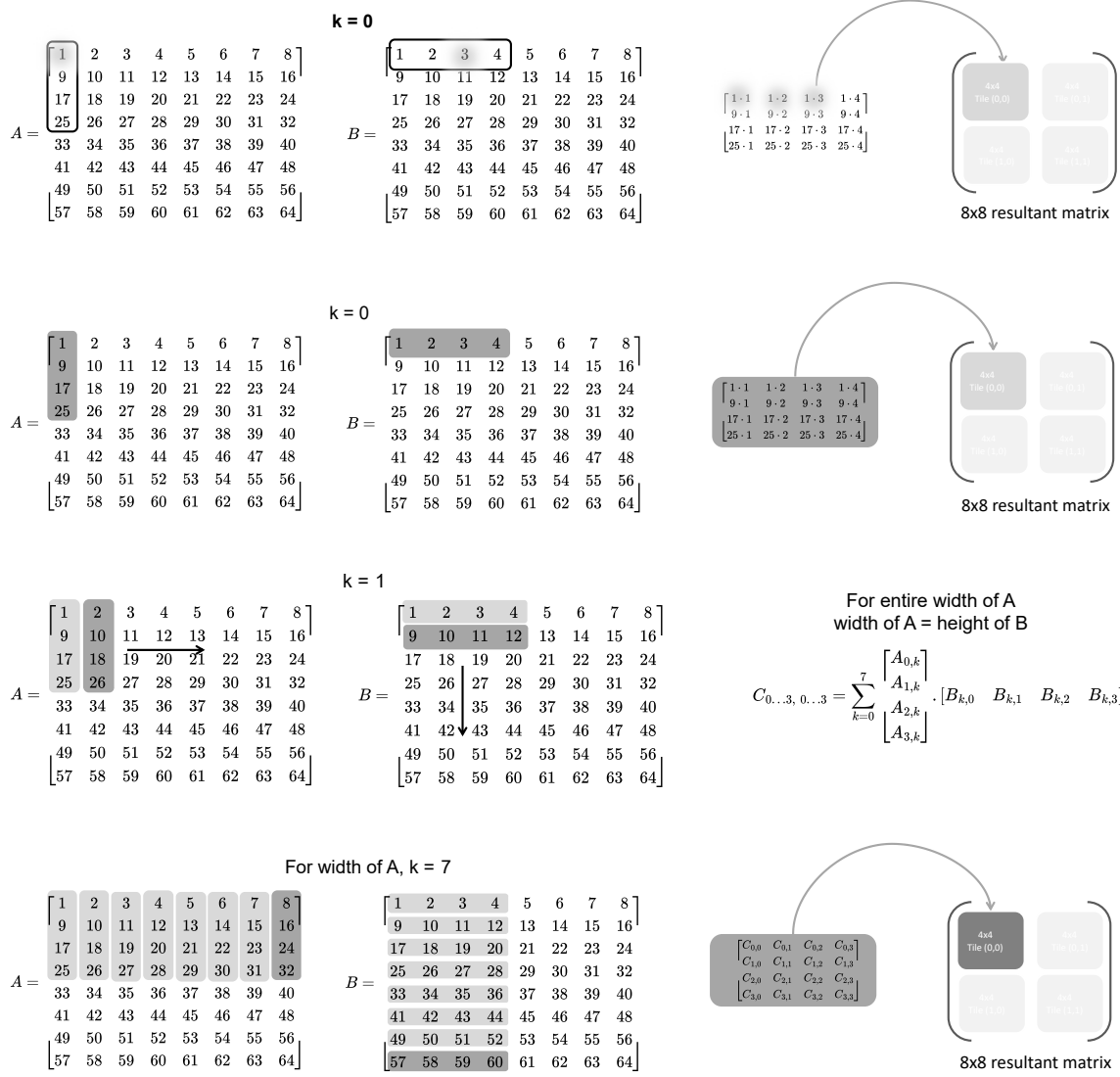


Figure 49: SIMD matrix multiplication.

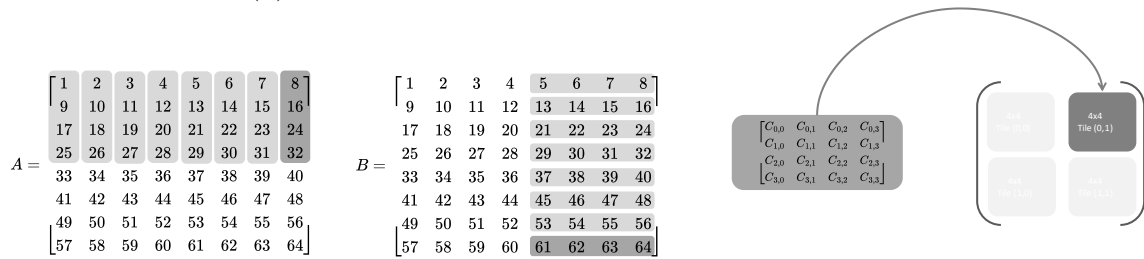
Fig. 50 illustrates the optimised SIMD matrix multiplication. This optimisation

$$\begin{bmatrix} A_0 \\ A_1 \\ A_2 \\ A_3 \end{bmatrix} \cdot [B_0 \ B_1 \ B_2 \ B_3] = \begin{bmatrix} A_0B_0 & A_0B_1 & A_0B_2 & A_0B_3 \\ A_1B_0 & A_1B_1 & A_1B_2 & A_1B_3 \\ A_2B_0 & A_2B_1 & A_2B_2 & A_2B_3 \\ A_3B_0 & A_3B_1 & A_3B_2 & A_3B_3 \end{bmatrix} \longrightarrow \mathbf{C}_{\text{tile}} = \begin{bmatrix} C_{0,0} & C_{0,1} & C_{0,2} & C_{0,3} \\ C_{1,0} & C_{1,1} & C_{1,2} & C_{1,3} \\ C_{2,0} & C_{2,1} & C_{2,2} & C_{2,3} \\ C_{3,0} & C_{3,1} & C_{3,2} & C_{3,3} \end{bmatrix}$$

(a) Register blocking for 16 intermediate result generation.



(b) Step-by-step evolution of matrix multiplication.



(c) Outer loops update and target the next tile in the sequence.

Figure 48: Optimised sequential matrix multiplication.

augmented the SIMD-blocked kernel by *transposing* matrix B and packing both operands into four-element words before execution. A helper routine was used to convert B from a column-major access pattern to a row-major one, so that the four 8-bit elements required by each *smaqa* reside in a single, contiguous 32-bit word. The contiguous row-major order enables faster memory fetch. Since all operand data are now perfectly aligned and already resident in SIMD registers, the execution happens without further address calculation or byte extraction. This eliminates the gather-on-the-fly packing overhead of the previous SIMD implementation, yielding an additional speed-up in practice while preserving the 4×4 tiling structure and the same accumulator framework.

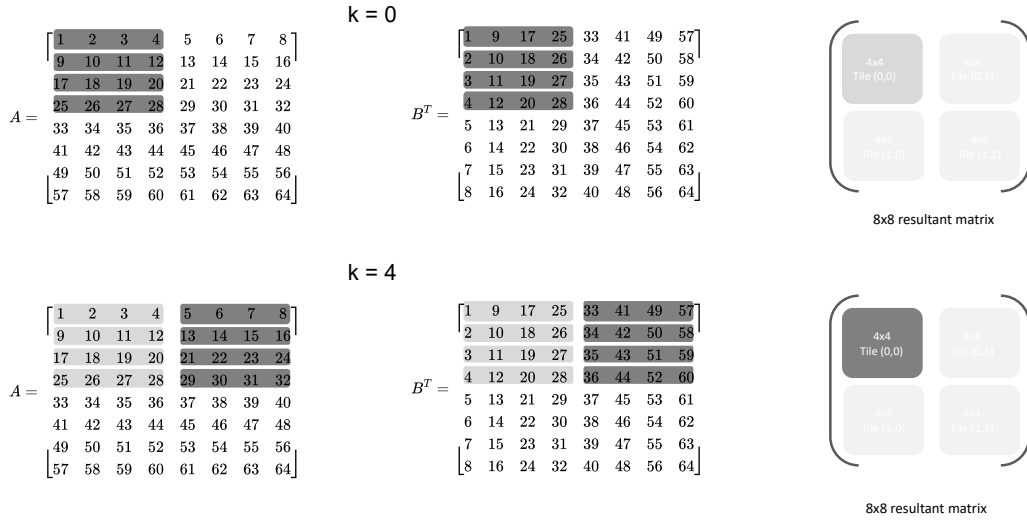


Figure 50: SIMD matrix multiplication with transposed matrix B.

Fig. 51 showcases the performance advantage of the tiled version over regular sequential kernel, using 8×8 matrices. This optimised version served as the baseline for SIMD performance comparison.

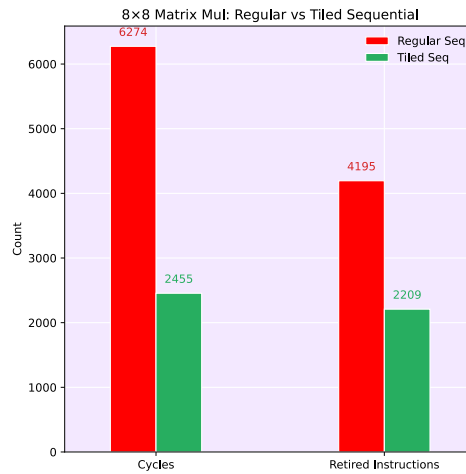


Figure 51: Performance comparison of regular and optimised matrix multiplication.

Fig. 52 presents the cycle speed-up and retired instruction count achieved by three matrix multiplication kernels: scalar, SIMD, and SIMD + transpose. The measurements were recorded across matrix sizes ranging from 8×8 to 64×64 . For the smallest matrix size (8×8), the SIMD variants exhibit a modest speed-up of approximately $1.65\times$, primarily due to the one-time overheads involved in packing the operands. In the case of the transpose variant, this includes the additional cost of reorganizing matrix B , which constitutes a significant fraction of the total computation for such small workloads. However, as the matrix dimensions increase, these fixed costs are amortised, and the benefits of the SIMD P-extension become more pronounced.

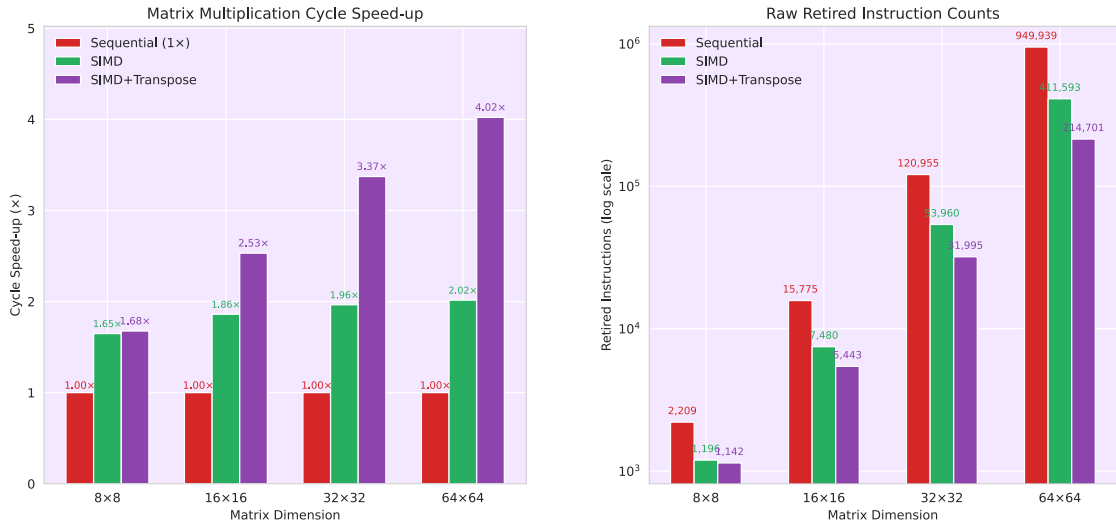


Figure 52: Sequential, SIMD and SIMD+transpose kernel performance in terms of (left) Cycle counts, (right) Retired instructions.

4.3 FIR filter

A Finite Impulse Response (FIR) filter is a fundamental digital signal processing (DSP) algorithm widely used for applications such as noise reduction, signal smoothing. The FIR filter operates by taking a finite sequence of input samples and producing each output sample as a weighted sum of the current and several previous input values. Mathematically, for an input sequence $x[n]$ and filter coefficients $h[k]$, the FIR filter output $y[n]$ is given by

$$y[n] = \sum_{k=0}^{N-1} h[k] \cdot x[n - k], \quad (2)$$

where N is the number of filter taps or coefficients or length of the filter. FIR filters are particularly well-suited for SIMD acceleration because the primary MAC operations across a sliding window of input samples exhibits data-level parallelism.

A simple N -tap finite-impulse-response (FIR) low-pass filter was applied to a synthetically generated signal. The composite input $x[n]$ was synthesised in Python

with $f_{\text{signal}} = 5 \text{ Hz}$, $f_{\text{interference}} = 50 \text{ Hz}$ and with some white noise. The signal was scaled and quantised such that the amplitude spanned the 8-bit range $[-128, 127]$, matching the data size targeted by the packed-SIMD instruction, *smaq*. The objective for the FIR filtering was to recover a low-frequency, 5 Hz target tone that is corrupted by a strong 50 Hz interference component and additive white noise.

Since the highest frequency present is $f_{\text{max}} = 50 \text{ Hz}$, the Nyquist criterion requires $f_s \geq 2f_{\text{max}} = 100 \text{ Hz}$. A sampling frequency of $f_s = 200 \text{ Hz}$ was thus selected. A data set of $N = 200$ samples was chosen which generated a one-second data record. Fig. 53 shows the 200-sample signal trace used as an input to FIR filter.

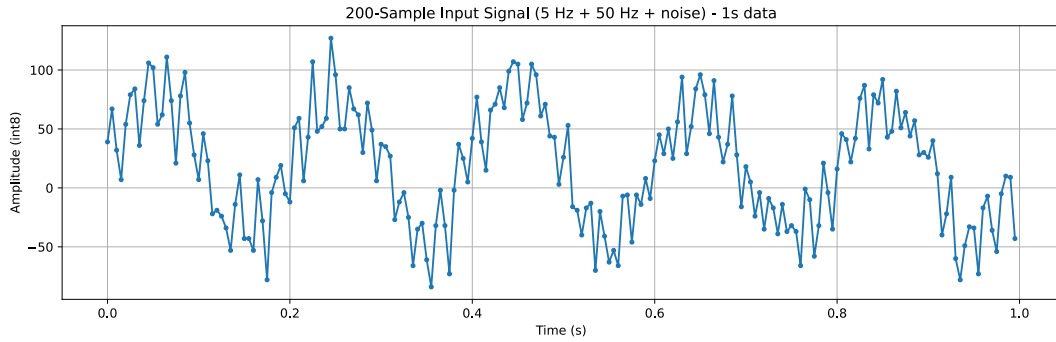


Figure 53: One-second input record for FIR filter.

The next step was to generate the FIR filter coefficients, $h[k]$, which define the impulse response of the low-pass filter. These coefficients were computed using the *firwin()* function from the SciPy signal processing library in Python. The parameters used for this function are listed in Table VII.

Table VII: Parameters for generating the FIR filter coefficients using *firwin()* function.

Parameter	Value
Filter type	Low Pass Filter
Filter length	32
Cutoff frequency	10Hz
Sampling frequency	200Hz
Window function	Hamming

By default, *firwin()* generates filter taps as floating-point values normalized between -1 and 1 . However, the P-extension operates exclusively on fixed-point data, and the *smaq* instruction specifically expects 8-bit signed integer operands. Therefore, each floating-point coefficient had to be scaled and quantised to fit within the 8-bit signed integer range $[-128, 127]$. To accomplish this scaling, three steps were carried out:

1. The maximum tap magnitude was determined: $h_{\text{max}} = 0.1008256896950585$.

2. To maximise precision while remaining in the range $[-128, 127]$, the largest power-of-two scale factor satisfying $2^{\text{power}} \times h_{\text{max}} < 128$ was chosen: $2^{10} = 1024$.
3. Each tap was quantised to Q0.10 fixed-point:

$$h_{\text{int8}}[k] = \text{clip}\left(\text{round}\left(1024 \times h_{\text{float}}[k]\right), [-128, 127]\right), \quad k = 0, \dots, 31.$$

The resulting 32-element vector of signed bytes retains the desired frequency response while allowing the FIR kernel to execute entirely in integer arithmetic. The float FIR coefficients and their equivalent scaled signed 8-bit integer values are shown in the Table VIII.

Table VIII: FIR float coefficients converted to equivalent signed integer values.

Index	Coefficient (float)	Q0.10 Scaled Value (int)
0	-0.0016466950312719214	-2
1	-0.001967447827333216	-2
2	-0.0025003674400324253	-3
3	-0.0029684278771570606	-3
...	...	...
14	0.09570411108306039	98
15	0.1008256896950585	103
...	...	...
31	-0.0016466950312719214	-2

To produce correct results, the final output must be scaled down by the same power-of-two factor. In hardware, this is efficiently implemented using bitwise right shifts. However, direct shifting leads to truncation, which, when accumulated over many operations, introduces systematic downward bias in the result. To mitigate this, rounding to the nearest integer is applied prior to truncation. This is achieved by adding half of the divisor before the shift: if the remainder is greater than or equal to half, the result rounds up; otherwise, it remains the same. This approach ensures unbiased scaling and improves numerical accuracy.

Fig. 54 shows the C implementation of the sequential FIR filter using a two-level SISD loop structure.

```

1 for (int n = FIR_TAPS - 1; n < INPUT_LENGTH; n++) {
2     ...
3     for (int k = 0; k < FIR_TAPS; k++) {
4         ...
5         acc += in[n - k] * fir_coeffs[k];
6         ...
7     }
8     out[n] = (acc + (1 << 9)) >> 10;
9 }

```

Figure 54: FIR filter SISD kernel.

For each valid output sample n (starting at `FIR_TAPS - 1` so that a full tap history is available), the outer loop iterates over the input record, while the inner loop performs a dot product between the most recent M input values and the M fixed-point coefficients. Each loop iteration executes a single multiply-add on 8-bit data, accumulating the result in a 32-bit register. After all taps have been processed, the accumulator is rounded to the nearest and right-shifted by the Q0.10 scale factor before being stored to the output array.

The SIMD FIR kernel again uses the P-extension's four-lane *smaqa* instruction. All 32 coefficients are first pre-packed into eight 32-bit words, each holding four consecutive 8-bit taps, so the inner loop operates on $M/4$ coefficient blocks instead of M individual taps. Each iteration packs four new input samples into a matching 32-bit word, issues a single *smaqa*, which performs four parallel MACs, and advances the tap index by four. Consequently, the inner loop executes only one-quarter as many iterations as the scalar kernel and completes four MACs per instruction rather than one. The rounding and Q0.10 down-scaling step is identical to the SISD path.

Fig. 55 (top) shows the FIR filtered output for one-second data, with the observed delay caused by the filter's group delay. For a linear-phase FIR filter with N taps, this delay equals $(N - 1)/2$ samples. Fig. 55 (bottom) shows the time-aligned signal with the input.

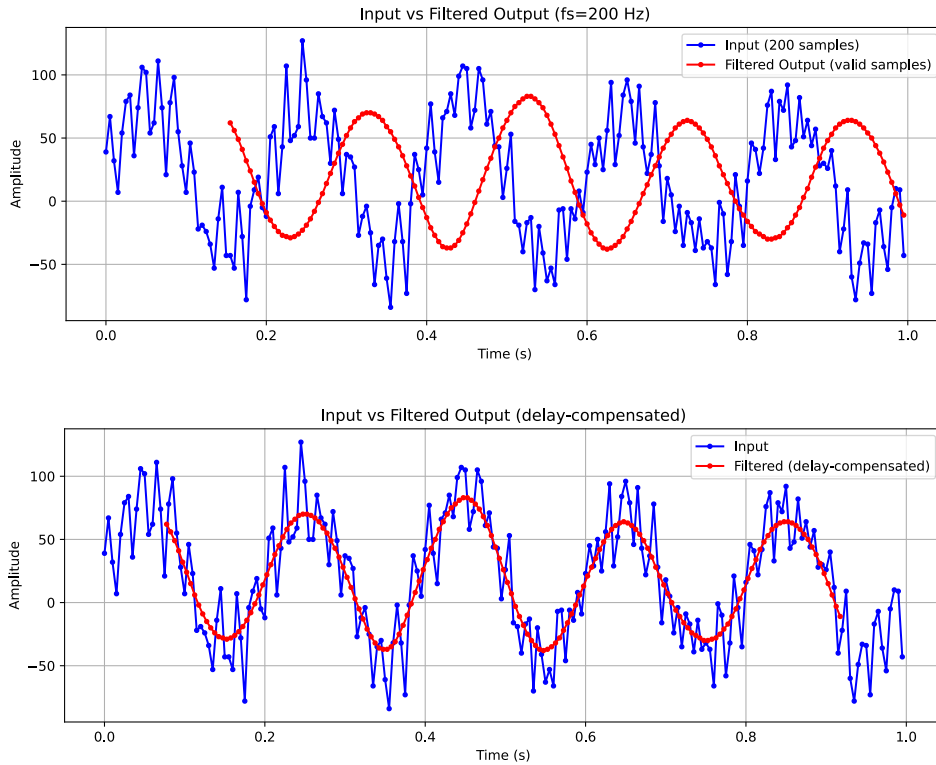


Figure 55: (Top) filtered FIR output, (bottom) time-shifted FIR output.

Fig. 56 illustrates the performance comparison between SISD and SIMD implementations of a 32-tap FIR filter for processing 200 input samples, sampled at 200 Hz.

The SIMD kernel achieves a 53% reduction in CPU cycle count and a 44% reduction

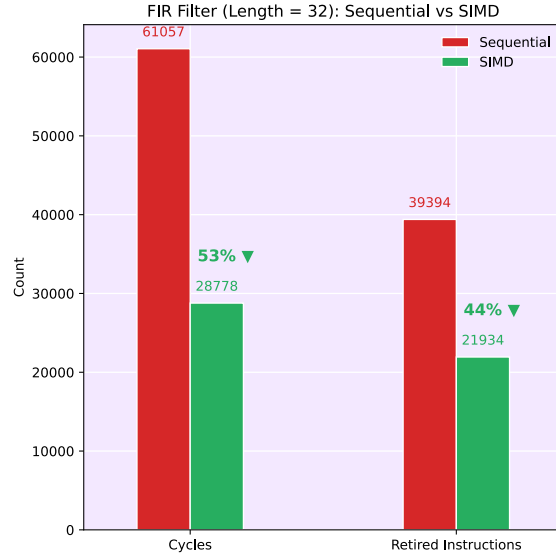


Figure 56: SISD and SIMD performance comparison for a 32-tap FIR filter processing 200 samples.

in retired instructions compared to the scalar baseline, demonstrating significant throughput gains in a realistic low-pass filtering scenario.

4.4 Sobel Edge Detection algorithm

The Sobel edge detection algorithm is a technique in image processing and computer vision for identifying the boundaries or edges within an image. It operates by performing a 2D spatial gradient measurement. It uses two fixed 3×3 convolution kernels. Fig. 57 shows these two kernels: one is used to estimate the gradient in the horizontal (x) direction, and the another is for estimating the gradient in the vertical (y) direction.

$$k_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}, \quad k_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

Horizontal
 Vertical

Figure 57: Sobel kernels.

The algorithm convolves these kernels with the input image to compute the gradient components at each pixel: G_x (horizontal change) and G_y (vertical change), Fig. 58 shows the convolution step in the Sobel operator for computing the gradients. The magnitude of the gradient at each pixel, which indicates the strength of the

14	51	46	201	172	183
21	67	3	194	164	191
12	35	38	196	175	229
39	25	23	152	156	216
39	10	62	167	144	167
69	35	20	162	186	219

 $\star$

-1	0	1
-2	0	2
-1	0	1

 $= G_x \text{ (Horizontal gradient)}$
 k_x

14	51	46	201	172	183
21	67	3	194	164	191
12	35	38	196	175	229
39	25	23	152	156	216
39	10	62	167	144	167
69	35	20	162	186	219

 $\star$

1	2	1
0	0	0
-1	-2	-1

 $= G_y \text{ (Vertical gradient)}$
 k_y

Figure 58: Gradient calculation in Sobel algorithm.

edge, is then traditionally computed as

$$G = \sqrt{G_x^2 + G_y^2}. \quad (3)$$

However, typically in embedded environments, a simpler approximation $G = |G_x| + |G_y|$ is often used for computational efficiency. This thesis adopts this approximation for gradient calculation. Firstly, it reduces the computational overhead of square root and square operation. Secondly, it makes the implementation more complaint with P-Extension instructions such as SIMD 16-bit absolute instruction (`pabsh`). After calculating the gradient magnitude for all pixels, the algorithm typically applies a clipping threshold to cap values that exceed a limit.

Each iteration in both implementations begins by computing three row-base pointers, r_0 (row $y-1$), r_1 (row y), and r_2 (row $y+1$). Fig. 59 shows the row-base pointers used for first pixel row of the image. These pointers are reused as the inner loop slides across the scan line, thereby avoiding repeated address arithmetic. Moreover for simplicity, boundary pixels were excluded from both SISD and SIMD implementations to eliminate the need for zero-padding the image edges.

y-1	14	51	46	201	172	183
y	21	67	3	194	164	191
y+1	12	35	38	196	175	229
	39	25	23	152	156	216
	39	10	62	167	144	167
	69	35	20	162	186	219

Figure 59: First scan line at the edge of an image.

The scalar Sobel kernel, shown in the Listing 1 of Appendix A, evaluates the horizontal and vertical gradients by performing nine signed MACs per pixel followed by two absolute-value operations and one addition. In total the pixel core requires

roughly 18 integer ALU operations and revisits memory nine times for the same three rows.

The SIMD Sobel implementation, shown in the Listing 2 of Appendix A, pre-packs the signed kernel coefficients into four-lane 32-bit registers, with the fourth lane explicitly set to zero. For each pixel, three consecutive 32-bit words, each containing four neighbouring unsigned 8-bit greyscale pixels, are loaded. These pixel values are multiplied with the packed signed 8-bit kernel coefficients using the four-lane *smaqasu* MAC instructions, which operates on signed and unsigned 8-bit elements. The 16-bit absolute (*pabsh*) instruction computes lane-wise absolute values, and 16-bit Cross Subtraction & Addition (*psahx*) performs horizontal addition of the 16-bit gradient components from the horizontal and vertical directions. The 16-bit unsigned clip (*pclipuh*) instruction then clips the result to 256, the maximum greyscale pixel value. All P-extension instructions are invoked via inline assembly wrapped in helper functions. Finally, the packed data is right-shifted by one byte to slide the 3×3 convolution window, loading only one new pixel per iteration. This approach halves the number of inner-loop iterations and reduces per-pixel ALU operations to approximately nine.

Applying the Sobel filter to a 48×48 pixel test image shows a modest but measurable benefit from the P-extension. The plot in Fig. 60 shows the SIMD Sobel kernel recording a 19% reduction in CPU cycles and a 15% decrease in retired instructions. Despite the overheads stemming from packing and loop control, the throughput gains

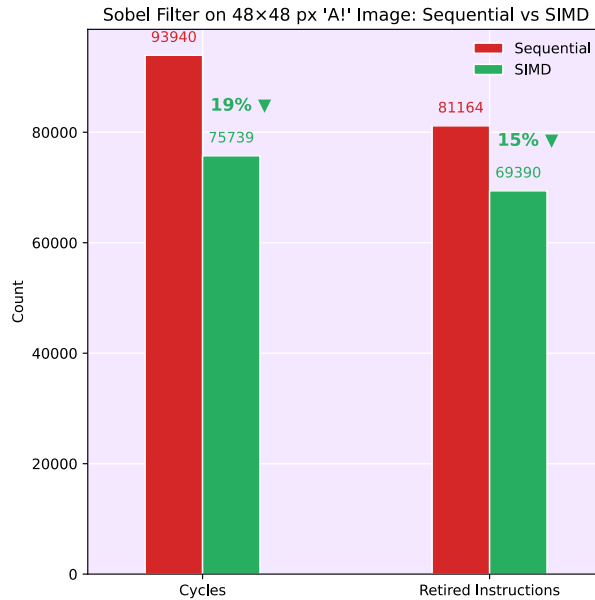


Figure 60: Performance comparison between SISR and SIMD kernels implementing Sobel Edge Detection algorithm.

originate from the combined contribution of P-Extension instructions: four-lane MAC operations, combined with two-lane absolute, horizontal addition, and clipping instructions. Fig. 61 shows the original greyscale 48×48 pixel “A!” glyph (left),

while the right pane shows the corresponding edge map produced by SIMD Sobel kernel. The output image confirms functional correctness of the implementation.

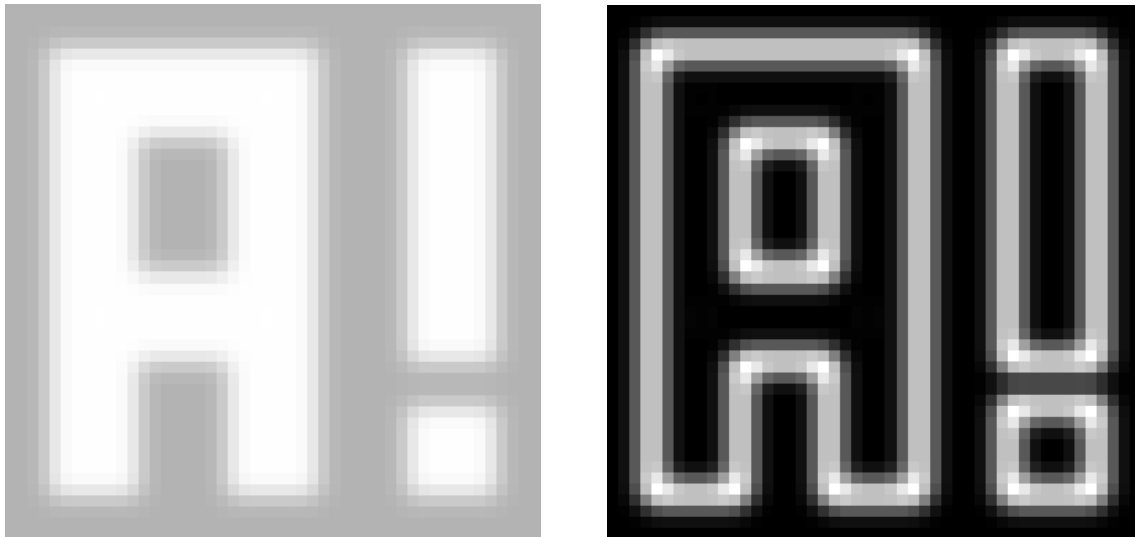


Figure 61: Original 48×48 pixel image (left) and its Sobel output(right).

5 Conclusion

This thesis aimed to implement the draft version of the RISC-V Packed-SIMD (P) extension on the A-Core processor, thereby equipping A-Core with efficient data-parallel processing capabilities. The work was structured into four key phases: (i) micro-architectural design of the P-extension ALU, (ii) integration of the new SIMD ALU and the associated control logic into A-Core’s processor pipeline, (iii) extension of the software toolchain to support the P-extension, and (iv) application-level benchmarking to demonstrate that meaningful DSP acceleration can be achieved using the P-extension.

A-Core’s arithmetic logic unit was extended to support parallel 8-bit and 16-bit operations, in accordance with the RISC-V P-extension specification. The new P-Extension ALU was implemented in Chisel, a hardware construction language embedded in Scala. The design followed the conventional ALU structure, consisting of a large multiplexer that selects among various processing units based on the decoded P-extension instruction. A key design goal was hardware reuse to optimise area efficiency. To achieve this, the architecture was centered around a single 4×9 -bit adder unit, which was reused to implement a range of operations including addition, subtraction, comparison, minimum/maximum, and absolute value computations.

Throughout the project, rigorous verification steps were undertaken to ensure the correctness and reliability of the extended design. These tests were carried out at the module-level during the development of the P-Extension in Chisel, and at the full system-level after the compiler support was enabled. At the module level, the new SIMD ALU components were tested using ChiselTest, leveraging self-checking unit tests to validate arithmetic results (including edge cases like overflow and saturation behaviour) against expected outcomes. These unit tests confirmed that the 8-bit and 16-bit subword operations produce correct results across a range of input patterns.

Once module-level testing was completed, the internal architecture of the A-Core processor was studied in detail, and the newly developed P-Extension ALU was integrated into the existing pipeline. This integration required modifications across multiple pipeline components, including the Decoder, ALU, Register File, CSR path, Pipeline Controller, the top-level ACoreBase, and the shared Scala utility library ACoreCommon.

The next objective was to develop practical C programs that could leverage the advantages of the P-Extension. However, since existing compilers do not yet support the P-Extension, the software toolchain had to be extended accordingly such that P-Extension instructions can be used while writing C programs. Specifically, the binutils assembler and disassembler were patched to recognise the custom P-Extension instruction mnemonics. This modification enabled the use of the new P-Extension instructions into C programs through the use of inline assembly.

Once the compiler was successfully built, system-level testing was conducted by developing a self-checking C test harness and executing it on the modified A-Core. This aimed to validate the end-to-end integration of the P-extension. The test programs exercised various P-extension instructions in sequence, using the same set

of random and corner cases previously applied during module-level testing with Chiseltest. Each instruction was tested by computing reference results and automatically comparing them against the outputs produced during program execution, thereby verifying functional correctness across the entire hardware–software stack.

Finally, the extended core’s performance was evaluated on a set of representative benchmark applications. Four classic DSP kernels were implemented in two versions (with and without P-extension) to characterize the benefits: (i) a vector dot product, (ii) a blocked matrix multiplication, (iii) a 32-tap finite impulse response (FIR) filter, and (iv) a Sobel edge detection algorithm. Across all these benchmarks, the P-extension enabled significant reductions in CPU cycle counts and the number of instructions retired.

The SIMD-accelerated dot product achieved nearly the theoretical four-fold speed-up on large vectors. In the matrix multiplication test, the benefits of the P-extension grew with problem size. For a small 8×8 matrix, the SIMD version showed a modest $1.6\times$ speed-up. This modest speed-up was due to one-time overheads for packing the data into SIMD registers, which becomes dominant for small size matrix. However, for larger matrices the parallel execution amortised these overheads and approached speed-up of $4\times$. The fixed-point FIR filter saw a notable performance gain wherein the total cycle count reduced by about 53% compared to the scalar implementation. Lastly, the SIMD-enhanced Sobel kernel ran about 19% faster in cycles and retired 15% fewer instructions than its scalar counterpart on a 48×48 -pixel test image. This modest speed-up for Sobel reflects the overhead of data packing and limited parallelism per pixel, yet it still demonstrates that the P-extension can accelerate real-world algorithms that involve mixed computation and memory access.

Looking forward, there are several directions for future work to build upon the achievements of this thesis. An immediate next step is to evaluate the hardware costs associated with the P-extension integration. While this thesis focused on functionality and performance, a detailed analysis of the area overhead introduced by the additional SIMD hardware would be valuable. Moreover, the possible affects on the top operating frequency due to the additional ALU logic and extra control paths can also be assessed. Additionally, future work could explore implementing a broader subset of the P-extension instructions and further integrating compiler support (e.g. developing high-level intrinsics) to make it easier for application developers to leverage the new instructions.

In conclusion, this thesis successfully implemented the RISC-V P-extension on the A-Core processor and demonstrated the practical benefits of packed SIMD functionality through real-world DSP applications. The work lays a strong foundation for future enhancements and optimisations of the RISC-V A-Core processor at Aalto University.

References

- [1] E. Cui, T. Li, and Q. Wei, “Risc-v instruction set architecture extensions: A survey,” *IEEE Access*, vol. 11, pp. 24 696–24 711, 2023.
- [2] K. Asanović and D. A. Patterson, “Instruction sets should be free: The case for risc-v,” Tech. Rep. UCB/EECS-2014-146, Aug 2014. [Online]. Available: <http://www2.eecs.berkeley.edu/Pubs/TechRpts/2014/EECS-2014-146.html>
- [3] RISC-V International, “History of risc-v,” <https://riscv.org/about/history/>, 2024, accessed: 2024-09-04.
- [4] SNS Insider, “RISC-V Tech Market to Surpass USD 11.50 Billion by 2032 at 29.66
- [5] Wevolver Staff, “2024 State of Edge AI Report,” Online report, Apr. 2024, sponsored by EDGE AI Foundation, Renesas, Synaptics, Arduino Pro, Imagimob, Eta Compute, Sony, Qualcomm. [Online]. Available: <https://www.wevolver.com/article/2024-state-of-edge-ai-report>
- [6] W. Shi and S. Dustdar, “The promise of edge computing,” *Computer*, vol. 49, no. 5, pp. 78–81, 2016.
- [7] F. Duarte, “Number of iot devices (2024),” Exploding Topics blog, Feb. 2024, estimated approximately 17.08 billion connected IoT devices in 2024. [Online]. Available: <https://explodingtopics.com/blog/number-of-iot-devices>
- [8] R.-V. International, “Risc-v p extension proposal,” 2024. [Online]. Available: <https://github.com/riscv/riscv-p-spec/blob/master/P-ext-proposal.adoc>
- [9] Andes Technology, “Andes Technology Unveils the AndesCore™ D23 – A Feature-Rich, Low-Power and Highly Secured Entry-Level RISC-V Processor,” December 2022. [Online]. Available: <https://www.andestech.com/en/2022/12/08/andes-technology-unveils-the-andescore-d23-a-feature-rich-low-power-and-highly-secured-entry-level-risc-v-processor/>
- [10] D. A. M. Koene, “Implementation and evaluation of packed-simd instructions for a risc-v processor,” Master’s thesis, Delft University of Technology, Aug. 2021, master’s thesis; Supervisors: Stephan Wong. [Online]. Available: <https://resolver.tudelft.nl/uuid:c4162ff8-9419-4434-852d-c1c3297df808>
- [11] F. Lastname, “Implementation and Evaluation of the RISC-V P-extension on Ibex,” Master’s thesis, Norwegian University of Science and Technology, Trondheim, Norway, 2024. [Online]. Available: <https://ntnuopen.ntnu.no/ntnu-xmlui/handle/11250/3153476>
- [12] A. University, “A-core gitlab repository,” <https://gitlab.com/a-core>, 2025.
- [13] C. Hamacher, Z. Vranesic, S. Zaky, and N. Manjikian, *Computer organization and embedded systems*. McGraw-Hill Higher Education, 2011.

- [14] S. R. Sarangi, *Basic Computer Architecture*, 1st ed. White Falcon Publishing.
- [15] Arm Ltd. (2024) What is an instruction set architecture (isa)? Accessed: 2024-11-01. [Online]. Available: <https://www.arm.com/glossary/isa>
- [16] C. Dragos, A. Chirita, and B. Neleş-Constantin, “Understanding the differences between arm and x86 isa’s,” 11 2021.
- [17] Lenovo. (2024) Instruction set architecture (isa) glossary. Accessed: 2024-11-01. [Online]. Available: <https://www.lenovo.com/us/outletus/en/glossary/instruction-set-architecture/>
- [18] A. Waterman, “Design of the risc-v instruction set architecture,” 2016.
- [19] D. Patterson and J. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*, ser. ISSN. Elsevier Science, 2020. [Online]. Available: <https://books.google.co.in/books?id=e8DvDwAAQBAJ>
- [20] Z. Ali, T. Tanveer, S. Aziz, M. Usman, and A. Azam, “Reassessing the performance of arm vs x86 with recent technological shift of apple,” in *2022 International Conference on IT and Industrial Technologies (ICIT)*, 2022, pp. 01–06.
- [21] S. Mangalwedhe, R. Kulkarni, and S. Y. Kulkarni, “Low power implementation of 32-bit risc processor with pipelining,” in *Proceeding of the Second International Conference on Microelectronics, Computing & Communication Systems (MCCS 2017)*, V. Nath and J. K. Mandal, Eds. Singapore: Springer Singapore, 2019, pp. 307–320.
- [22] R. Núñez-Prieto, D. Castells-Rufas, and L. Terés-Terés, “Risco2: Implementation and performance evaluation of risc-v processors for low-power co2 concentration sensing,” *Micromachines*, vol. 14, no. 7, 2023. [Online]. Available: <https://www.mdpi.com/2072-666X/14/7/1371>
- [23] Y. He and X. Chen, “Survey and comparison of pipeline of some risc and cisc system architectures,” in *2023 8th International Conference on Computer and Communication Systems (ICCCS)*, 2023, pp. 785–790.
- [24] A. Holdings, *Arm Architecture Reference Manual for A-profile Architecture*, <https://developer.arm.com/documentation/ddi0487/latest/>, 2023, accessed: 2024-11-02. [Online]. Available: <https://developer.arm.com/documentation/ddi0487/latest/>
- [25] U. o. E. EPCC, “The evolution of risc architecture,” <https://www.epcc.ed.ac.uk/whats-happening/articles/evolution-risc-architecture>, 2023, accessed: 2024-11-02. [Online]. Available: <https://www.epcc.ed.ac.uk/whats-happening/articles/evolution-risc-architecture>

- [26] E. Blem, J. Menon, and K. Sankaralingam, “Power struggles: Revisiting the risc vs. cisc debate on contemporary arm and x86 architectures,” in *2013 IEEE 19th International Symposium on High Performance Computer Architecture (HPCA)*, 2013, pp. 1–12.
- [27] R.-V. International, “14 years of risc-v: A journey of innovation and firsts,” <https://riscv.org/blog/2024/05/14-years-of-risc-v-a-journey-of-innovation-and-firsts/>, 2024, accessed: 2024-11-02. [Online]. Available: <https://riscv.org/blog/2024/05/14-years-of-risc-v-a-journey-of-innovation-and-firsts/>
- [28] A. Waterman, Y. Lee, D. A. Patterson, and K. Asanovic, “The risc-v instruction set manual, volume i: Base user-level isa,” *EECS Department, UC Berkeley, Tech. Rep. UCB/EECS-2011-62*, vol. 116, pp. 1–32, 2011.
- [29] F. Zaruba and L. Benini, “The cost of application-class processing: Energy and performance analysis of a linux-ready 1.7-ghz 64-bit risc-v core in 22-nm fdsoi technology,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 27, no. 11, pp. 2629–2640, 2019.
- [30] A. Waterman, K. Asanović, and J. Hauser, *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Document Version 20211203*, RISC-V International, 2021. [Online]. Available: <https://five-embeddev.com/riscv-priv-isa-manual/Priv-v1.12/riscv-privileged.html>
- [31] D. Patterson and A. Waterman, *The RISC-V Reader: an open architecture Atlas*. Strawberry Canyon, 2017.
- [32] J. L. Hennessy and D. A. Patterson, *Computer Architecture, Sixth Edition: A Quantitative Approach*, 6th ed. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2017.
- [33] M. J. Flynn, “Some computer organizations and their effectiveness,” *IEEE Transactions on Computers*, vol. C-21, no. 9, pp. 948–960, 1972.
- [34] M. E. A. Ibrahim, M. Rupp, and H. A. H. Fahmy, “Code transformations and simd impact on embedded software energy/power consumption,” in *2009 International Conference on Computer Engineering Systems*, 2009, pp. 27–32.
- [35] G. Mitra, B. Johnston, A. P. Rendell, E. McCreath, and J. Zhou, “Use of simd vector operations to accelerate application code performance on low-powered arm and intel platforms,” in *2013 IEEE International Symposium on Parallel Distributed Processing, Workshops and Phd Forum*, 2013, pp. 1107–1116.
- [36] R.-V. Foundation, “Risc-v ratified specifications,” <https://riscv.org/specifications/ratified/>.
- [37] R.-V. P. vs. V Extension Discussion, “P extension maturity (p vs. v),” https://lists.riscv.org/g/tech-p-ext/topic/p_extension_maturity_p_vs_v/80373046.

- [38] O. Simola, “Physical implementation of a risc-v processor,” Master’s thesis, Aalto University, School of Electrical Engineering, Espoo, Finland, Mar. 2023. [Online]. Available: <https://aaltodoc.aalto.fi/items/efe24aaf-5415-4d45-b850-4c39510b9751>
- [39] A-Core Project Contributors, “Acorechip: Risc-v based a-core chisel implementation (pextalu branch),” https://gitlab.com/a-core/a-core_chisel/ACoreChip/-/tree/dev/pextalu, 2025.
- [40] Y. Lee, A. Waterman, H. Cook, B. Zimmer, B. Keller, A. Puggelli, J. Kwak, R. Jevtic, S. Bailey, M. Blagojevic, P.-F. Chiu, R. Avizienis, B. Richards, J. Bachrach, D. Patterson, E. Alon, B. Nikolic, and K. Asanovic, “An agile approach to building risc-v microprocessors,” *IEEE Micro*, vol. 36, no. 2, pp. 8–20, 2016.
- [41] K. Beck, M. Beedle, A. Van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries *et al.*, “Manifesto for agile software development,” 2001.
- [42] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avizienis, J. Wawrzynek, and K. Asanović, “Chisel: Constructing hardware in a scala embedded language,” in *DAC Design Automation Conference 2012*, 2012, pp. 1212–1221.
- [43] L. Pezzarossa and M. Schoeberl, “Transitioning to chisel in university education: Experiences and lessons learned,” in *2023 IEEE Nordic Circuits and Systems Conference (NorCAS)*, 2023, pp. 1–7.
- [44] J. Bachrach, H. D. Vo, B. C. Richards, Y. Lee, A. Waterman, R. Avizienis, J. Wawrzynek, and K. Asanović, “Chisel: Constructing hardware in a scala embedded language,” *DAC Design Automation Conference 2012*, pp. 1212–1221, 2012. [Online]. Available: <https://api.semanticscholar.org/CorpusID:1501345>
- [45] M. Schoeberl, *Digital Design with Chisel*. Kindle Direct Publishing, 2019. [Online]. Available: <https://github.com/schoeberl/chisel-book>
- [46] A. Kamkin, M. Chupilko, M. Lebedev, S. Smolov, and G. Gaydadjiev, “High-level synthesis versus hardware construction,” in *2023 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2023, pp. 1–6.
- [47] M. Schoeberl, H. J. Damsgaard, L. Pezzarossa, O. Keszocze, and E. R. Jellum, “Hardware generators with chisel,” in *2024 27th Euromicro Conference on Digital System Design (DSD)*, 2024, pp. 168–175.
- [48] J. Im and S. Kang, “Comparative analysis between verilog and chisel in risc-v core design and verification,” in *2021 18th International SoC Design Conference (ISOCC)*, 2021, pp. 59–60.

- [49] C. Developers, “Chiseltest: A testing framework for chisel hardware designs,” <https://github.com/ucb-bar/chiseltest>, 2025.
- [50] A. Dobis, K. Laeuffer, H. J. Damsgaard, T. Petersen, K. J. H. Rasmussen, E. Tolotto, S. T. Andersen, R. Lin, and M. Schoeberl, “Verification of chisel hardware designs with chiselverify,” *Microprocessors and Microsystems*, vol. 96, p. 104737, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0141933122002666>
- [51] A. Izraelevitz, J. Koenig, P. Li, R. Lin, A. Wang, A. Magyar, D. Kim, C. Schmidt, C. Markley, J. Lawson, and J. Bachrach, “Reusability is firrtl ground: Hardware construction languages, compiler frameworks, and transformations,” in *2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2017, pp. 209–216.
- [52] Andes Technology, “AndesCore™ D23 – Entry-Level RISC-V Core with Secure and Power-Efficient Features,” <https://www.andestech.com/en/products-solutions/andescore-processors/riscv-d23/>, 2022.
- [53] H. E. GmbH, “HighTec C/C++ Compiler Suite Supports Andes’ ISO 26262 Certified RISC-V IP for Automotive Safety and Security Applications,” https://www.presseagentur.com/hightec/detail.php?pr_id=7200&lang=en, May 2025.
- [54] Andes Technology Corporation, “AndesCore D45: 32-bit superscalar risc-v cpu core with dsp/simd extensions,” <https://www.andestech.com/en/products-solutions/andescore-processors/riscv-d45/>, 2025.
- [55] HPMicro Semiconductor and Andes Technology, “Hpm6000 series of real-time risc-v microcontrollers with dual andescore d45,” Online press release, Dec. 2021, feature: up to 800MHz, dual-core D45, local memory + caches, DSP/SIMD (P-extension). [Online]. Available: <https://www.andestech.com/en/2021/12/02/hpmicro-semiconductor-announces-the-release-of-the-hpm6000-series-of-microcontrollers-with-an>
- [56] M. Gautschi, P. D. Schiavone, A. Traber, I. Loi, A. Pullini, D. Rossi, E. Flamand, F. K. Gürkaynak, and L. Benini, “Near-threshold risc-v core with dsp extensions for scalable iot endpoint devices,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 25, no. 10, pp. 2700–2713, 2017.
- [57] V. Martinoli, Y. Teglia, A. Bouagoun, and R. Leveugle, “Cva6’s data cache: Structure and behavior,” 2022. [Online]. Available: <https://arxiv.org/abs/2202.03749>
- [58] O. Group, “Cva6 requirements specification,” https://docs.openhwgroup.org/projects/cva6-user-manual/02_cva6_requirements/cva6_requirements_specification.html, 2025.
- [59] —, “Cva6 risc-v out-of-order core,” <https://github.com/openhwgroup/cva6>, 2025.

- [60] P. P. Chu, *RTL Hardware Design Using VHDL: Coding for Efficiency, Portability, and Scalability*. Wiley-IEEE Press, 2006.
- [61] K. Saso and Y. Hara-Azumi, “Revisiting simple and energy efficient embedded processor designs toward the edge computing,” *IEEE Embedded Systems Letters*, vol. 12, no. 2, pp. 45–49, 2020.
- [62] A. S. Nery, L. Jóźwiak, M. Lindwer, M. Cocco, N. Nédjah, and F. M. França, “Hardware reuse in modern application-specific processors and accelerators,” *Microprocessors and Microsystems*, vol. 37, no. 6, pp. 684–692, 2013. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S014193311200097X>
- [63] A.-C. D. Team, “A-core chisel guide,” https://gitlab.com/a-core/a-core_documentation/tutorials/-/blob/master/a-core_chisel_guide.md, 2024.
- [64] A-Core Contributors, “ACoreBase Chisel Source Code – dev/pextalu Branch,” https://gitlab.com/a-core/a-core_chisel/ACoreBase/-/tree/dev/pextalu/src/main/scala/acorebase?ref_type=heads, 2025.
- [65] H. Sandid, “Adding custom instructions to the risc-v isa,” <https://hsandid.github.io/posts/risc-v-custom-instruction/>, 2023.
- [66] Reddit User: u/Starryspectrum, “How to extend risc-v p extension in riscv-gcc and riscv-binutils,” https://www.reddit.com/r/RISCV/comments/xertz5/how_to_extend_riscv_p_extension_in_riscvgcc_and/, 2022.
- [67] riscv-collab contributors, “Issue #1095: Support for custom p-extension instructions in riscv-gnu-toolchain,” <https://github.com/riscv-collab/riscv-gnu-toolchain/issues/1095>, 2023.
- [68] H. Sultan, “Modified binutils-gdb with p-extension support – hrsp/p-extension branch,” <https://github.com/iamhsultan/binutils-gdb-fork/tree/hrsp/p-extension>, 2025.
- [69] A-Core Contributors, “P-Extension Test Suite (pexttest) – pextalu Branch,” https://gitlab.com/a-core/a-core_software/pexttest/-/tree/pextalu?ref_type=heads, 2025.
- [70] A-Core Software Contributors, “test_matmul.c — matrix-multiplication test program,” Source code, A-Core Software `riscv-c-tests` repository, 2025. [Online]. Available: https://gitlab.com/a-core/a-core_software/riscv-c-tests/-/blob/master/src/matmul/test_matmul.c

Appendix A

```

/* 3x3 Sobel kernels */
const int8_t KX0[3] = { -1, 0, +1 };
const int8_t KX1[3] = { -2, 0, +2 };
const int8_t KX2[3] = { -1, 0, +1 };

const int8_t KY0[3] = { +1, +2, +1 };
//const int8_t KY1[3] = { 0, 0, 0 };
const int8_t KY2[3] = { -1, -2, -1 };

for (int y = 1; y < height - 1; ++y) {
    const uint8_t *r0 = src + (y - 1) * width;
    const uint8_t *r1 = src + y * width;
    const uint8_t *r2 = src + (y + 1) * width;
    uint8_t *d = dst + y * width;

    for (int x = 1; x < width - 1; ++x) {
        /* horizontal gradient */
        int gx =
            KX0[0]*r0[x-1] + KX0[1]*r0[x] + KX0[2]*r0[x+1] +
            KX1[0]*r1[x-1] /*+ KX1[1]*r1[x]*/ + KX1[2]*r1[x+1] +
            KX2[0]*r2[x-1] + KX2[1]*r2[x] + KX2[2]*r2[x+1];

        /* vertical gradient */
        int gy =
            KY0[0]*r0[x-1] + KY0[1]*r0[x] + KY0[2]*r0[x+1] +
            KY2[0]*r2[x-1] + KY2[1]*r2[x] + KY2[2]*r2[x+1];

        /* magnitude and clamp */
        int mag = abs_int(gx) + abs_int(gy);
        d[x] = (uint8_t)(mag > 255 ? 255 : mag);
    }
}

```

Listing 1: SISD implementation of Sobel edge detection algorithm in C.

```

const int32_t KX0 = 0x000100FF; // [0, +1, 0, -1] in little-endian lanes
const int32_t KX1 = 0x000200FE; // [0, +2, 0, -2]
const int32_t KX2 = 0x000100FF; // [0, +1, 0, -1]

const int32_t KY0 = 0x00010201;
const int32_t KY2 = 0x00FFFEFF;

for (int y = 1; y < height - 1; ++y) {
    const uint8_t *r0 = src + (y - 1)*width;
    const uint8_t *r1 = src + y * width;
    const uint8_t *r2 = src + (y + 1)*width;
    uint8_t *d = dst + y * width;

    // Pack the registers with pixel data
    uint32_t W0 = load32u(r0);
    uint32_t W1 = load32u(r1);
    uint32_t W2 = load32u(r2);

    for (int x = 1; x < width - 1; ++x) {
        // SIMD sobel arithmetic
        int32_t gx = 0, gy = 0;
        gx = smaquisu(gx, KX0, W0);
        gx = smaquisu(gx, KX1, W1);
        gx = smaquisu(gx, KX2, W2);

        gy = smaquisu(gy, KY0, W0);
        gy = smaquisu(gy, KY2, W2);

        int32_t gxy = (gy << 16) | (uint16_t)gx;
        gxy = pabsh(gxy);
        gxy = psahx(gxy);
        gxy = pclipuh(gxy, 8);
        d[x] = (uint8_t)gxy;

        // Prepare the registers for next iteration.
        // Slide each window right by one byte and pull in the new pixel
        W0 = (W0 >> 8) | ((uint32_t)r0[x+3] << 24);
        W1 = (W1 >> 8) | ((uint32_t)r1[x+3] << 24);
        W2 = (W2 >> 8) | ((uint32_t)r2[x+3] << 24);
    }
}

```

Listing 2: SIMD implementation of Sobel edge detection algorithm in C using P-Extension instructions.